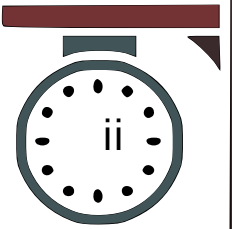




Listen to the whispers web timing attacks that actually works

2nd Edition

by Zarvan and Chaos



اطلاعات و چکیده مقاله

این کتاب به منظور راهنمای جامع ای برای حملات زمان‌بندی در امنیت وب نوشته شده است در ویراستار دوم سعی بر افزایش خوانایی و فرم دهی صفحه ها شده است.

نویسنده : zarvanox01 

ویراستار : Chaos 

تاریخ انتشار : 2026/4/24 

 درباره مقاله : در دنیای امنیت سایبری، زمان همیشه یک فاکتور حیاتی است؛ چه برای مهاجم و چه برای مدافع. این مقاله به بررسی عمیق حملات زمان‌بندی (Timing Attacks) در وب می‌پردازد و تکنیک‌هایی مانند افزایش سیگنال، کاهش نویز، و بهره‌گیری از دوقلوهای باگ (Bug Twins یعنی دو Bug که رفتار مشابهی دارند و یکی از آنها قابل مشاهده و دیگری Blind است) را برای کشف و اکسپلویت کردن باگ‌های Blind شرح می‌دهد. اگر به تست نفوذ حرفه‌ای علاقه‌مندید و می‌خواهید امنیت برنامه‌های وب را به سطح بالاتری برسانید، این مقاله را از دست ندهید.

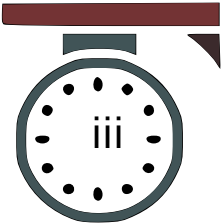
کانال نویسنده : web_articles 

کانال ویراستار : Chaos_nexus_tech 

Praise for Living in Flow

369

Listen to the whispers web timing attacks that actually works



سخن نویسنده :

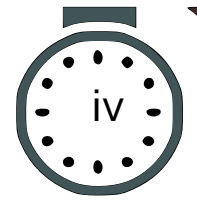
در دل سکوت و تاریکی، جایی که داده‌ها بی صدا می‌لغزند و زمان در
واحد‌های ناپیدا معنا پیدا می‌کند، تنها آن کس پیروز است که نگاهش فراتر
از سطح باشد. نه صدا، نه نشانه؛ فقط ذره‌ای تفاوت در تپش‌های زمان. **این**
نبرد دانش است با جهل، دقت با شتاب، و هکر با ساختار. آن که بلد باشد
سکوت را بخواند، می‌تواند جهان را بازنویسی کند.

سخن ویراستار :

سلام و درود خدمت همگانی خوانندگان .

آنچه که مسئله اصلی من برای ورود به ویراستاری مقاله کرد؛ این چنین بود که خوانایی چنین مطالبی
خوبی که توسط دوست خوبم zarvan نوشته شده است را بالا ببرم و کیفیت محتوای موجود در
سطح جامعه امنیت سایبری ایران را یک قدم رو به جلو ببرم.

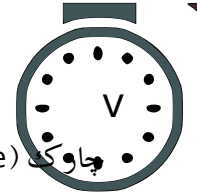
امید است در آینده بتوانیم در ویراستارهای بعدی مثال‌های عملی بیشتر، توضیحات مرحله به مرحله
بهتر و درنهایت پل‌ای ایجاد کنیم تا انتقال این مطالب هم با حالتی باشد که خواننده بتواند به راحتی
متن را درک و در دنیای واقعی از آن استفاده کند .



فهرست مطالب

Contents

2	 Scoped SSRF و SSRF چیست؟
2	 Reverse Proxy چیست؟
3	 فرانت اند و بک اند چیست؟
3	 حملات زمان بندی چیست؟
3	 مقایسه و اینفو گرافیک
4	 سیگنال و نویز یعنی چی؟
4	 هدرهای سفارشی چیست؟
4	 تونل کردن درخواست HTTP چیست؟
5	 HTTP Request Smuggling چیست؟
5	 توابع زمان ثابت (Constant-Time Functions) چیست؟
5	 مفهوم زنجیره کردن (Chaining) در حملات چیست؟
6	 DNS و DNS Caching چیست؟
6	 ابزارهای Param Miner و Turbo Intruder چیست؟
7	 Pingback چیست؟
7	 DNS Label Length چیست؟
7	 Timing Oracle چیست؟
8	 Jitter چیست؟
8	 مفهوم قفل کردن و دسترسی همزمان
10	 مسیر کد (Code Path) یعنی چی؟



- 11چاوگ (Quartile) چیست؟
- 12چطور کار می کنه؟
- 14حملات تایمینگ وب چیه؟
- 15Cross-site Search (جست و جوی بین سایتی):
- 15Username Enumeration (شمارش نام کاربری):
- 16:A Probe for Potential Race Conditions
- 16:A Lab-Proven Attack
- 17What if ...
- 17:A Theoretical Attack That Would Be Absolutely Amazing... If It Actually Worked
- 20مشکل تحقیقات قبلی حملات تایمینگ
- 20محیط تست
- 21پیشرفت در دقت حملات تایمینگ
- 21هنوز چه چالش هایی داریم؟
- 22سه تکنیک کلیدی
- 24ایجاد حملات تایمینگ که همه جا کار کنن
- 24ابزار Param Miner:
- 25ارزش واقعی تحقیق
- 25عوامل کلیدی موفقیت حملات تایمینگ
- 26پاسخ به سؤالات دشوار
- 26هدف اصلی حملات تایمینگ
- 27مقایسه رشته ها (String Comparison)
- 29چالش در دنیای واقعی



- 29.....واحد برای موفقیت حمله
- 31.....تقسیم نویز به دو بخش:
- 33.....جتر شبکه چیه؟
- 33.....چرا جتر شبکه دشمن حملات تایمینگ؟
- 34.....تغییرات توی 5 سال گذشته
- 34.....لو کال کردن حملات تایمینگ
- 37.....مشکل ترتیب ثابت درخواست ها
- 39.....Portable کردن حملات تایمینگ
- 40.....Single-Packet Attack معرفی
- 41.....یه مشکل جدید (Catch)
- 43.....راه حل ساده برای همگام سازی دو بسته:
- 47.....عملی کردن حملات تایمینگ
- 48.....چطور نویز سرور رو کم کنیم؟
- 49.....مثال واقعی توی حمله
- 49.....چطور سیگنال رو زیاد کنیم؟
- 51.....مثال واقعی توی حمله
- 52.....فناوری های مدرن و تأخیر
- 53.....Hidden attack surface
- 54.....روش جایگزین برای اندازه گیری زمان:
- 55.....Discovery overload
- 57.....سخت ترین مشکل
- 58.....Proving the concept
- 63.....Server-side injection



64.....	Fully blind SQLi	حمله
65.....	Blind JSON Injection	
67.....	Blind Server-Side Parameter Pollution	
68.....	Bug doppelgangers	
70.....	Reverse proxy misconfigurations	
72.....	SSRF محدود (Scoped SSRF)	
75.....	شناسایی SSRF های محدود شده (Detecting scoped SSRF)	
81.....	سناریو	
84.....	جستجوی مسیرهای پنهان	
84.....	جستجو برای مسیرهای مخفی	
85.....	روش کار Param Miner	
87.....	دور زدن فایروال (Firewall bypass)	
89.....	Firewall bypass - invisible route variant	
92.....	دور زدن قوانین فرانت اند (Front-End Rule Bypass)	
95.....	Front-End Impersonation Attacks	
99.....	زنجیره کردن (Chaining)	
103.....	نقشه راه تحقیقات زمان بندی (Timing research roadmap)	
104.....	آینده حملات زمان بندی	
105.....	دفاع (Defence)	

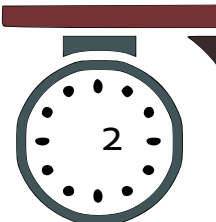


SECTION

1

مفاهیم

پایه



قبل از اینکه وارد جزئیات مقاله بشیم، بیایم با چند مفهوم کلیدی آشنا بشیم که توی این مقاله زیاد ازشون استفاده می‌شه:

SSRF و Scoped SSRF چیست؟

SSRF (Server-Side Request Forgery) یه نوع آسیب‌پذیری امنیتی که به مهاجم اجازه می‌ده از یه سرور آسیب‌پذیر درخواست‌هایی به جاهایی بفرسته که نباید بتونه بهشون دسترسی داشته باشه. مثلاً می‌تونه به سرورهای داخلی شرکت (که از بیرون قابل دسترسی نیستن) یا حتی سرورهای خارجی درخواست بفرسته. این می‌تونه باعث دسترسی به اطلاعات حساس یا حتی کنترل سرور بشه.

Scoped SSRF یه نوع محدودتر از SSRF هست. توی این حالت، سرور فقط به یه سری آدرس‌های خاص (مثلاً سابدامنه‌های شرکت مثل *.example.com) اجازه می‌ده درخواست‌ها رو فوروارد کنه. این محدودیت باعث می‌شه تشخیص Scoped SSRF با روش‌های معمولی (مثل فرستادن درخواست به سرور خودت و دیدن اینکه آیا درخواست میاد یا نه) سخت‌تر بشه، ولی همچنان خطرناکه چون می‌تونه به مهاجم دسترسی به سیستم‌های داخلی بده.

Reverse Proxy چیست؟

Reverse Proxy یه سرور واسطه‌ست که بین کاربر (مثلاً مرورگر شما) و سرورهای اصلی (بک‌اند) قرار می‌گیره. وقتی یه درخواست به ریورس پراکسی می‌فرستی، اون درخواست رو بر اساس یه سری قوانین (مثلاً هدر Host) به سرور مناسب فوروارد می‌کنه. مثلاً اگه هدر Host توی درخواست app.example.com باشه، پراکسی ممکنه درخواست رو به سرور داخلی app بفرسته. توی این مقاله، Reverse Proxy ها نقش مهمی دارن، چون اگه درست پیکربندی نشده باشن، می‌تونن به مهاجم اجازه بدن به سیستم‌های داخلی دسترسی پیدا کنه.



فرانت‌اند و بک‌اند چیست؟

فرانت‌اند (Front-End): فرانت‌اند بخشی از سیستم وب هست که مستقیم با کاربر در ارتباطه. مثلاً وقتی به وب‌سایت رو باز می‌کنی، سروری که صفحه وب رو بهت نشون می‌ده و درخواستت رو پردازش می‌کنه، فرانت‌اند محسوب می‌شه. فرانت‌اند معمولاً قوانین امنیتی داره (مثلاً چک کردن اینکه آیا کاربر ادمینه یا نه) و درخواست‌ها رو به بک‌اند می‌فرسته.

بک‌اند (Back-End): بک‌اند بخشی از سیستم وب هست که کارهای اصلی (مثل پردازش داده‌ها، ذخیره اطلاعات توی دیتابیس، یا اجرای منطق برنامه) رو انجام می‌ده. بک‌اند معمولاً از بیرون قابل دسترسی نیست و فقط از طریق فرانت‌اند می‌تونی باهاش ارتباط برقرار کنی. توی این مقاله، مهاجم‌ها سعی می‌کنن با دور زدن فرانت‌اند، مستقیم با بک‌اند حرف بزنن و از اعتماد بک‌اند به فرانت‌اند سوءاستفاده کنن.

حملات زمان‌بندی چیست؟

حملات زمان‌بندی (Timing Attacks) به نوع حمله هستن که مهاجم با اندازه‌گیری زمان پاسخ سرور، اطلاعاتی درباره رفتار داخلی سرور به دست میاره. مثلاً اگر به سرور برای یه درخواست خاص 25 میلی‌ثانیه طول بکشه و برای یه درخواست دیگه 20 میلی‌ثانیه، این تفاوت 5 میلی‌ثانیه‌ای می‌تونه به مهاجم بگه که سرور داره چه کاری انجام می‌ده (مثلاً آیا به DNS Lookup انجام داده یا نه). توی این مقاله، از حملات زمان‌بندی برای تشخیص Scoped SSRF و پیدا کردن آسیب‌پذیری‌ها استفاده می‌کنیم.

مقایسه و اینفوگرافیک

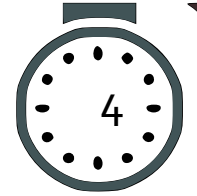
حملات تایمینگ با بقیه حملات فرق دارن. مثلاً توی Spectre/Meltdown، داری حافظه CPU رو می‌خونی خیلی پیچیده‌ست و نیاز به دسترسی لوکال داره. تایمینگ ساده‌تره؛ فقط با یه درخواست HTTP کار می‌کنه. یا Brute-Force که باید میلیون‌ها رمز تست کنی تایمینگ با چند درخواست جواب می‌ده. این تفاوت یعنی تایمینگ یه ابزار مخفی و سریع برای هکرهاست.

یه خط زمانی که درخواست `GET /login?user=admin` رو نشون می‌ده.

Processing: 100ms، Connection: 30ms، DNS Lookup: 20ms (چون کاربر واقعیه).

حالا `user=nobody`: Processing فقط 70ms.

این تفاوت 30ms سیگنال ماست یه نمودار ساده که اینو نشون بده، توی ذهنت می‌مونه!



سیگنال و نویز یعنی چی؟

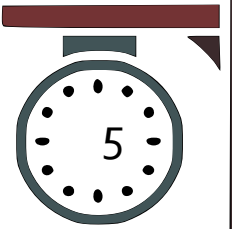
توی حملات زمان‌بندی، سیگنال همون تأخیره که می‌خوایم اندازه بگیریم (مثلاً 5 میلی‌ثانیه تفاوت بین دو درخواست). نویز به تغییرات تصادفی توی زمان پاسخ اشاره داره که ممکنه به خاطر عواملی مثل شلوغی شبکه یا بار سرور ایجاد بشه (مثلاً یه دفعه پاسخ 22 میلی‌ثانیه بشه و دفعه بعد 28 میلی‌ثانیه). اگه نویز زیاد باشه، تشخیص سیگنال (تأخیر واقعی) سخت می‌شه، برای همین باید راه‌هایی پیدا کنیم که نویز رو کم کنیم یا سیگنال رو بزرگ‌تر کنیم.

هدرهای سفارشی چیست؟

هدرهای سفارشی (Custom Headers) هدرهای HTTP هستن که یه شرکت برای خودش تعریف می‌کنه تا اطلاعات خاصی رو بین فرانت‌اند و بک‌اند منتقل کنه. مثلاً یه هدر به اسم X-Is-Admin می‌تونه به بک‌اند بگه که درخواست از یه ادمین اومده یا نه. بک‌اند معمولاً به این هدرها اعتماد کامل داره، چون فکر می‌کنه فقط فرانت‌اند می‌تونه اونا رو تنظیم کنه. اما اگه یه مهاجم بتونه این هدرها رو جعل کنه (مثلاً با استفاده از Reverse Proxy)، می‌تونه وانمود کنه که یه کاربر مورد اعتماد (مثلاً ادمین) هست و به بخش‌های حساس دسترسی پیدا کنه.

تونل کردن درخواست HTTP چیست؟

تونل کردن درخواست (HTTP Request Tunneling) یعنی یه درخواست رو طوری بفرستی که از یه مسیر غیرمعمول (مثلاً یه پراکسی یا یه آسیب‌پذیری) به بک‌اند برسه، بدون اینکه فرانت‌اند بتونه هدرها رو تغییر بده. مثلاً توی یه حمله، می‌تونی یه درخواست رو از یه مسیر خاص (مثلاً یه پراکسی آسیب‌پذیر) بفرستی و هدرهای جعل شده رو مستقیم به بک‌اند برسونی، بدون اینکه فرانت‌اند بتونه اونا رو بازبینی کنه.



HTTP Request Smuggling چیست؟

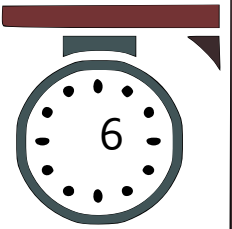
HTTP Request Smuggling (قاچاق درخواست HTTP) به نوع حمله‌ست که توی اون مهاجم از تفاوت توی نحوه پردازش درخواست‌های HTTP توسط سرورها (مثلاً فرانت‌اند و بک‌اند) سوءاستفاده می‌کنه. مثلاً اگه فرانت‌اند و بک‌اند روی اینکه یه درخواست کجا تموم می‌شه و درخواست بعدی کجا شروع می‌شه توافق نداشته باشن، مهاجم می‌تونه یه درخواست جعلی رو قاچاق کنه و به بک‌اند بفرسته. این می‌تونه باعث بشه بک‌اند یه درخواست خطرناک (مثلاً یه درخواست که به یه کاربر دیگه صدمه می‌زنه) رو اجرا کنه. توی این مقاله، Reverse Proxy ها می‌تونن برای این نوع حمله استفاده بشن.

توابع زمان ثابت (Constant-Time Functions) چیست؟

توابع زمان ثابت (Constant-Time Functions) توابعی هستن که همیشه یه زمان ثابت طول می‌کشن تا اجرا بشن، چه ورودی درست باشه چه غلط. مثلاً اگه داری رمز عبور کاربر رو با رمز ذخیره‌شده مقایسه می‌کنی، نباید زمان مقایسه به این بستگی داشته باشه که رمز درست یا نه. اگه مقایسه برای رمز درست 20 میلی‌ثانیه طول بکشه و برای رمز غلط 10 میلی‌ثانیه، مهاجم می‌تونه از این تفاوت زمان استفاده کنه و بفهمه که رمز درست بوده یا نه. توابع زمان ثابت این مشکل رو حل می‌کنن، چون همیشه یه زمان ثابت (مثلاً 20 میلی‌ثانیه) طول می‌کشن.

مفهوم زنجیره کردن (Chaining) در حملات چیست؟

زنجیره کردن (Chaining) توی حملات امنیتی یعنی ترکیب چند آسیب‌پذیری یا تکنیک مختلف برای رسیدن به یه هدف بزرگ‌تر. مثلاً توی این مقاله، می‌تونن یه Scoped SSRF رو با کنترل یه سابدامنه و تغییر رکورد DNS ترکیب کنی تا به یه Full SSRF تبدیلش کنی. این کار مثل اینه که چند ابزار رو با هم استفاده کنی تا یه قفل بزرگ‌تر رو باز کنی. زنجیره کردن باعث می‌شه حملات قوی‌تر و خطرناک‌تر بشن.



DNS و DNS Caching چیست؟

DNS (Domain Name System) سیستمی که نام دامین‌ها (مثل example.com) رو به آدرس‌های IP (مثل 192.0.2.1) تبدیل می‌کنه. وقتی یه سرور می‌خواد به یه دامین وصل بشه، یه درخواست DNS Lookup می‌فرسته تا آدرس IP اون دامین رو پیدا کنه. این فرآیند یه کم زمان می‌بره (مثلاً 5 میلی‌ثانیه).

DNS Caching یعنی سرور نتیجه این Lookup رو برای یه مدت ذخیره (کش) می‌کنه، تا اگه دوباره به همون دامین وصل بشه، نیازی به Lookup دوباره نداشته باشه و سریع‌تر کار کنه. مثلاً اگه سرور یه بار آدرس example.com رو پیدا کنه و کش کنه، دفعه بعد که بخواد به example.com وصل بشه، دیگه Lookup نمی‌کنه و زمان کمتری (مثلاً 1 میلی‌ثانیه) طول می‌کشه. توی این مقاله، از این تفاوت زمان برای تشخیص آسیب‌پذیری‌ها استفاده می‌کنیم.

ابزارهای Turbo Intruder و Param Miner چیست؟

Param Miner و Turbo Intruder ابزارهایی هستن که برای تست امنیت وب‌سایت‌ها و پیدا کردن آسیب‌پذیری‌ها استفاده می‌شن.

Param Miner: یه افزونه برای Burp Suite (یه ابزار تست نفوذ) هست که به محقق‌های امنیتی کمک می‌کنه پارامترها و هدرهای مخفی رو توی درخواست‌های HTTP پیدا کنن. توی این مقاله، از Param Miner برای پیدا کردن Reverse Proxy ها و حدس زدن هدرهای مفید استفاده می‌کنیم.

Turbo Intruder: یه ابزار دیگه برای Burp Suite هست که برای فرستادن تعداد زیادی درخواست به سرور و تحلیل پاسخ‌ها (مثلاً برای حملات زمان‌بندی) استفاده می‌شه. توی این مقاله، از Turbo Intruder برای بهبود حملات زمان‌بندی و سریع‌تر کردن فرآیند تست استفاده می‌کنیم.



Pingback چیست؟

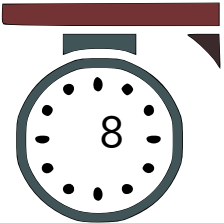
Pingback به روشی که توی تست‌های امنیتی استفاده می‌شه. توی این روش، مهاجم به درخواست به سرور می‌فرسته و می‌خواه ببینه آیا سرور به یه آدرس خاص (مثلاً سرور خود مهاجم) وصل می‌شه یا نه. اگر سرور به آدرس مهاجم وصل بشه، یعنی یه آسیب‌پذیری (مثل SSRF) وجود داره. توی Scoped SSRF، چون سرور فقط به آدرس‌های خاصی اجازه می‌ده درخواست بفرسته، روش Pingback معمولاً کار نمی‌کنه.

DNS Label Length چیست؟

توی DNS، هر بخش از یه دامین (مثل aaa.tوی aaa.example.com) به لیبل (Label) حساب می‌شه. استاندارد DNS می‌گه هر لیبل نمی‌تونه بیشتر از 63 کاراکتر باشه. اگر یه لیبل بیشتر از 63 کاراکتر باشه، کلاینت DNS معمولاً از انجام Lookup خودداری می‌کنه. توی این مقاله، از این قانون برای ایجاد تفاوت توی زمان پاسخ سرور و تشخیص آسیب‌پذیری‌ها استفاده می‌کنیم.

Timing Oracle چیست؟

Timing Oracle (اوراکل زمان‌بندی) به مفهوم توی حملات زمان‌بندی که به یه رفتار توی سرور اشاره داره که باعث می‌شه زمان پاسخ سرور اطلاعات حساسی رو فاش کنه. مثلاً اگر سرور برای یه ورودی خاص (مثلاً یه رمز درست) 20 میلی‌ثانیه طول بکشه و برای یه ورودی دیگه (مثلاً رمز غلط) 10 میلی‌ثانیه، این تفاوت زمان می‌تونه به مهاجم بگه که کدوم ورودی درست بوده. توی این مقاله، از Timing Oracle ها برای پیدا کردن آسیب‌پذیری‌ها (مثل Scoped SSRF) و فهمیدن رفتار داخلی سرور استفاده می‌کنیم.



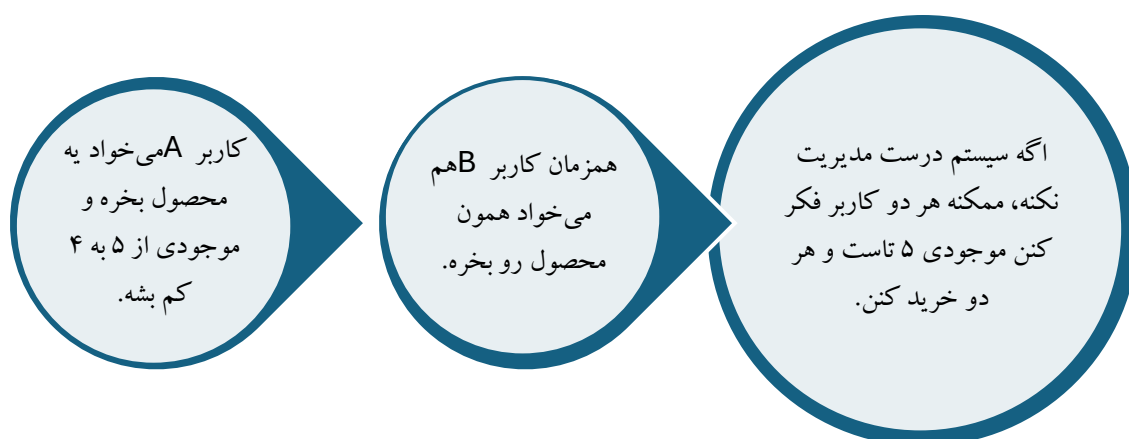
Jitter چیست؟

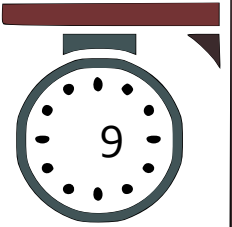
Jitter (لرزش) توی حملات زمان‌بندی به تغییرات تصادفی توی زمان پاسخ سرور اشاره داره که می‌تونه به خاطر عواملی مثل شلوغی شبکه، بار سرور، یا تأخیرهای غیرقابل‌پیش‌بینی ایجاد بشه. مثلاً اگه یه سرور معمولاً 20 میلی‌ثانیه طول بکشه تا پاسخ بده، ولی یه دفعه به خاطر نویز شبکه 22 میلی‌ثانیه یا 18 میلی‌ثانیه بشه، این تغییرات تصادفی Jitter حساب می‌شن. Jitter می‌تونه تشخیص سیگنال‌های کوچک (مثل تفاوت‌های زمانی) رو سخت‌تر کنه، برای همین توی حملات زمان‌بندی سعی می‌کنیم با تکنیک‌هایی Jitter رو کم کنیم.

مفهوم قفل کردن و دسترسی همزمان

توی یه وب‌سایت، وقتی چند نفر همزمان دارن با یه سیستم کار می‌کنن (مثلاً چند کاربر همزمان می‌خوان یه محصول رو توی یه فروشگاه آنلاین بخرن)، اطلاعات توی دیتابیس (مثلاً تعداد موجودی اون محصول) ذخیره می‌شه. دیتابیس معمولاً به شکل جدول‌هایی که هر ردیف (Row) توی جدول یه داده خاص رو نشون می‌ده (مثلاً یه ردیف برای محصول X که موجودیش 5 تاست).

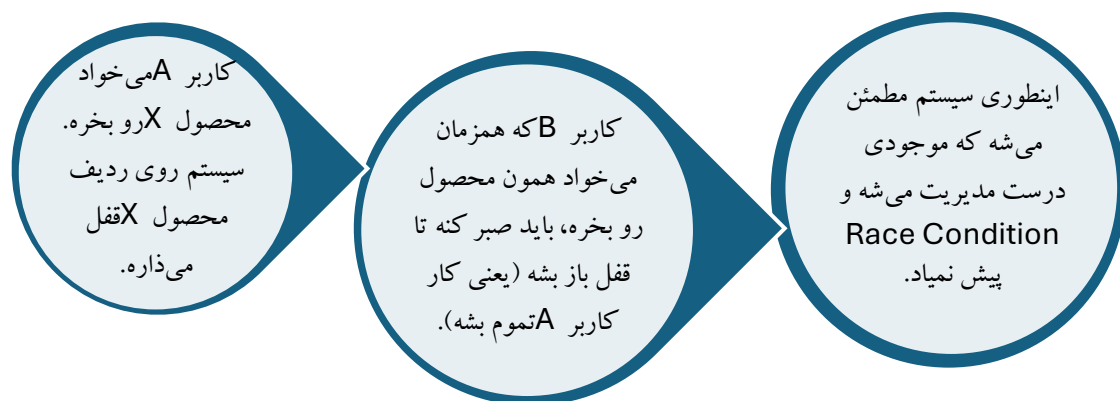
دسترسی همزمان یعنی دو یا چند نفر (یا چند درخواست) همزمان بخوان یا همون ردیف توی دیتابیس کار کنن (مثلاً موجودی محصول X رو کم کنن). اگه این دسترسی کنترل نشه، ممکنه مشکل پیش بیاد. مثلاً:



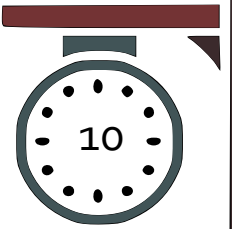


در حالی که فقط باید به نفر می تونست بخره (چون موجودی فقط برای به نفر کافی بود). این مشکل به اسم Race Condition شناخته می شه.

قفل کردن (Locking) به روشه برای جلوگیری از این مشکل. وقتی سیستم روی به ردیف توی دیتابیس قفل می ذاره، یعنی به بقیه می گه: الان فقط به نفر می تونه با این ردیف کار کنه، بقیه باید صبر کنن تا کار این تموم بشه. مثلاً:



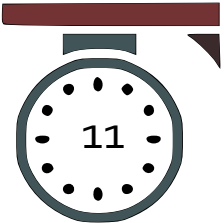
توی حمله زمان بندی چی می شه؟ توی این حمله، مهاجم با اندازه گیری زمان پاسخ سرور می فهمه که آیا سرور روی به ردیف قفل گذاشته یا نه. اگه قفل گذاشته باشه، زمان پاسخ بیشتر می شه (چون سرور منتظر می مونه تا قفل باز بشه). این اطلاعات می تونه به مهاجم کمک کنه که بفهمه سیستم چطوری کار می کنه و از Race Condition سوءاستفاده کنه.



مسیر کد (Code Path) یعنی چی؟

مسیر کد به مجموعه مراحل اشاره دارد که سرور برای پردازش یک درخواست (Request) طی می‌کند. وقتی یک درخواست (مثلاً یک درخواست HTTP مثل GET /) به سرور می‌فرستی، سرور یک سری کد اجرا می‌کند تا به اون درخواست جواب بده. این کدها می‌تونن شامل مراحل مختلفی باشن، مثل:

- بررسی هدرهای درخواست (مثلاً چک کردن کوکی‌ها یا توکن احراز هویت).
 - جستجو توی دیتابیس (مثلاً پیدا کردن اطلاعات کاربر).
 - بارگذاری فایل‌ها (مثلاً لود کردن یک فایل HTML یا CSS).
 - انجام محاسبات (مثلاً محاسبه یک جمع یا تولید یک گزارش).
 - ارسال پاسخ به کاربر (مثلاً برگردوندن یک صفحه وب).
- هر کدوم از این مراحل یک بخش از مسیر کد هستن. حالا، مسیر کد می‌تونه کوتاه یا بلند باشه:
- **مسیر کد کوتاه:** یعنی سرور با کمترین مراحل ممکن جواب بده. مثلاً اگر یک درخواست به یک صفحه ساده بفرستی که فقط یک متن سلام توش هست و سرور فقط باید یک فایل HTML رو برگردونه، این یک مسیر کد کوتاهه، چون مراحل کمی داره (فقط لود کردن فایل و ارسالش).
 - **مسیر کد بلند:** یعنی سرور کلی کار انجام بده تا جواب بده. مثلاً اگر یک درخواست به یک صفحه داشبورد بفرستی که سرور باید دیتابیس رو جستجو کنه، اطلاعات کاربر رو بگیره، یک سری محاسبات انجام بده، چند تا فایل (مثل CSS و JavaScript) رو بارگذاری کنه و بعد یک صفحه پیچیده برگردونه، این یک مسیر کد بلنده، چون مراحل زیادی داره.



چارک (Quartile) چیست؟

چارک به مفهوم آماریه که توی تحلیل داده‌ها استفاده می‌شه و به ما کمک می‌کنه به مجموعه داده (مثلاً به لیست از زمان‌های پاسخ سرور) رو به چهار قسمت مساوی تقسیم کنیم تا پراکندگی داده‌ها رو بهتر بفهمیم. فرض کن 30 تا زمان پاسخ سرور داری، مثلاً از 50 تا 340 میلی‌ثانیه. برای پیدا کردن چارک‌ها، اول داده‌ها رو از کوچیک به بزرگ مرتب می‌کنی. بعد:

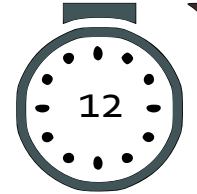
- **چارک اول (Q1) یا چارک پایین:** 25 درصد داده‌های اول رو نشون می‌ده، یعنی سریع‌ترین زمان‌ها (مثلاً توی این لیست، 7-8 تا از سریع‌ترین زمان‌ها، از 50 تا 120 میلی‌ثانیه).

- **چارک دوم (Q2) یا میانه:** 50 درصد داده‌ها رو به دو قسمت مساوی تقسیم می‌کنه (مثلاً وسط این لیست، حدود 195 میلی‌ثانیه).

- **چارک سوم (Q3) یا چارک بالا:** 75 درصد داده‌های اول رو نشون می‌ده، یعنی 25 درصد داده‌های بزرگ‌تر بالای این مرز (مثلاً توی این لیست، حدود 275 میلی‌ثانیه).

توی حملات زمان‌بندی، ما معمولاً از چارک پایین (Bottom Quartile) استفاده می‌کنیم، چون این **زمان‌ها کمترین نویز** (تغییرات تصادفی، مثل تأخیرهای شبکه یا سرور) رو دارن و پایدارترن. مثلاً اگه بخوای ببینی به پارامتر روی زمان پاسخ سرور تأثیر گذاشته یا نه، چارک پایین دو مجموعه زمان (با پارامتر و بدون پارامتر) رو مقایسه می‌کنی تا ببینی آیا از هم جدا هستن یا هم‌پوشانی دارن.

از اینجا به بعد، وارد مباحث اصلی می‌شیم که در ابتدا به توضیحات مقدماتی این حمله اختصاص داده و به تدریج به بخش‌های عملی می‌رسیم. یادتون باشه هر چقدر با مباحث تئوری و نحوه عملکرد این نوع آسیب‌پذیری‌ها آشنا باشید، نه تنها در فهم این آسیب‌پذیری خاص، بلکه در تحلیل و بررسی هر آسیب‌پذیری دیگه‌ای که بخواید مطالعه کنید، موفق‌تر عمل می‌کنید. در نهایت، چیزی که شما رو در تسلط بر هر آسیب‌پذیری به مهارت واقعی می‌رسونه، تجربه عملی و کار کردن روی اون هست.



چطور کار می‌کنه؟

فرض کنید به وب اپلیکیشن داره رمز عبور کاربر رو با به مقدار ذخیره شده توی پایگاه داده مقایسه می‌کنه. اگه این مقایسه کاراکتر به کاراکتر انجام بشه، هر بار که به کاراکتر درست وارد بشه، زمان بیشتری صرف می‌شه. به مهاجم می‌تونه با فرستادن درخواست‌های مکرر و اندازه‌گیری زمان پاسخ، کاراکترهای رمز عبور رو یکی‌یکی حدس بزنه و رازهای سرور رو فاش کنه.

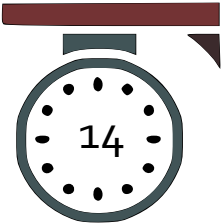
وبسایت‌ها پر از آسیب‌پذیری‌هایی هستن که مثل به پیشگو، رازهای پنهانشون رو از طریق Timing Oracle ها فاش می‌کنن. وقتشه که به این زمزمه‌های زمان‌بندی گوش بدیم! توی این مقاله، به سری تکنیک جدید برای حملات زمان‌بندی معرفی می‌کنم تا بتونیم رازهای سرورها رو بیرون بکشیم؛ از تنظیمات اشتباه و مخفی گرفته تا مسیرهای مخفی به مناطق ممنوعه، تزریق داده Blind به ساختارهای اطلاعاتی، و سطح‌های حمله نامرئی.

این به تهدید تئوری نیست! هر تکنیک رو با نمونه‌های واقعی از دنیای بیرون و روی هدف‌های مختلف نشون می‌دم. پیشرفت‌های جدید، این حملات رو هم دقیق‌تر و هم کارآمدتر کرده؛ حالا می‌تونید توی عرض ده ثانیه به اختلاف زمانی زیر به میلی‌ثانیه رو بدون نیاز به شرایط خاص یا تنظیمات آزمایشگاهی شناسایی کنید. به زبون ساده، می‌خوام حمله‌های زمان‌بندی‌ای رو بهتون یاد بدم که واقعاً بتونید ازشون استفاده کنید.

برای کمک بهتون، به سری ابزار Open Source و قدرتمند در اختیارتون می‌ذارم که هم می‌تونن به صورت خودکار اکسپلویت کنن، هم بتونید باهشون اسکریپت‌های حمله سفارشی بنویسید. به چالش CTF هم براتون آماده کردم تا مهارت‌هاتون رو تقویت کنید.

می‌خوای این مسیر رو جدی‌تر دنبال کنی؟ من کمکت می‌کنم ایده‌های حمله‌ت رو از تئوری به واقعیت تبدیل کنی. چطوری؟ با روشی که از تست هزاران ایده روی وبسایت‌های مختلف به دست اومده. خیلی وقته که این کانال جانبی قدرتمند و همه‌جا حاضر رو نادیده گرفتیم. برای شروع، بیایم با مفاهیم پایه‌ای که توی بخش بعدی توضیح دادم آشنا بشیم.





حملات تایمینگ وب چیه؟

حملات تایمینگ وب به استفاده از تفاوت‌های زمانی توی پاسخ‌های سرور برای کشف اطلاعات مخفی می‌گن. مثلاً اگه به سرور برای یه درخواست خاص (مثل ورود با نام کاربری درست) بیشتر طول بکشه تا جواب بده، می‌تونی بفهمی که یه چیزی اون پشت داره اتفاق می‌افته (مثلاً مقایسه رمز عبور).

چرا بدنامن؟

این حملات دو مشکل بزرگ دارن:

وعده‌های بزرگ: توی تئوری می‌گن می‌تونن رمزها رو حدس بزنی یا دیتابیس رو هک کنی، ولی توی عمل کار نمی‌کنن.

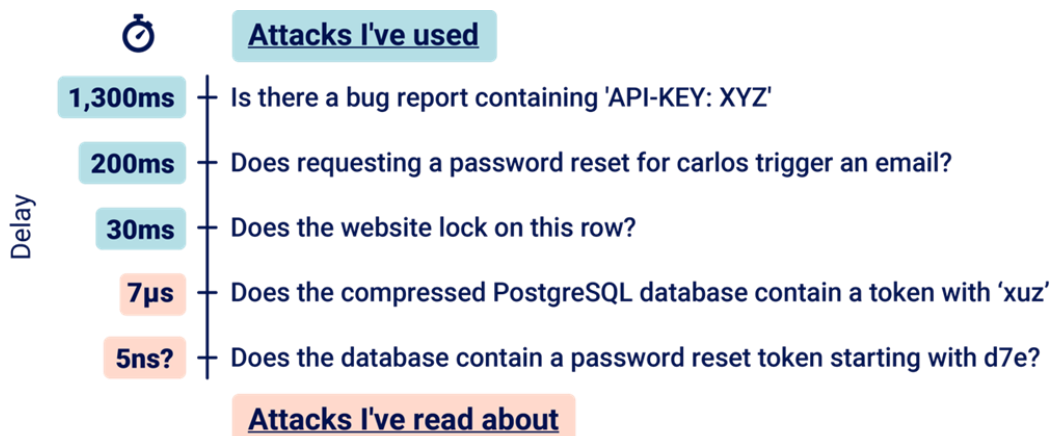
عدم کارایی عملی: مثال‌ها معمولاً توی محیط آزمایشگاهی (Lab) کار می‌کنن، ولی توی دنیای واقعی (با نویز شبکه و سرور) شکست می‌خورن. مثلاً توی تصویر دیدیم که تفاوت 5 نانوثانیه عملاً غیرقابل تشخیصه.

فرصت ازدست‌رفته:

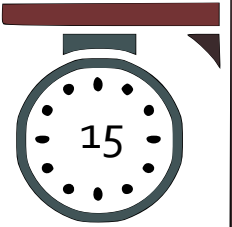
به خاطر این شهرت بد، ما یه فرصت بزرگ رو نادیده گرفتیم. حملات تایمینگ می‌تونن خیلی قدرتمند باشن اگه درست طراحی بشن.

اولین تلاش: اول حملاتی پیدا شد که فقط توی تئوری کار می‌کردن (مثل همون 5 نانوثانیه توی تصویر). این‌ها توی دنیای واقعی به خاطر نویز شبکه و سرور قابل تشخیص نبودن.

دومین تلاش: این بار به حملاتی پرداخته شد که توی دنیای واقعی کار کردن (مثل همون 1,300ms تا 30ms توی تصویر) و همچنین حملاتی که فقط درباره‌شون خونده بودیم. هدف این هست که بفهمیم چرا یه سری کار می‌کنن و یه سری نه.



Listen to the whispers web timing attacks that actually works



از بالا که به تصویر نگاه کنیم، این ها نمونه هایی از آسیب پذیری هایی هست که تست شده و به توضیح هر کدام به صورت جداگونه میپردازیم:

Cross-site Search (جست و جوی بین سایتی):

توضیح: این حمله برای پیدا کردن اطلاعات حساس توی یه سیستم دیگه از طریق تفاوت زمانی استفاده می شه.

توی تصویر، سؤال آیا یه گزارش باگ با API-KEY: XYZ وجود داره؟ با تأخیر ms1,300 بود.

چطور کار می کنه؟ فرض کن یه وبسایت گزارش باگ داری که می تونی توی اون جست و جو کنی. اگه سرچ کنی API-KEY: XYZ و سرور ms1,300 طول بکشه تا جواب بده، ولی برای API-KEY: ABC فقط ms100 طول بکشه، می فهمی که XYZ وجود داره، چون سرور برای پیدا کردنش توی دیتابیس وقت بیشتری صرف کرده.

مثال ساده: یه سایت مثل HackerOne رو تصور کن. اگه سرچ API-KEY: secret123 بیشتر طول بکشه، می فهمی که یه گزارش با این کلید وجود داره.

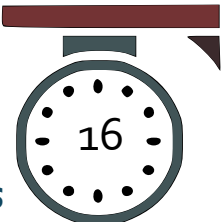
Username Enumeration (شمارش نام کاربری):

توضیح: این حمله برای فهمیدن اینکه یه نام کاربری خاص توی سیستم وجود داره یا نه استفاده می شه.

توی تصویر، سؤال آیا درخواست ریست رمز عبور برای carlos باعث ارسال ایمیل می شه؟ با تأخیر 200ms بود.

چطور کار می کنه؟ اگه برای ریست رمز عبور کاربر carlos سرور 200ms طول بکشه (چون ایمیل می فرسته)، ولی برای یه کاربر ناموجود مثل nobody فقط 50ms طول بکشه، می فهمی که carlos وجود داره.

مثال ساده: توی یه سایت ورود، اگه وارد کنی username=admin و سرور 100ms طول بکشه (چون داره رمز رو چک می کنه)، ولی برای username=fakeuser فقط 20ms طول بکشه، می فهمی که admin یه کاربر واقعیه.



:A Probe for Potential Race Conditions

توضیح: این حمله برای پیدا کردن Race Condition (به مشکل توی برنامه نویسی که دو عملیات همزمان باعث خطا می شن) استفاده می شه. توی تصویر، سؤال آیا وبسایت روی این ردیف قفل می کنه؟ با تأخیر 30ms بود.

چطور کار می کنه؟ اگه سرور برای قفل کردن به ردیف توی دیتابیس (مثلاً برای جلوگیری از تغییر همزمان) 30ms طول بکشه، ولی بدون قفل 10ms طول بکشه، می فهمی که به Race Condition ممکنه وجود داشته باشه.

مثال ساده: فرض کن به سایت فروش بلیط داری. اگه دو نفر همزمان بلیط آخر رو بخرن و سرور قفل نکنه، ممکنه هر دو بلیط رو بگیرن. اگه قفل کردن 30ms طول بکشه، می تونی این مشکل رو تشخیص بدی.

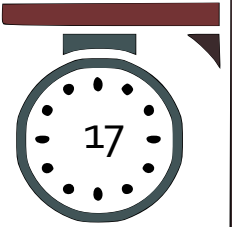
:A Lab-Proven Attack

توضیح: این حمله توی محیط آزمایشگاهی کار کرده، ولی توی دنیای واقعی نه.

توی تصویر، سؤال آیا دیتابیس فشرده شده PostgreSQL به توکن با xuz داره؟ با تأخیر $7\mu s$ (7 میکروثانیه) بود.

چطور کار می کنه؟ توی به محیط کنترل شده، سرور برای پیدا کردن توکن xuz توی دیتابیس فشرده شده 7 میکروثانیه بیشتر طول کشید، ولی این تفاوت توی دنیای واقعی به خاطر نویز کم می شه.

مثال ساده: توی به آزمایشگاه، اگه دیتابیس برای پیدا کردن به توکن خاص 7 میکروثانیه بیشتر طول بکشه، می تونی بفهمی که توکن وجود داره. ولی توی اینترنت، نویز شبکه (مثلاً 50ms) این تفاوت رو از بین می بره.



What if ...

A Theoretical Attack That Would Be Absolutely Amazing... If It :Actually Worked

توضیح: این حمله فقط توی تئوریه و توی عمل کار نمی‌کنه.

توی تصویر، سؤال آیا دیتابیس یه توکن ریست رمز عبور با d7e داره؟ با تأخیر 5ns (5 نانوثانیه) بود.

چطور کار می‌کنه؟ توی تئوری، اگه توکن با d7e شروع بشه، مقایسه کاراکتر به کاراکتر 5 نانوثانیه بیشتر طول می‌کشه. ولی این تفاوت خیلی کوچیک توی دنیای واقعی قابل تشخیص نیست.

مثال ساده: فرض کن یه توکن ریست رمز عبور داری که می‌خواهی حدس بزنی. اگه با d7e شروع بشه، مقایسه 5 نانوثانیه بیشتر طول می‌کشه، ولی این زمان توی نویز شبکه (مثلاً 100ms) گم می‌شه.

تصویر یه نمودار عمودی با محور Delay هست که دو دسته‌بندی رو نشون می‌ده:

Attacks I've Used (حملاتی که استفاده کردم): این‌ها حملاتی هستن که نویسنده (جیمز کتل) تونسته توی دنیای واقعی باهاشون موفقیت کسب کنه.

Attacks I've Read About: این‌ها حملاتی هستن که فقط توی مقالات یا تئوری دیدتشون، ولی توی عمل کار نکردن یا خیلی سختن.

محور عمودی تأخیر رو نشون می‌ده (از 5 نانوثانیه تا 1,300 میلی‌ثانیه) و هر حمله با یه سؤال همراهه که اون حمله می‌تونه بهش جواب بده.

Listen to the whispers web timing attacks that actually works



به تصویر پایین نگاه کنید.

تفاوت بزرگ: حملاتی که جیمز تونسته استفاده کنه (1,300ms تا 30ms) تفاوت زمانی بزرگی دارن و توی دنیای واقعی قابل تشخیصن. اما حملاتی که فقط خونده (7μs تا 5ns) تفاوت زمانی خیلی کوچیکی دارن و توی عمل کار نمی کنن.

چالش: حملات تئوری (پایین نمودار) به خاطر نویز شبکه و سرور توی دنیای واقعی گم می شن. جیمز می خواد این شکاف رو پر کنه و حملات عملی تر بسازه.

مثال:

فرض کن یه وبسایت داری که یه فرم ورود داره. می خوای ببینی آیا یه نام کاربری خاص (مثل admin) وجود داره یا نه:

درخواست 1: `username=admin&password=wrong`

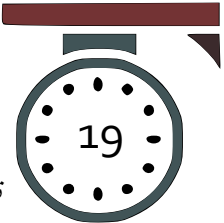
درخواست 2: `username=nonexistent&password=wrong`

اگه سرور برای admin بیشتر طول بکشه (مثلاً 100ms) چون داره رمز رو با دیتابیس مقایسه می کنه، ولی برای nonexistent سریع تر (50ms) جواب بده، می تونی بفهمی که admin وجود داره. این یه حمله تایمینگ ساده ست که توی دنیای واقعی کار می کنه.

در جستجوی روش های جدیدی که واقعاً جواب میدن، تمرکز رو گذاشتم روی فاصله ی بین این دو دسته، که خیلی هم زیاده:



Listen to the whispers web timing attacks that actually works



تصویر به نمودار عمودی با محور Time Gap به میکروثانیه (μs) هست که شکاف بین حملات عملی

و تئوری رو نشون می‌ده:

$30,000\mu s$ (30,000 میکروثانیه یا 30ms): سؤال آیا وبسایت برای این ردیف توی دیتابیس قفل اعمال می‌کنه، یعنی برای جلوگیری از اینکه چند نفر همزمان بتونن اطلاعات اون ردیف (مثلاً موجودی به محصول) رو تغییر بدن، دسترسی رو محدود می‌کنه و فقط به یه نفر اجازه می‌ده باهاش کار کنه؟ که یه حمله عملی برای پیدا کردن Race Condition بود.

$7\mu s$ (7 میکروثانیه): سؤال آیا دیتابیس فشرده‌شده PostgreSQL یه توکن با xuz داره؟ که یه حمله آزمایشگاهی بود.

فاصله: بین $30,000\mu s$ و $7\mu s$ یه شکاف بزرگ وجود داره که نشون می‌ده حملات عملی (با تأخیرهای بزرگ) توی دنیای واقعی کار می‌کنن، ولی حملات با تأخیرهای کوچک (مثل $7\mu s$) نه.

این تصویر تأکید می‌کنه که حملات تایمینگ توی دنیای واقعی به تأخیرهای بزرگ (مثل 30ms) نیاز دارن تا از نویز شبکه و سرور متمایز بشن.

حملات با تأخیرهای کوچک (مثل $7\mu s$) توی محیط آزمایشگاهی کار می‌کنن، ولی توی دنیای واقعی به خاطر نویز (مثلاً تأخیر شبکه 50ms) غیرقابل تشخیصه.

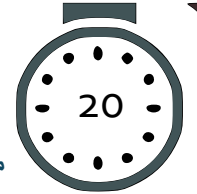
فرض کن می‌خواهی ببینی یه وبسایت یه API Key خاص رو توی دیتابیسش داره یا نه:

درخواست 1: `search=API-KEY:secret123`

درخواست 2: `search=API-KEY:fake123`

اگه درخواست اول $1,000ms$ طول بکشه (چون سرور داره دیتابیس رو سرچ می‌کنه) و درخواست دوم $100ms$ طول بکشه، می‌فهمی که `secret123` وجود داره. این یه حمله عملی با تأخیر بزرگه (مثل $30,000\mu s$). ولی اگه تفاوت فقط $7\mu s$ باشه، نمی‌تونی تشخیص بدی.

Listen to the whispers web timing attacks that actually works



مشکل تحقیقات قبلی حملات تایمینگ

تحقیقات روی حملات تایمینگ معمولاً روی یک هدف خاص (مثل یک وبسایت مشخص) تمرکز دارند. این باعث می‌شود ارزششون توی دنیای واقعی کم بشه، چون تکنیک‌هایی که برای یک وبسایت خاص کار می‌کنن، ممکنه برای سایت‌های دیگه کار نکنن.

مثال: فرض کن یک حمله تایمینگ برای یک وبسایت خاص طراحی کنی که برای پیدا کردن یک پارامتر مخفی 500ms تأخیر ایجاد می‌کنه. اگه این وبسایت یک سیستم کش خاص داشته باشه، این حمله فقط برای همون وبسایت کار می‌کنه و روی سایت‌های دیگه که کش متفاوتی دارن، جواب نمی‌ده.

پس باید تکنیک‌هایی بسازیم که روی هر هدف زنده‌ای کار کنن، یعنی بتونن روی هر وبسایتی توی دنیای واقعی اعمال بشن، نه فقط یک وبسایت خاص.

برای همین، تکنیک‌های جدید رو روی یک مجموعه بزرگ از وبسایت‌ها تست کردیم تا مطمئن بشیم واقعاً کار می‌کنن.

محیط تست

یک محیط تست با 30,000 وبسایت زنده ساختیم. این وبسایت‌ها از دو منبع انتخاب شدن:

bbscope: یک ابزار که دامنه‌های برنامه‌های باگ‌بانتی (Bug Bounty) رو جمع‌آوری می‌کنه.

Rapid7's Project Sonar DNS Database: یک دیتابیس بزرگ از دامنه‌ها و ساب‌دامین‌ها که با اسکن DNS جمع‌آوری شده.

این محیط تست یک فایل پروژه 20 گیگابایتی توی Burp Suite بوده که شامل همه وبسایت‌های شناخته‌شده با برنامه باگ‌بانتیه.

Listen to the whispers web timing attacks that actually works



پیشرفت در دقت حملات تایمینگ

قبل از این تحقیق، کوچکترین تفاوت زمانی که تونسته بودیم اکسپلویت کنیم، 30,000 میکروثانیه (ms30) بود، یعنی به تأخیر 30 میلی ثانیه‌ای که می‌تونستیم بفهمیم سرور به رفتار خاص داره یا نه. این همون تأخیر توی مثال آیا وبسایت برای این ردیف توی دیتابیس قفل اعمال می‌کنه، یعنی برای جلوگیری از اینکه چند نفر همزمان بتونن اطلاعات اون ردیف (مثلاً موجودی به محصول) رو تغییر بدن، دسترسی رو محدود می‌کنه و فقط به یه نفر اجازه می‌ده باهاش کار کنه؟ توی بخش قبله. حالا با پیشرفت‌های جدید توی دقت حملات تایمینگ، می‌تونیم تفاوت‌های خیلی کوچیک‌تر، یعنی 200 میکروثانیه (ms0.2) رو هم تشخیص بدیم، یعنی یه تأخیر خیلی ناچیز که قبلاً نمی‌تونستیم ببینیم.

چطور ممکن شده؟ محقق تکنیک‌هایی رو توسعه داده که نویز (مثل تأخیرهای تصادفی شبکه که باعث می‌شه زمان پاسخ سرور یه کم بالا و پایین بشه) رو کم می‌کنن و سیگنال (همون تفاوت زمانی واقعی که نشون می‌ده سرور چه کاری داره انجام می‌ده) رو واضح‌تر می‌کنن، طوری که حتی یه تغییر خیلی کوچیک توی زمان پاسخ رو هم می‌تونیم بفهمیم. این تکنیک‌ها بعداً توی مقاله توضیح داده می‌شن.

مثال ساده: فرض کن قبلاً فقط می‌تونستی بفهمی یه نفر 30 ثانیه دیرتر از قرارش به مهمونی رسیده، چون ساعت فقط دقیقه‌ها رو نشون می‌داد. حالا ساعت دقیق‌تر شده و می‌تونی بفهمی یه نفر فقط 0.2 ثانیه دیرتر رسیده، چون ساعت ثانیه‌ها و حتی کسری از ثانیه رو هم نشون می‌ده. این دقت خیلی بالاتره و کاربردهای بیشتری داره، چون می‌تونی مشکلات ریزتر سرور رو پیدا کنی.

هنوز چه چالش‌هایی داریم؟

ما دقت حملات تایمینگ رو به $\mu s200$ رسوندیم، ولی توی دنیای واقعی هنوز یه سری دیوار جلومونه که باید بشکنیمشون. بیایم چندتا شون رو ببینیم.

اولین دیوار، نویز شبکه‌ست. فرض کن توی یه کافی‌شاپ با وای‌فای شلوغ داری کار می‌کنی تأخیر یه دفعه ms50 می‌شه، یه دفعه ms100! سیگنال $\mu s200$ ما توی این Jitter گم می‌شه. برای حلش، می‌تونیم نمونه‌های بیشتری بگیریم (مثلاً 50 تا درخواست) و میانگین بگیریم، ولی این زمان‌بره. یا شاید یه روز با یه ابزار جدید نویز رو پیش‌بینی کنیم هنوز راه درازی داریم.

دومین چالش، سرورهای بی‌ثباتن. یه سرور تحت بار زیاد ممکنه برای یه درخواست ساده ms500 طول بکشه، فقط چون داره 1000 تا کاربر رو همزمان هندل می‌کنه. اینجا تشخیص یه تأخیر $\mu s200$ به خاطر پردازش یه هدر مثل پیدا کردن سوزن توی انبار کاه می‌شه! یه راه اینه که فقط چارک پایین زمان‌ها رو نگاه کنیم مثلاً از 20 تا درخواست، 5 تا پایین رو تحلیل کنیم ولی بازم کامل نیست.



سومین مشکل، دفاع‌های جدید. بعضیاً عمداً تأخیر تصادفی به پاسخ‌هاشون اضافه می‌کنن مثلاً بین 0 تا ms10.

اینجوری سیگنال $\mu s200$ ما غرق می‌شه توی نویز و تشخیص سخت می‌شه. مثلاً به سرور ممکنه اینجوری جواب بده:

- درخواست 1: ms210 ($\mu s200$ سیگنال + ms10 تأخیر تصادفی)

- درخواست 2: ms195 (بدون سیگنال + ms5 تأخیر تصادفی)

اینجا دیگه نمی‌فهمی تأخیر واقعی چیه! این چالش‌ها نشون می‌ده که حملات تایمینگ هنوز به علم در حال پیشرفته ما فقط باید باهوش‌تر بشیم.

سه تکنیک کلیدی

این پیشرفت‌ها باعث شدن که محقق بتونه تکنیک‌های جدیدی بسازه که روی سیستم‌های مختلف کار کنن. سه تکنیک اصلی که توی این مقاله بررسی می‌کنیم:



این سه تکنیک روی سیستم‌های متنوع تست شدن و نتایج خوبی دادن. توی ادامه مقاله، هر کدوم رو عمیق‌تر بررسی می‌کنیم.

Listen to the whispers web timing attacks that actually works



چرا $\mu s30,000$ به $\mu s200$ مهمه؟

30,000 میکروثانیه ($ms30$) به تأخیر نسبتاً بزرگه، یعنی اگه سرور 30 میلی ثانیه دیرتر جواب بده، توی دنیای واقعی می تونی به راحتی این تفاوت رو بینی و بفهمی که سرور به کار خاص داره انجام می ده، ولی این اندازه تأخیر به مشکل داره: ممکنه به خاطر نویز شبکه (مثلاً تأخیرهای تصادفی شبکه که گاهی 50 میلی ثانیه یا بیشتر می شن) این تفاوت گم بشه و نتونی درست تشخیص بدی. اما 200 میکروثانیه ($ms0.2$) خیلی خیلی دقیق تره، یعنی به تأخیر خیلی کوچیک که می تونه رفتارهای ریز و ظریف سرور رو نشون بده، مثلاً بتونی بفهمی سرور داره به هدر خاص (مثلاً به دستور مخفی) رو پردازش می کنه یا به خطای کوچیک توی کارش هست که قبلاً نمی تونستی بینی.

مثال ساده: فرض کن می خوای بفهمی به وبسایت به پارامتر مخفی به اسم debug داره یا نه،

یعنی می خوای بینی اگه این پارامتر رو بفرستی، سرور باهاش به کار خاص می کنه یا نه:

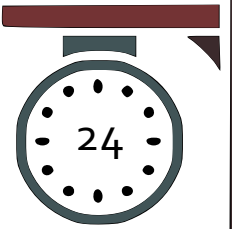
درخواست 1: `http://example.com/?debug=true` (داری می پرسی اگه debug رو فعال کنم،

چی می شه؟)

درخواست 2: `http://example.com/?random=true` (به پارامتر الکی می فرستی که مقایسه

کنی).

اگه درخواست اول 200 میکروثانیه بیشتر طول بکشه (چون سرور داره debug رو پردازش می کنه و به کم بیشتر فکر می کنه)، می فهمی که این پارامتر مخفی وجود داره و سرور باهاش کار می کنه. قبلاً برای اینکه بتونی همچین چیزی رو تشخیص بدی، باید تأخیر خیلی بزرگ تر، یعنی 30,000 میکروثانیه می داشت، که خیلی سخت تر بود، چون تأخیرهای بزرگ معمولاً به خاطر نویز شبکه گم می شدن و نمی تونستی مطمئن باشی که این تأخیر به خاطر پردازش پارامتره یا مشکل شبکه.



ایجاد حملات تایمینگ که همه جا کار کنن

محقق می‌خواد تکنیک‌هایی رو معرفی کنه که حملات تایمینگ رو از یه چیز تئوری یا محدود به یه هدف خاص، به یه ابزار عملی تبدیل کنه که روی هر وب‌سایتی توی دنیای واقعی کار کنه.

مثال ساده: فرض کن یه روش داری که فقط روی یه سایت خاص (مثل یه فروشگاه آنلاین) کار می‌کنه و می‌تونی بفهمی یه محصول خاص توی انبارشون هست یا نه. محقق می‌خواد این روش رو طوری تغییر بده که روی هر سایتی (مثل سایت‌های خبری، بانکی، یا شبکه‌های اجتماعی) کار کنه.

ابزار Param Miner:

جیمز می‌گه سه تکنیک اصلی که توی بخش قبلی معرفی کرد (کشف سطح حمله مخفی، تزریق سمت سرور، و پیکربندی اشتباه پروکسی‌ها) توی ابزار Param Miner پیاده‌سازی شدن.

Param Miner چیه؟ یه ابزار Open Source توی Burp Suite که برای پیدا کردن پارامترهای مخفی، هدرها، و کوکی‌ها استفاده می‌شه. جیمز خودش توسعه‌دهنده اصلی این ابزاره.

اگه بخوای، می‌تونی همین الان مقاله رو ببندی و این تکنیک‌ها رو با Param Miner تست کنی.

مثال ساده: فرض کن می‌خوای ببینی یه سایت پارامتر مخفی `admin=true` رو قبول می‌کنه یا نه. Param Miner می‌تونه این پارامتر رو حدس بزنه و با تایمینگ بفهمه که آیا سرور این پارامتر رو پردازش می‌کنه یا نه.

PortSwigger/
param-miner





ارزش واقعی تحقیق

این سه تکنیک فقط به نمونه از چیزایی هستن که با حملات تایمینگ می تونی انجام بدی. حملات تایمینگ پتانسیل خیلی زیادی دارن و می تونن توی موقعیت های مختلف بهت کمک کنن.

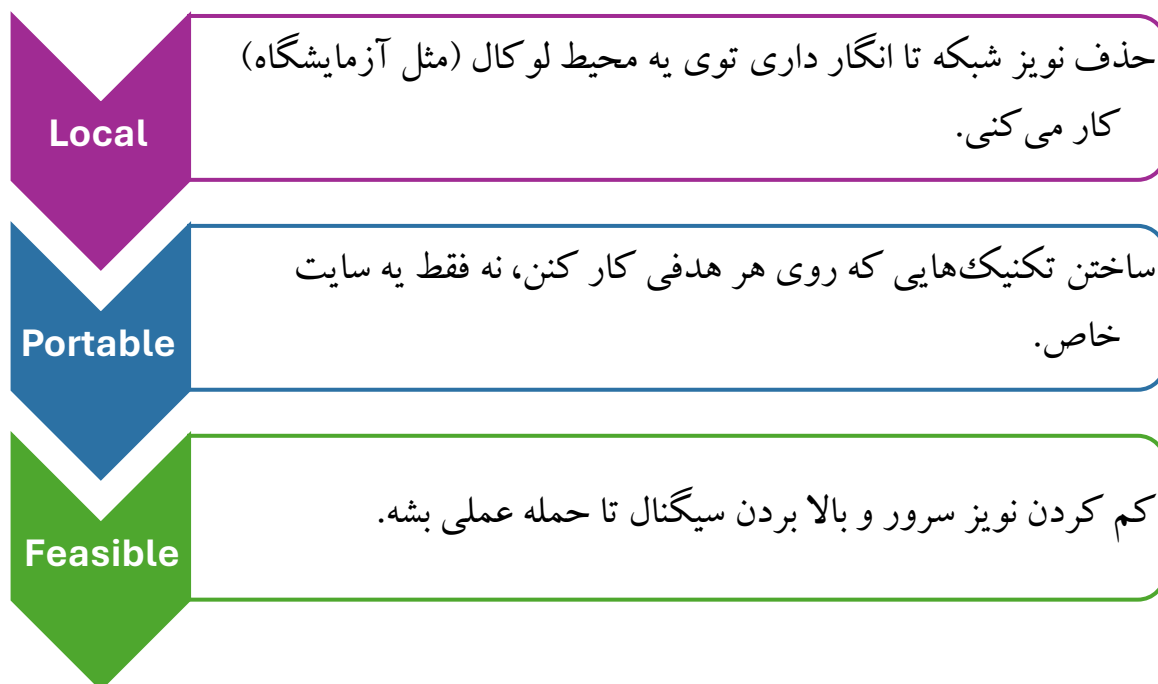
اما برای اینکه این پتانسیل رو بفهمی، باید از اول شروع کنی و ببینی چه عواملی باعث موفقیت یا شکست حملات تایمینگ توی دنیای واقعی می شن.

مثال ساده: حملات تایمینگ مثل یه جعبه ابزارن. این سه تکنیک مثل یه پیچ گوشتی، چکش، و انبردستن، ولی توی جعبه ابزارت می تونی ابزارهای دیگه ای هم بذاری و برای کارهای مختلف ازشون استفاده کنی.

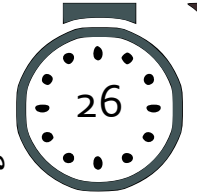
عوامل کلیدی موفقیت حملات تایمینگ

توی این بخش می خوایم عوامل کلیدی که باعث می شن حملات تایمینگ توی دنیای واقعی کار کنن یا شکست بخورن رو بررسی کنیم و راه حل هایی برای غلبه بر این مشکلات ارائه بدیم.

سه هدف اصلی داریم:



Listen to the whispers web timing attacks that actually works



مثال ساده: فرض کن می‌خواهی ببینی چه نفر توی یه شهر دیگه چه ساعتی از خواب بیدار می‌شه. اگه بخوای از راه دور حدس بزنی (مثل زنگ زدن بهش)، تأخیر شبکه (مثل قطعی خط) کار رو سخت می‌کنه (نویز). جیمز می‌خواد این تأخیر رو حذف کنه (Local)، روش رو طوری بکنه که برای هر کسی کار کنه (Portable کردن)، و مطمئن بشه که می‌تونی جواب رو سریع بگیری (امکان‌پذیر کردن).

پاسخ به سؤالات دشوار

حملات تایمینگ وب فقط برای اکسپلویت نیستن، بلکه برای پاسخ دادن به سؤالاتی هستن که نمی‌تونی از پاسخ سرور بفهمیشون. این یه دیدگاه جدیدیه که کاربردهای حملات تایمینگ رو گسترده‌تر می‌کنه.

اشتباه رایج:

خیلی‌ها فکر می‌کنن حملات تایمینگ فقط برای هک کردن یا پیدا کردن باگ‌های امنیتی (مثل دزدیدن رمزها). اما این یه اشتباهه، چون این طرز فکر باعث می‌شه کاربردهای دیگه این حملات رو نبینی.

مثال ساده: فرض کن یه سایت داری که می‌خواهی ببینی یه کاربر خاص (مثل admin) وجود داره یا نه. اگه فقط به پاسخ سرور نگاه کنی (مثلاً نام کاربری یا رمز اشتباهه)، نمی‌تونی بفهمی admin وجود داره یا نه. اما با تایمینگ می‌تونی بفهمی، چون اگه admin وجود داشته باشه، سرور برای مقایسه رمز بیشتر طول می‌کشه.

هدف اصلی حملات تایمینگ

حملات تایمینگ در اصل برای پاسخ دادن به سؤالات سختی که از پاسخ سرور (مثل کد وضعیت HTTP یا متن صفحه) نمی‌تونی بفهمیشون.

مثال ساده: سؤال اینه: آیا این سایت یه پارامتر مخفی به اسم debug داره؟ اگه سرور برای debug=true بیشتر طول بکشه (مثلاً 100ms) تا برای random=true (مثلاً 50ms)، می‌فهمی که debug وجود داره، حتی اگه پاسخ سرور همون 200 OK باشه.



تجربه جیمز با ریست رمز عبور:

جیمز تحقیقش رو با یه حمله تایمینگ روی سیستم ریست رمز عبور شروع کرد، ولی نتیجه خوبی نگرفت. اما این تجربه بهش کمک کرد که فاصله بین تئوری و عمل رو بهتر بفهمه.

سیستم ریست رمز عبور چطور کار می‌کنه؟

خیلی از سایت‌ها برای ریست رمز عبور یه توکن مخفی (مثلاً یه کد تصادفی مثل d7e8f9) توی دیتابیس ذخیره می‌کنن.

این توکن رو توی یه لینک (مثلاً `example.com/reset?token=d7e8f9`) به ایمیل کاربر می‌فرستن.

وقتی کاربر روی لینک کلیک می‌کنه، سایت توکنی که کاربر فرستاده (d7e8f9) رو با توکن توی دیتابیس مقایسه می‌کنه.

مقایسه رشته‌ها (String Comparison)

مقایسه رشته‌ها (مثل مقایسه توکن کاربر با توکن دیتابیس) معمولاً کاراکتر به کاراکتر انجام می‌شه.

اگه کاراکترها یکی باشن، مقایسه ادامه پیدا می‌کنه تا به آخر رشته برسه یا یه کاراکتر غیرهمسان پیدا کنه. هرچه کاراکترهای بیشتری همسان باشن، مقایسه بیشتر طول می‌کشه.

مثال ساده:

اگه توکن توی دیتابیس d7e8f9 باشه:

مقایسه با d7e8f9 (کاملاً همسان) تا آخر ادامه پیدا می‌کنه و بیشتر طول می‌کشه.

مقایسه با d7f (فقط دو کاراکتر اول همسان) زودتر تموم می‌شه، چون توی کاراکتر سوم (e با f) غیرهمسان می‌شن.



⌚ Time →

String comparison

d7ea



00:47

d7fa



00:46

توضیح تصویر:

تصویر به نمودار افقی با محور Time هست که مقایسه دو رشته (d7fa و d7ea) رو نشون می‌ده:

بالا: مقایسه رشته d7ea با خودش (یعنی توکن توی دیتابیس d7ea هست و توکن ارسالی هم d7ea هست):

کاراکترها: d با d (همسان)، 7 با 7 (همسان)، e با e (همسان)، a با a (همسان).

زمان: 00:47 (47 ثانیه، فرضی).

پایین: مقایسه رشته d7fa با d7ea (یعنی توکن توی دیتابیس d7ea هست و توکن ارسالی d7fa هست):

کاراکترها: d با d (همسان)، 7 با 7 (همسان)، e با f (غیرهمسان، اینجا مقایسه متوقف می‌شه).

زمان: 00:46 (46 ثانیه، فرضی).

تحلیل: چون d7ea با خودش کاملاً همسانه، مقایسه تا آخر ادامه پیدا می‌کنه و 47 ثانیه طول می‌کشه. اما d7fa توی کاراکتر

سوم غیرهمسان می‌شه و مقایسه زودتر تموم می‌شه (46 ثانیه).

نکته:



این تصویر به مثال فرضیه و زمان‌ها (47 و 46 ثانیه) فقط برای توضیح مفهوم. توی دنیای واقعی، مقایسه

کاراکترها خیلی سریع‌تره (در حد نانوثانیه) و این تفاوت‌ها توی نویز گم می‌شن.

هدف تصویر اینه که نشون بده اگه کاراکترهای بیشتری همسان باشن، مقایسه بیشتر طول می‌کشه.



این فقط به مثال فرضیه. توی دنیای واقعی، مقایسه هر کاراکتر توی CPU حدود 5 نانوثانیه (0.000000005 ثانیه) طول می کشه.

چرا مشکل داره؟ این زمان خیلی کوچیکه و توی نویز شبکه (مثلاً تأخیر 50ms) یا نویز سرور (مثلاً بار پردازشی) گم می شه. برای همین، اکسپلویت کردنش عملاً غیرممکنه.

مثال ساده: فرض کن می خوای ببینی به نفر توی یه شهر دیگه چه ساعتی بیدار می شه. اگه بخوای با زنگ زدن بفهمی و فقط 5 نانوثانیه تفاوت داشته باشه، نمی تونی تشخیص بدی، چون تأخیر خط تلفن (مثلاً 1 ثانیه) این تفاوت رو از بین می بره.

راه حل برای موفقیت حمله

برای اینکه به حمله تایمینگ واقعاً کار کنه، باید دو کار کنی:

سیگنال رو بزرگ تر کنی: یعنی تفاوت زمانی که می خوای تشخیص بدی رو بیشتر کنی (مثلاً از 5 نانوثانیه به 200 میکروثانیه برسونی).

نویز رو کم کنی: یعنی عواملی که روی زمان پاسخ تأثیر می ذارن (مثل تأخیر شبکه) رو کاهش بدی.

این بخش قراره روی این موضوع تمرکز کنیم که چطور این کار رو انجام بدیم.

مثال ساده: توی همون مثال زنگ و کنسرت، برای شنیدن زنگ باید صدای زنگ رو بلندتر کنی (سیگنال رو بیشتر کنی) و صدای کنسرت رو کم کنی (نویز رو کم کنی).

چرا تعداد اندازه گیری ها کافی نیست؟

یه روش رایج برای کم کردن نویز اینه که چندین بار اندازه گیری کنی (مثلاً 100 بار درخواست بفرستی و میانگین بگیری). اما این روش خوب مقیاس پذیر نیست.

چرا؟ اگه نویز خیلی بیشتر از سیگنال باشه (مثلاً نویز 50 میلی ثانیه و سیگنال 5 نانوثانیه)، برای اینکه سیگنال رو تشخیص بدی، باید میلیاردها بار اندازه گیری کنی.



مشکل: این تعداد اندازه گیری خیلی طول می کشه (مثلاً چند سال!) و احتمالاً تا اون موقع سایت هدف از کار افتاده یا تغییر کرده.

مثال ساده: فرض کن می خوای ببینی یه نفر توی یه شهر دیگه چه ساعتی بیدار می شه، ولی خط تلفن 10 دقیقه تأخیر تصادفی داره (نویز) و تفاوت زمانی که می خوای تشخیص بدی 5 ثانیه ست (سیگنال). اگه 100 بار زنگ بزنی، ممکنه بازم نتونی تشخیص بدی. برای تشخیص دقیق، باید میلیاردها بار زنگ بزنی که عملاً غیرممکنه.

$$\text{success} = \frac{\text{signal}}{\text{noise}}$$

مثال کلی برای فهم بهتر:

فرض کن می خوای ببینی یه سایت یه پارامتر مخفی debug داره یا نه:

درخواست 1: `example.com/?debug=true`

درخواست 2: `example.com/?random=true` اگه درخواست اول 200 میکروثانیه بیشتر طول بکشه (سیگنال)، ولی تأخیر شبکه 50 میلی ثانیه باشه (نویز)، سیگنال توی نویز گم می شه. برای موفقیت، باید سیگنال رو به 500 میکروثانیه برسونی (مثلاً با کند کردن سرور) و نویز رو به 10 میکروثانیه کم کنی (مثلاً با حذف تأخیر شبکه).



تقسیم نویز به دو بخش:

نویز (عوامل مزاحمی که روی زمان پاسخ تأثیر می‌دارن) رو می‌تونیم به دو قسمت تقسیم کنی:

Network Noise (Jitter) (نویز شبکه یا جتر): این نویز به خاطر تغییرات توی شبکه به وجود میاد، مثل تأخیر توی ارسال و دریافت بسته‌ها.

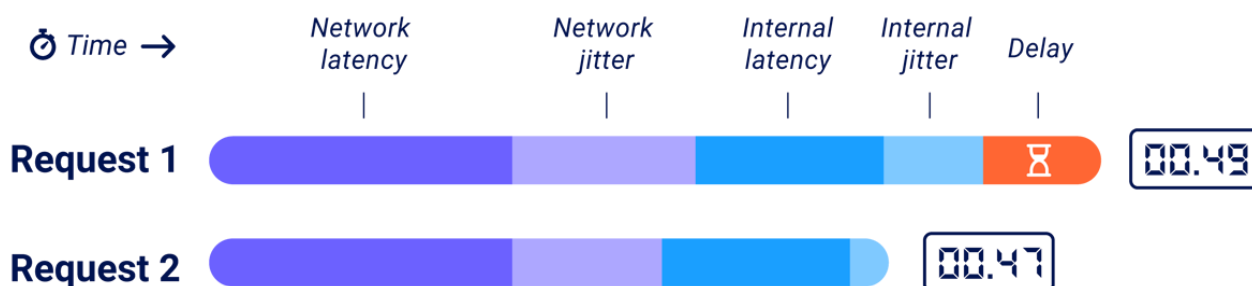
Server Noise (Internal Jitter) (نویز سرور یا جتر داخلی): این نویز به خاطر عوامل داخلی سرور به وجود میاد، مثل بار پردازشی سرور یا مشکلات سخت‌افزاری.

مثال ساده: فرض کن می‌خواهی ببینی یه نفر توی یه شهر دیگه چه ساعتی بهت زنگ می‌زنه. اگه خط تلفن گاهی تأخیر داشته باشه (مثلاً 10 دقیقه تأخیر تصادفی)، این نویز شبکه‌ست (Network Jitter). اگه خود اون نفر گاهی مشغول باشه و دیرتر زنگ بزنه (مثلاً 5 دقیقه تأخیر)، این نویز سروره (Internal Jitter).

چرا این تقسیم‌بندی مهمه؟

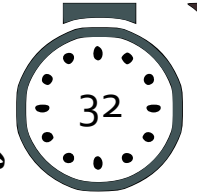
برای اینکه بتونی نویز رو کم کنی، باید بفهمی نویز از کجا میاد. نویز شبکه و نویز سرور عوامل مختلفی دارن و برای هر کدوم باید راه‌حل متفاوتی پیدا کنی.

توی ادامه این بخش قراره این دو نوع نویز رو بیشتر توضیح بدیم و راه‌حل‌هایی برای کم کردنشون ارائه بدیم.



تصویر یه نمودار افقی با محور زمان (Time) هست که دو درخواست (Request 1 و Request 2) رو نشون می‌ده و زمان پاسخشون رو با جزئیات تجزیه کرده:

Listen to the whispers web timing attacks that actually works



درخواست 1: زمان کل: 00:49 (49 ثانیه، فرضی).

اجزای زمان:

Network Latency (تأخیر شبکه): بخش بنفش تیره، زمانیه که بسته از سیستم تو به سرور می‌ره.

Network Jitter (جتر شبکه): بخش بنفش روشن، تغییرات تصادفی توی تأخیر شبکه‌ست.

Internal Latency (تأخیر داخلی): بخش آبی تیره، زمانیه که سرور درخواست رو پردازش می‌کنه.

Internal Jitter (جتر داخلی): بخش آبی روشن، تغییرات تصادفی توی پردازش سروره.

Delay: بخش نارنجی، تأخیر اضافی به خاطر یه عملیات خاص (مثلاً دسترسی به دیتابیس).

درخواست 2 (Request 2): زمان کل: 00:47 (47 ثانیه، فرضی).

اجزای زمان: همون بخش‌ها رو داره، ولی بخش نارنجی (Delay) نداره، برای همین زمانش کمتره (47 ثانیه).

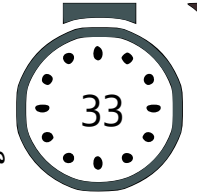
تحلیل:

تفاوت زمان: درخواست 1 به خاطر وجود بخش Delay (تأخیر اضافی) 2 ثانیه بیشتر طول کشیده (49 ثانیه در مقابل 47 ثانیه).

نویز شبکه (Network Jitter): بخش بنفش روشن نشون‌دهنده تغییرات تصادفی توی تأخیر شبکه‌ست. مثلاً ممکنه یه بار 10ms تأخیر داشته باشه، یه بار 50ms.

نویز سرور (Internal Jitter): بخش آبی روشن نشون‌دهنده تغییرات تصادفی توی پردازش سروره. مثلاً اگه سرور مشغول باشه، پردازش کندتر می‌شه.

هدف تصویر: نشون دادن اینکه زمان پاسخ یه درخواست از چند بخش تشکیل شده (تأخیر شبکه، جتر شبکه، تأخیر داخلی، جتر داخلی، و تأخیر اضافی) و نویز (جتر) توی هر دو بخش شبکه و سرور وجود داره.



مثال:

فرض کن می‌خواهی ببینی به سایت به پارامتر مخفی debug داره یا نه:

درخواست 1: `example.com/?debug=true`

درخواست 2: `example.com/?random=true` اگر درخواست اول ms200 طول بکشد و درخواست دوم

ms150، این ms50 تفاوت سیگنال توئه. اما اگر تأخیر شبکه (Network Jitter) بین ms10 تا ms100 تغییر کنه و سرور هم گاهی ms20 کندتر باشه (Internal Jitter)، این ms50 توی نویز گم می‌شه.

جتر شبکه چیه؟

جتر شبکه (Network Jitter) به تغییرات تصادفی توی تأخیر شبکه می‌گن. تأخیر شبکه (Latency) همون زمانیه که یه بسته (Packet) از سیستم تو به سرور می‌ره و برمی‌گرده.

مثال ساده: فرض کن به یه دوست توی یه شهر دیگه زنگ بزنی. اگر یه بار 1 ثانیه طول بکشد تا صدات بهش برسه، یه بار 5 ثانیه، این تغییر تصادفی (1 تا 5 ثانیه) جتر شبکه‌ست.

چرا جتر شبکه دشمن حملات تایمینگه؟

جتر شبکه بزرگ‌ترین مشکل حملات تایمینگ از راه دور (Remote Timing Attacks). چون این تغییرات تصادفی باعث می‌شن سیگنال (تفاوت زمانی که می‌خواهی تشخیص بدی) توی نویز گم بشه.

مثال ساده: اگر بخوای ببینی دوستت چه ساعتی بیدار می‌شه و همیشه 5 ثانیه دیرتر بهت زنگ بزنه (سیگنال)، ولی جتر شبکه بین 1 تا 10 ثانیه تغییر کنه (نویز)، نمی‌تونی اون 5 ثانیه رو تشخیص بدی.

چرا می‌گن توی سیستم لوکال کار می‌کنه، ولی توی سیستم راه دور نه؟

وقتی یه نفر یه حمله تایمینگ رو توی یه سیستم لوکال (Local System) تست می‌کنه (مثلاً توی یه شبکه داخلی بدون تأخیر)، جتر شبکه وجود نداره و حمله کار می‌کنه.

Listen to the whispers web timing attacks that actually works



اما وقتی می گن این توی سیستم راه دور (Remote System) کار نمی کنه، منظور اینه که جتر شبکه (مثلاً تأخیر 50ms) سیگنال رو از بین می بره.

مثال ساده: توی یه شبکه لوکال، اگه یه سرور برای یه درخواست 200ms طول بکشه و برای یه درخواست دیگه 150ms، می تونی 50ms تفاوت رو تشخیص بدی. اما توی اینترنت، اگه جتر شبکه 100ms باشه، این 50ms کم می شه.

تغییرات توی 5 سال گذشته

5 سال پیش (یعنی حدود سال 2019)، این حرف درست بود و جتر شبکه واقعاً حملات تایمینگ از راه دور رو غیرممکن می کرد. اما حالا با تکنیک های جدید (که بعداً توضیح می دیم)، می شه این مشکل رو حل کرد.

لوکال کردن حملات تایمینگ

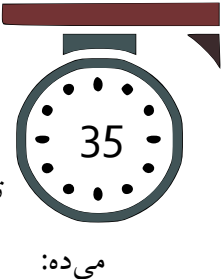
توی این بخش می خوایم نشون بدیم که چطور می تونیم حملات تایمینگ رو لوکال کنیم، یعنی طوری بکنی که انگار داری توی یه محیط لوکال (مثل یه شبکه داخلی بدون تأخیر) کار می کنی، حتی اگه از راه دور به سرور وصل باشی.

روش (2020) Timeless Timing Attacks:

توی سال 2020، مقاله ای به اسم Timeless Timing Attacks به روش جدید معرفی کرد. این روش می گه که می تونی با استفاده از پروتکل HTTP/2، نویز شبکه (جتر شبکه) رو کاملاً از اندازه گیری ها حذف کنی.

چطور کار می کنه؟

با HTTP/2 می تونی دو درخواست رو توی یه بسته TCP بذاری و همزمان به سرور بفرستی. اینطوری هر دو درخواست توی یه زمان به سرور می رسن و نویز شبکه (تغییرات تصادفی توی تأخیر) حذف می شه. بعدش می تونی ترتیب رسیدن پاسخ ها رو نگاه کنی و بفهمی کدوم درخواست بیشتر طول کشیده تا سرور پردازشش کنه.



تصویر به نمودار افقی با محور Time هست که دو درخواست (Request 1 و Request 2) رو نشون

می‌ده:

درخواست 1 (Request 1): زمان کل: 00:49 (49 ثانیه، فرضی).

اجزای زمان:

Network Latency (تأخیر شبکه): بخش بنفش تیره، زمانیه که بسته از سیستم تو به سرور می‌ره.

Network Jitter (جتر شبکه): بخش بنفش روشن، تغییرات تصادفی توی تأخیر شبکه‌ست.

Internal Latency (تأخیر داخلی): بخش آبی تیره، زمانیه که سرور درخواست رو پردازش می‌کنه.

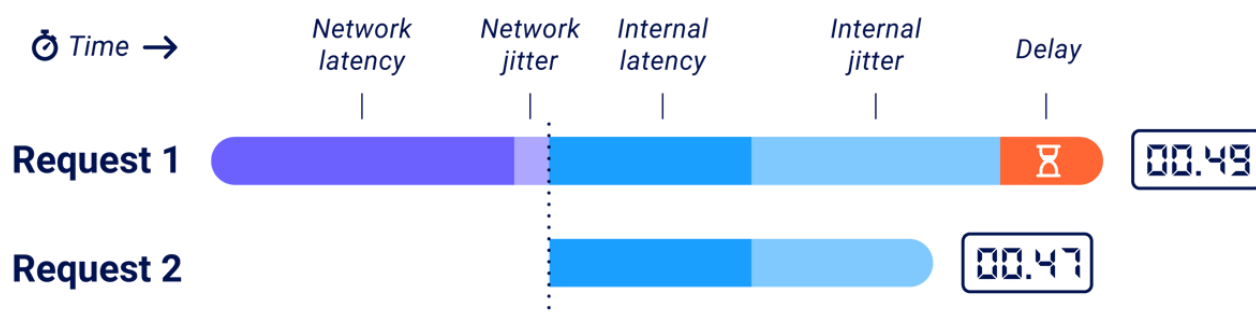
Internal Jitter (جتر داخلی): بخش آبی روشن، تغییرات تصادفی توی پردازش سرور.

Delay (تأخیر): بخش نارنجی، تأخیر اضافی به خاطر یه عملیات خاص (مثلاً دسترسی به دیتابیس).

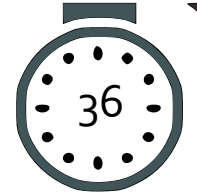
درخواست 2 (Request 2): زمان کل: 00:47 (47 ثانیه، فرضی).

اجزای زمان: همون بخش‌ها رو داره، ولی بخش نارنجی (Delay) نداره، برای همین زمانش کمتره (47 ثانیه).

نکته مهم: بخش Network Latency و Network Jitter برای هر دو درخواست یکسانه، چون هر دو توی یه بسته TCP فرستاده شدن.



Listen to the whispers web timing attacks that actually works



تصویر نشون می‌ده که با فرستادن دو درخواست توی یه بسته TCP (با استفاده از HTTP/2)، نویز شبکه (Network Jitter) حذف شده. برای همین، بخش Network Latency و Network Jitter برای هر دو درخواست (Request 1 و Request 2) یکسانه.

تفاوت زمان بین دو درخواست (49 ثانیه برای درخواست 1 و 47 ثانیه برای درخواست 2) فقط به خاطر بخش Delay (تأخیر اضافی) توی درخواست 1 هست.

این تصویر تأیید می‌کنه که اگه دو درخواست همزمان به سرور برسن، می‌تونن با نگاه کردن به ترتیب پاسخ‌ها (درخواست 2 زودتر جواب داده) بفهمی که درخواست 1 بیشتر طول کشیده.

اهمیت این روش:

این کشف (فرستادن دو درخواست توی یه بسته TCP با HTTP/2) بزرگ‌ترین منبع نویز (یعنی جتر شبکه) رو حذف کرد. این باعث شد که بتونی تفاوت‌های زمانی خیلی کوچیک‌تر رو تشخیص بدی و مرزهای قابل تشخیص تغییر کنه، یعنی چیزایی که قبلاً نمی‌تونستی ببینی، حالا قابل تشخیص شدن.

یه مشکل کوچیک:

این روش یه مشکل کوچیک داره (که بهش Catch می‌گیم). این مشکل توی بخش بعدی (Sticky Request- Order Problem) توضیح داده می‌شه. این مشکل اینه که هرچند نویز شبکه حذف شده، ولی به خاطر جریان TLS، سرور یه **تمایل** به درخواست اول داره و این می‌تونه تأخیرهایی که می‌خوای تشخیص بدی رو مخفی کنه.



مشکل ترتیب ثابت درخواست‌ها

یه مشکل توی روش قبلی (لوکال کردن حملات با HTTP/2) وجود داره که بهش Sticky Request-Order Problem می‌گن.

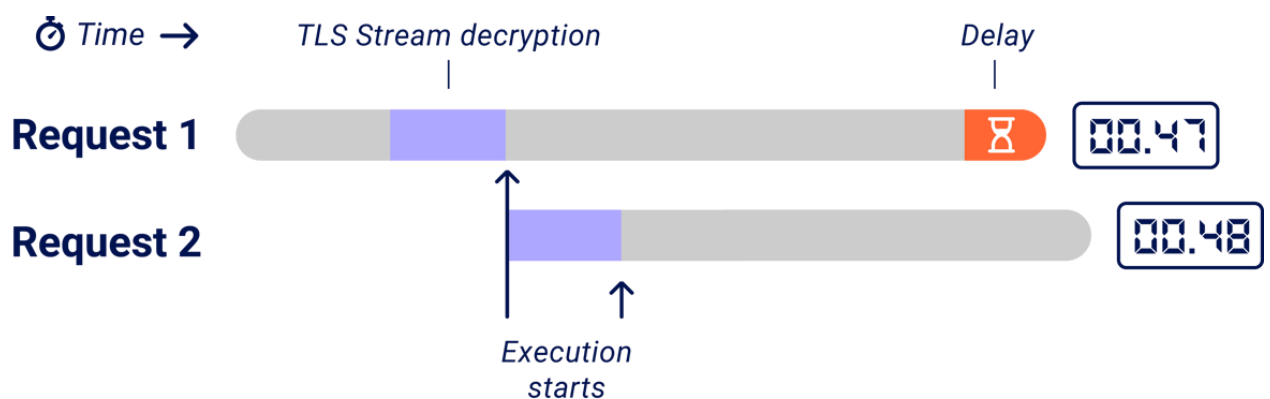
مشکل چیه؟ توی لایه HTTP/2، دو درخواست کاملاً همزمان (Concurrent) فرستاده می‌شن، ولی توی لایه پایین‌تر (TLS)، داده‌ها به صورت یه جریان (Stream) رمزگشایی می‌شن. این یعنی یه درخواست همیشه اولینه و زودتر رمزگشایی می‌شه.

اولیت‌دهی سرور: اگه این تکنیک رو تست کنی، می‌بینی که وبسایت‌ها یه **تمایل** به جواب دادن به درخواست اول دارن، یعنی معمولاً درخواست اول رو زودتر جواب می‌دن. این **تمایل** به چند دلیل ممکنه باشه: زمان رمزگشایی درخواست دوم بیشتر طول می‌کشه (چون باید منتظر تموم شدن رمزگشایی درخواست اول بمونه).

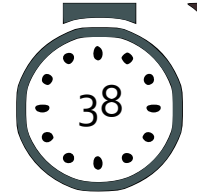
منابع سرور (مثل CPU) برای درخواست اول بیشتر در دسترسن.

چرا این مشکل بده؟ این تمایل باعث می‌شه تأخیری که می‌خوای تشخیص بدی (مثلاً یه تأخیر اضافی توی درخواست ۱) توی این تمایل گم بشه و نتونی درست تشخیص بدی.

تصویر یه نمودار افقی با محور Time هست که دو درخواست (Request 1 و Request 2) رو نشون می‌ده:



Listen to the whispers web timing attacks that actually works



درخواست 1 (Request 1): زمان کل: 00:47 (47 ثانیه، فرضی).

اجزای زمان:

TLS Stream Decryption (رمزگشایی جریان TLS): بخش بنفش، زمانیه که سرور درخواست رو رمزگشایی می‌کنه.

Execution (اجرا): بخش خاکستری، زمانیه که سرور درخواست رو پردازش می‌کنه.

Delay (تأخیر): بخش نارنجی، تأخیر اضافی به خاطر یه عملیات خاص (مثلاً دسترسی به دیتابیس).

درخواست 2 (Request 2): زمان کل: 00:48 (48 ثانیه، فرضی).

اجزای زمان: همون بخش‌ها رو داره، ولی بخش نارنجی (Delay) نداره.

فلش‌ها: نشون می‌دن که رمزگشایی (TLS Stream Decryption) برای درخواست 1 تموم شده و اجرا شروع شده، قبل از اینکه رمزگشایی درخواست 2 تموم بشه.

تصویر نشون می‌ده که هرچند درخواست 1 و 2 همزمان فرستاده شدن، ولی به خاطر جریان TLS، درخواست 1 زودتر رمزگشایی شده و اجرا رو شروع کرده.

درخواست 1 یه تأخیر اضافی (Delay) داره (بخش نارنجی)، ولی چون زودتر شروع کرده، بازم 47 ثانیه طول کشیده (سریع‌تر از درخواست 2 که 48 ثانیه طول کشیده).

این تمایل به درخواست اول باعث می‌شه تأخیر اضافی که می‌خوای تشخیص بدی، توی این تمایل گم بشه.

راه‌حل نویسندگان

نویسندگان مقاله Timeless Timing Attacks این مشکل (تمایل به درخواست اول) رو دیدن و برای حلش یه راه‌حل پیشنهاد دادن. اونا پارامترهای ساختگی (Dummy Parameters) به درخواست اول اضافه کردن تا پردازشش کندتر بشه. هدفشون این بود که اجرا دو درخواست دوباره همگام (Resynchronize) بشه و تمایل به درخواست اول از بین بره.

Portable کردن حملات تایمینگ

توی این بخش می‌خواهیم حملات تایمینگ رو Portable کنیم، یعنی تکنیک‌هایی بسازیم که روی هر هدفی (نه فقط یه هدف خاص) کار کنن و نیازی به تنظیمات خاص برای هر هدف نداشته باشن.

مشکل محیط‌های آزمایشگاهی: محیط‌های آزمایشگاهی (Lab Environments) معمولاً نویز کمتری نسبت به اهداف واقعی (Real Targets) دارن، ولی یه مشکل ظریف‌تر هم وجود داره. وقتی روی یه هدف خاص (مثل یه وب‌سایت مشخص) تمرکز می‌کنی، تکنیک‌هایی که پیدا می‌کنی معمولاً مختص همون هدفن و برای استفاده روی هدف‌های دیگه نیاز به تنظیمات زیادی (Tuning) دارن. این باعث می‌شه این تکنیک‌ها برای کسانی که با محدودیت زمانی کار می‌کنن (مثلاً باگ‌بانترها که باید سریع باگ پیدا کنن) خیلی ارزشمند نباشن.

مشکل پارامترهای ساختگی (Dummy Parameter Padding): جیمز می‌گه روش پارامترهای ساختگی (که توی بخش قبلی برای حل مشکل Sticky Request-Order Problem پیشنهاد شده بود) یه مثال از این مشکله. این روش به این بستگی داره که هدف چطور پارامترها رو پردازش می‌کنه و چقدر ظرفیت پردازشی (Processing Capacity) توی اون لحظه داره.

چون ظرفیت پردازشی سرور به سیستم‌های دیگه (مثلاً تعداد کاربرایی که همزمان دارن از سرور استفاده می‌کنن) بستگی داره، اضافه کردن پارامترهای ساختگی ممکنه به جای کم کردن نویز، نویز رو بیشتر کنه.

توی یه سیستم آزمایشگاهی، دیدم که تعداد پارامترهای موردنیاز برای کند کردن درخواست اول، توی یه بازه 10 دقیقه‌ای تغییر کرده (مثلاً یه بار 5 پارامتر کافی بوده، 10 دقیقه بعد 10 پارامتر لازم بوده).

نیاز به یه راه‌حل بهتر: ما به یه روش نیاز داریم که مشکل Sticky Request-Order رو حل کنه، ولی نیازی به تنظیمات خاص برای هر هدف نداشته باشه، یعنی یه روش کلی که روی هر سیستمی کار کنه.



معرفی Single-Packet Attack

محقق یه روش جدید به اسم Single-Packet Attack رو معرفی می کنه که خودش پارسال برای پیدا کردن Race Condition توسعه داده. این روش یه نقطه شروع خوب برای حل مشکل Sticky Request-Order بدون نیاز به تنظیمات خاصه.

این روش درخواست رو تکه تکه (Fragment) می کنه تا اندازه بسته حیاتی رو کم کنه. بسته حیاتی همون بسته ایه که درخواست رو کامل می کنه و باعث می شه سرور پردازش رو شروع کنه.

چطور کار می کنه؟

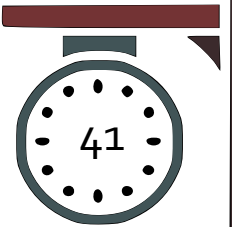
توی این روش، بخش اصلی درخواست ها (Bulk of the Requests) توی چند بسته اولیه فرستاده می شه. بعد، درخواست ها با یه بسته نهایی خیلی کوچیک کامل می شن و پردازش شروع می شه. توی نمودار بعدی، این بسته حیاتی نهایی با خط سیاه مشخص شده.

تصویر سه بسته TCP (TCP Packet) رو نشون می ده که هر کدوم شامل چند درخواست (به رنگ نارنجی و آبی) هستن:

بسته اول (چپ): یه بسته TCP که شامل 4 درخواست: دو درخواست نارنجی و دو درخواست آبی، به صورت 2×2 مرتب شدن.

بسته دوم (وسط): یه بسته TCP که بازم شامل 4 درخواست (دو نارنجی و دو آبی)، ولی این بار به صورت متفاوتی مرتب شدن (مثلاً یه درخواست نارنجی بالا، یه آبی کنارش، و همین طور ادامه داره).

بسته سوم (راست): یه بسته TCP که فقط شامل 2 درخواست (یه نارنجی و یه آبی). این بسته با خط سیاه مشخص شده و به عنوان Critical Packet معرفی شده.



تصویر نشون می‌ده که توی روش Single-Packet Attack، درخواست‌ها تکه‌تکه می‌شن.

بسته‌های اول و دوم (چپ و وسط) بخش اصلی درخواست‌ها رو می‌فرستن.

بسته سوم (راست) که با خط سیاه مشخص شده، Critical Packet هست. این بسته خیلی کوچیکه (فقط 2 درخواست داره) و درخواست‌ها رو کامل می‌کنه تا سرور پردازش رو شروع کنه.

هدف این روش اینه که با کم کردن اندازه بسته حیاتی، بتونی زمان پردازش رو دقیق‌تر کنترل کنی و مشکل Sticky Request-Order رو حل کنی.

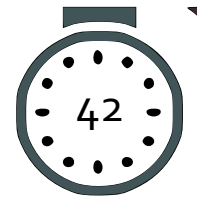


یه مشکل جدید (Catch)

روش Single-Packet Attack (که توی بخش قبلی معرفی کردیم) یه مشکل جدید به وجود میاره. بعضی از سرورها به محض اینکه هدرهای درخواست HTTP رو دریافت می‌کنن، شروع به پردازش می‌کنن و منتظر بدنه درخواست نمی‌مونن. این یعنی اگه هدرها توی بسته‌های جداگانه فرستاده بشن، سرور ممکنه پردازش یه درخواست رو زودتر از دیگری شروع کنه و این باعث می‌شه دو درخواست همزمان پردازش نشن.

راه حل:

برای حل این مشکل، باید پشته شبکه سیستم عامل رو متقاعد کنیم که فریم‌های هدر رو توی یه بسته (Single Packet) ادغام کنه. اینطوری، مهم نیست که سرور توی کدوم مرحله (بعد از هدر یا بعد از بدنه) شروع به پردازش می‌کنه، چون هر دو درخواست همزمان به سرور می‌رسن و پردازششون با هم شروع می‌شه.



هستن:

تصویر دو بسته TCP (TCP Packet) رو نشون می ده که هر کدوم شامل چند درخواست (به رنگ نارنجی و آبی)

بسته اول (چپ): یه بسته TCP که شامل 8 درخواست: 4 درخواست نارنجی و 4 درخواست آبی، به صورت 2×4 مرتب شدن. این بسته بخش اصلی درخواست ها (Bulk of the Requests) رو نشون می ده که توی بسته های اولیه فرستاده می شن.

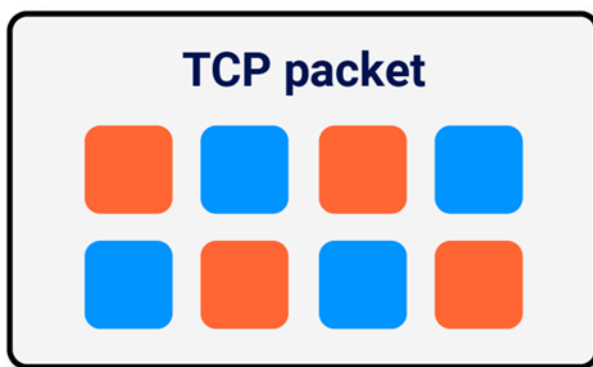
بسته دوم (راست): یه بسته TCP که فقط شامل 2 درخواست (یه نارنجی و یه آبی). این بسته همون بسته حیاتی (Critical Packet) هست که توی بخش قبلی با خط سیاه مشخص شده بود. این بسته درخواست ها رو کامل می کنه و باعث می شه سرور پردازش رو شروع کنه.

تصویر ادامه همون مفهوم Single-Packet Attack از بخش قبله.

بسته اول (چپ) بخش اصلی درخواست ها رو می فرسته (8 درخواست).

بسته دوم (راست) بسته حیاتی که فقط 2 درخواست داره و درخواست ها رو کامل می کنه تا سرور پردازش رو شروع کنه.

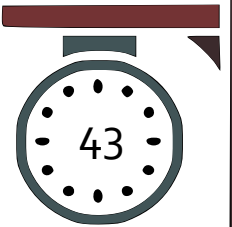
این تصویر تأیید می کنه که توی روش Single-Packet Attack، بخش اصلی درخواست ها توی یه بسته بزرگ فرستاده می شه و یه بسته کوچک (بسته حیاتی) برای کامل کردن درخواست ها و شروع پردازش استفاده می شه.



چرا فقط دو بسته حیاتی؟

ممکنه تعجب کنید که چرا من درخواست ها رو فقط به دو بسته حیاتی تقسیم کردم، به جای اینکه برای هر هدر HTTP یه بسته جداگانه بفرستم. اگه می تونستیم برای هر هدر یه بسته بفرستیم، ایده آل بود، چون کنترل بیشتری روی زمان بندی داشتیم.

Listen to the whispers web timing attacks that actually works



اما به مشکل وجود داره: استاندارد HTTP/2 (که بهش RFC می گن) اجازه نمی ده فریم های هدر از درخواست های مختلف با هم قاطی بشن. برای همین، این روش بعیده که کار کنه.

راه حل ساده برای همگام سازی دو بسته:

پیاده سازی این روش همگام سازی دو بسته (Dual-Packet Sync) خیلی آسون بود. فقط کافیه به فریم پینگ (Ping Frame) اضافی اضافه کنی!

این فریم پینگ به بسته بی ضرر و قربانی هست که باعث می شه سیستم عامل فریم های هدر بعدی رو با هم ادغام کنه. اینطوری مطمئن می شیم که هدرهای دو درخواست توی به بسته فرستاده می شن و همزمان به سرور می رسن.

غیرفعال کردن TCP_NODELAY

برای هر درخواست بدون بدنه، به فریم پینگ بفرست.

هدرها رو بفرست.

به فریم داده خالی رو نگه دار.

برای هر درخواست با بدنه: هدرها و بدنه رو بفرست، به جز آخرین بایت.

به فریم داده شامل آخرین بایت رو نگه دار.

۱۰۰ میلی ثانیه صبر کن.

به فریم پینگ بفرست.

فریم های پایانی رو بفرست.



تحلیل:

disable TCP_NODELAY: غیرفعال کردن TCP_NODELAY، به تنظیم توی پروتکل TCP که باعث می‌شه بسته‌ها سریع‌تر فرستاده بشن. غیرفعال کردنش باعث می‌شه سیستم‌عامل بسته‌ها رو بیشتر نگه داره و با هم ادغام کنه.

send a ping frame: به فریم پینگ بفرست. این همون فریم پینگ اضافیه که جیمز توی متن گفت.

for each request with no body:: برای هر درخواستی که بدنه (Body) نداره:

send the headers: هدرها رو بفرست.

withhold an empty data frame: به فریم داده خالی رو نگه دار (یعنی نفرست).

for each request with a body:: برای هر درخواستی که بدنه داره:

send the headers, and the body except the final byte: هدرها و بدنه رو بفرست، به جز بایت آخر.

withhold a data frame containing the final byte: به فریم داده که شامل بایت آخر هست رو نگه دار (یعنی نفرست).

wait for 100ms: 100 میلی‌ثانیه صبر کن.

send a ping frame: به فریم پینگ دیگه بفرست.

send the final frames: فریم‌های نهایی (مثل فریم داده خالی یا فریم بایت آخر) رو بفرست.

این به دستورالعمل عملی برای پیاده‌سازی روش Single-Packet Attack و همگام‌سازی دو بسته رو نشون می‌ده.

هدف این دستورالعمل اینه که مطمئن بشیم هدرهای دو درخواست توی به بسته فرستاده می‌شن و همزمان به سرور می‌رسن:

غیرفعال کردن TCP_NODELAY و فرستادن فریم پینگ باعث می‌شه سیستم‌عامل هدرها رو با هم ادغام کنه.

Listen to the whispers web timing attacks that actually works



نگه داشتن فریم‌های نهایی (مثل بایت آخر بدنه) و صبر کردن 100 میلی ثانیه، به سیستم عامل زمان می‌دهد تا بسته‌ها رو آماده کنه.

فرستادن فریم‌های نهایی توی آخر باعث می‌شه سرور پردازش دو درخواست رو همزمان شروع کنه.

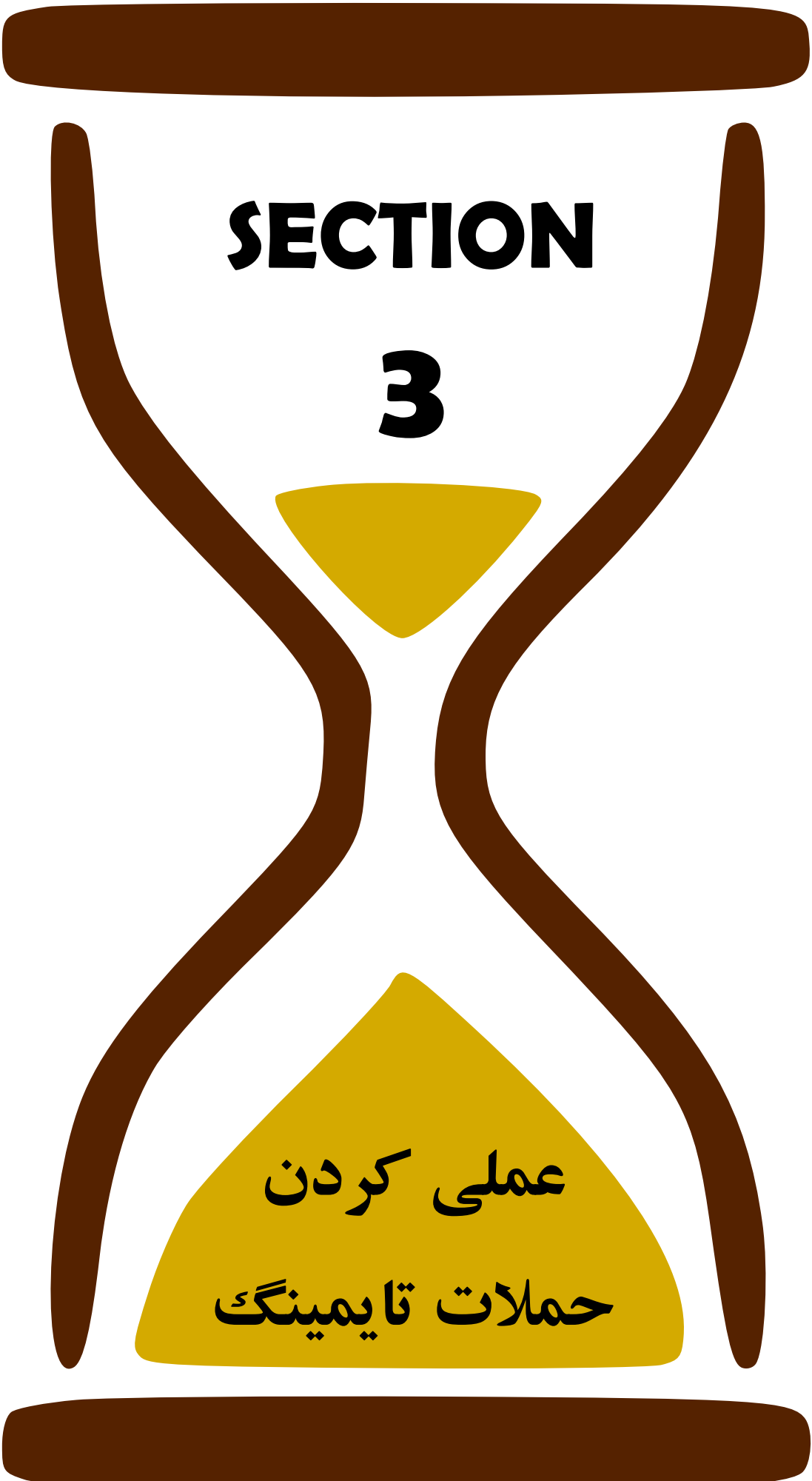
اضافه کردن به Burp Suite

ما این تکنیک بهبود یافته (Single-Packet Attack) با همگام‌سازی دو بسته (رو به محض اینکه پیداش کردیم، توی حمله تک‌بسته‌ای (Single-Packet Attack) داخلی Burp Suite ادغام کردیم. برای همین، ممکنه تو همین حالا هم ازش استفاده کرده باشی و از مزایاش بهره برده باشی!

کار روی hzspace

جیمز می‌گه در حال حاضر داریم با توسعه‌دهنده hzspace کار می‌کنیم تا این تکنیک رو توی اون ابزار هم اضافه کنیم.

hzspace یه ابزار Open Source برای کار با پروتکل HTTP/2 هست که برای تست‌های امنیتی استفاده می‌شه. (جالبه بدونید که توسعه دهنده این ابزار ایرانی هست)

An hourglass graphic with a brown frame and two yellow chambers. The top chamber is smaller than the bottom one. The word 'SECTION' is in the top chamber, the number '3' is in the neck, and Persian text is in the bottom chamber.

SECTION

3

عملی کردن
حملات تایمینگ



عملی کردن حملات تایمینگ

توی این بخش می‌خواهیم حملات تایمینگ رو عملی کنیم، یعنی راه‌هایی پیدا کنیم که این حملات توی دنیای واقعی کار کنن.

هدف بعدی: نویز سرور

حالا که نویز شبکه با روش‌های قبلی (مثل Single-Packet Attack و HTTP/2) حذف شده، هدف بعدی نویز سرور هست.

نویز سرور رو دست کم نگیرید. این نویز از منابع زیادی میاد، مثل:

- بار روی سرور هدف (مثلاً تعداد کاربرایی که دارن از سرور استفاده می‌کنن).
- سیستم‌های دیگه‌ای که سرور باهاشون تعامل داره (مثلاً دیتابیس یا API‌های خارجی).
- سیستم‌های مجازی دیگه‌ای که روی همون سخت‌افزار فیزیکی اجرا می‌شن (مثلاً سرورهای ابری).
- حتی شاید آب‌وهوای نزدیک دیتاسنتر 😂😂😂

نمی‌تونم ادعا کنم که با روش بهبودیافته Single-Packet Attack دقیقاً چه تأخیری رو می‌تونی تشخیص بدی، چون این موضوع به نویز سرور بستگی داره و نویز هر سرور با سرور دیگه فرق داره. مثلاً یه سرور ممکنه نویز کمی داشته باشه و بتونی تأخیر 200 میکروثانیه رو تشخیص بدی، ولی یه سرور دیگه ممکنه نویز زیادی داشته باشه و حتی تأخیر 50 میکروثانیه هم توی نویزش گم بشه، برای همین هر ادعایی درباره میزان تأخیر به سرور خاص بستگی داره و نمی‌تونی یه عدد کلی بگی.



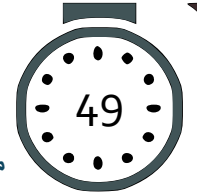
چطور نویز سرور رو کم کنیم؟

برای کم کردن نویز سرور، یعنی کاهش تغییرات تصادفی توی زمان پاسخ سرور که می تونه تشخیص تأخیرها رو سخت کنه، باید این کارها رو بکنی:

کوتاه ترین مسیر کد (Code Path) رو انتخاب کن، یعنی درخواست هایی بفرستی که سرور با کمترین مراحل پردازش جواب بده، مثلاً به جای درخواست یه صفحه پیچیده که سرور باید دیتابیس رو جستجو کنه، چند تا فایل (مثل CSS و JavaScript) رو بارگذاری کنه و محاسبات انجام بده (که باعث می شه زمان پاسخش بالا و پایین بشه)، یه درخواست ساده به یه مسیر استاتیک بفرستی (مثل یه فایل HTML که فقط یه متن سلام توش هست) تا سرور سریع و با کمترین کار اضافی جواب بده و زمان پاسخش ثابت تر بمونه.

از ویژگی های عملکردی مثل کش کردن (Caching)، یعنی ذخیره کردن پاسخ های سرور برای درخواست های تکراری، مثلاً اگه یه صفحه رو چند بار درخواست می کنی، سرور همون پاسخ قبلی رو از حافظه بده و دوباره پردازش نکنه)، استفاده دوباره از اشیا (Object Reuse)، یعنی اگه سرور برای هر درخواست یه شیء جدید می سازه، مثلاً یه شیء برای مدیریت دیتابیس، همون شیء قبلی رو دوباره استفاده کنه تا زمان ساختن شیء جدید کم بشه)، و استفاده دوباره از اتصال (Connection Reuse)، یعنی به جای اینکه برای هر درخواست یه اتصال جدید به سرور باز کنی، از همون اتصال قبلی استفاده کنی تا زمان برقراری ارتباط کمتر بشه و نویز ناشی از تأخیرهای شبکه کم بشه) نهایت استفاده رو بکن.

مهاجم های حرفه ای ممکنه از تکنیک های DoS (Denial of Service) مثل CPDoS (Cache Poisoning) Denial of Service، یعنی با دستکاری کش سرور، کاری کنن که پاسخ های اشتباه به کاربرهای دیگه برگرده و سرور مشغول بشه) یا مصرف منابع (Resource Consumption)، یعنی با فرستادن هزاران درخواست جعلی، سرور رو انقدر مشغول کنن که منابعش مثل CPU یا حافظه پر بشه) استفاده کنن تا نویز ناشی از کاربرهای دیگه رو کم کنن (مثلاً با مشغول کردن سرور، کاری کنن که کاربرهای دیگه نتونن درخواست بفرستن و سرور فقط به درخواست های مهاجم جواب بده، مثل اینکه یه نفر توی یه فروشگاه شلوغ انقدر سرو صدا کنه که بقیه مشتری ها نتونن با فروشنده حرف بزنن).



فرض کن به مهاجم می‌خواد بفهمه به وب‌سایت بانکی به پارامتر مخفی به اسم `debug=true` داره یا نه. برای اینکه نویز سرور کم بشه و بتونه تأخیرهای کوچیک (مثلاً 200 میکروثانیه) رو تشخیص بده، اول به درخواست ساده به مسیر استاتیک مثل `http://bank.com/static/test.html` می‌فرسته (که فقط به فایل HTML ساده رو برمی‌گردونه و مسیر کد کوتاهی داره). بعد با استفاده از کش کردن، مطمئن می‌شه که سرور پاسخ‌های تکراری رو سریع‌تر برگردونه (مثلاً اگه چند بار همون فایل رو درخواست کنه، سرور از کش استفاده کنه). در نهایت، برای کم کردن نویز ناشی از کاربرهای دیگه، مهاجم به حمله CPDoS انجام می‌ده و کش سرور رو دستکاری می‌کنه تا کاربرهای دیگه پاسخ‌های اشتباه بگیرن و از سرور خارج بشن (مثلاً به صفحه خطا ببینن و دیگه درخواست نفرستن). حالا مهاجم دو درخواست می‌فرسته: یکی با `http://bank.com/static/test.html?debug=true` و یکی با `http://bank.com/static/test.html?random=true`. اگه درخواست اول 200 میکروثانیه بیشتر طول بکشه (چون سرور داره `debug` رو پردازش می‌کنه)، مهاجم می‌فهمه که این پارامتر مخفی وجود داره، و چون نویز سرور رو کم کرده، این تأخیر کوچیک رو تونسته تشخیص بده.

چطور سیگنال رو زیاد کنیم؟

برای زیاد کردن سیگنال، یعنی افزایش تفاوت زمانی که می‌خوای توی پاسخ‌های سرور تشخیص بدی تا راحت‌تر بفهمی سرور چه رفتاری داره (مثلاً آیا به پارامتر مخفی رو پردازش می‌کنه یا نه)، باید روی مسیرهای کند کد تمرکز کنی و اونا رو حتی کندتر کنی، یعنی مسیرهایی رو پیدا کن که سرور برای پردازششون زمان بیشتری می‌بره و با ترفندهایی این زمان رو بیشتر کنی:

از ورودی‌های تصادفی استفاده کن تا کش سمت سرور رو دور بزنی، یعنی با فرستادن مقادیر تصادفی توی درخواست‌ها (مثلاً به جای اینکه همیشه `id=1?` بفروستی، هر بار به مقدار تصادفی مثل `id=abc123?` یا `id=xyz789?` بفروستی)، کاری کنی که سرور نتونه از کش استفاده کنه (چون سرور معمولاً پاسخ‌های تکراری رو کش می‌کنه تا سریع‌تر جواب بده، مثلاً اگه `id=1?` رو چند بار بفروستی، سرور همون پاسخ قبلی رو از کش برمی‌گردونه و سریع جواب می‌ده، ولی با ورودی تصادفی، سرور مجبور می‌شه هر بار درخواست رو از اول پردازش کنه و کندتر بشه).



دسترسی به منابع کند (Slow Resources) رو ایجاد کن، مثلاً درخواست‌هایی بفرست که سرور مجبور بشه به یه دیتابیس کند وصل بشه یا یه فایل بزرگ رو از یه سرور خارجی لود کنه (مثلاً اگه سرور برای یه درخواست خاص باید یه جدول بزرگ توی دیتابیس رو جستجو کنه، یا یه فایل 10 مگابایتی رو از یه سرور کند دانلود کنه، این کار زمان پردازش رو بیشتر می‌کنه و سیگنال رو قوی‌تر می‌کنه).

حجم کار رو چند برابر کن، یعنی درخواست‌هایی بفرست که سرور رو بیشتر مشغول کنن و زمان پردازش رو افزایش بدن (مثلاً اگه سرور یه پارامتر search رو می‌گیره و باهاش یه جستجو توی دیتابیس انجام می‌ده، یه ورودی پیچیده مثل:

?search=a* OR b* OR c*

بفرست که سرور مجبور بشه کلی رکورد رو بررسی کنه، یا اگه سرور یه پارامتر data رو می‌گیره و باهاش یه محاسبه سنگین انجام می‌ده، یه داده بزرگ و پیچیده بفرست که سرور بیشتر مشغول بشه).

مثال: یه درخواست نشون داده شده که چند هدر با پیشوند ثابت (Fixed Prefix) داره تا تأخیر سرور رو بیشتر کنه. این درخواست این شکلیه:

```
GET / HTTP/1.1
X-Uaa: a
X-Ubb: a
X-Ucc: a
{256}
```

این درخواست چند هدر با پیشوند X-U- داره (مثل X-Uaa، X-Ubb، X-Ucc) و تعدادشون 256 تاست (256 هدر). هدف اینه که سرور رو مشغول کنه، چون سرور باید همه این هدرها رو بررسی کنه و این باعث تأخیر می‌شه.

Listen to the whispers web timing attacks that actually works



مثال واقعی توی حمله

فرض کن می‌خواهی بفهمی به وب‌سایت فروشگاه‌های به پارامتر مخفی به اسم `discount=true` دارد یا نه. می‌دونی که سرور اگر این پارامتر رو ببینه، به تخفیف اعمال می‌کنه و این کار به تأخیر 300 میکروثانیه‌ای ایجاد می‌کنه (چون باید به محاسبه تخفیف انجام بده). برای زیاد کردن این سیگنال (تأخیر 300 میکروثانیه‌ای)، اول به مسیر `http://shop.com/search?query=phone` که سرور پیدا می‌کنی، مثلاً صفحه جستجوی محصولات (که سرور برای پردازشش باید به جدول بزرگ توی دیتابیس رو جستجو کنه (به مسیر کنه). بعد برای دور زدن کش، هر بار به مقدار تصادفی به درخواست اضافه می‌کنی (مثلاً `http://shop.com/search?query=phone&rand=abc123` و دفعه بعد `rand=xyz789`). برای کندتر کردن بیشتر، به منبع کنه رو هدف می‌دی، مثلاً توی کوئری جستجو به عبارت پیچیده می‌فرستی (`?query=phone* AND category=electronics OR brand=samsung`) که سرور مجبور بشه کلی رکورد رو بررسی کنه. در نهایت، حجم کار رو زیاد می‌کنی، مثلاً به درخواست با داده بزرگ می‌فرستی (`?query=` و بعد به رشته خیلی طولانی) که سرور رو مشغول کنه. حالا دو درخواست می‌فرستی یکی با:

<http://shop.com/search?query=phone&discount=true&rand=abc123>

و یکی با:

<http://shop.com/search?query=phone&rand=abc123>

اگر درخواست اول 800 میکروثانیه طول بکشه (300 میکروثانیه برای تخفیف + 500 میکروثانیه برای مسیر کنه) و درخواست دوم 500 میکروثانیه، تفاوت زمانی (سیگنال) به 300 میکروثانیه می‌رسه که به راحتی قابل تشخیصه.



فناوری‌های مدرن و تأخیر

فناوری‌های وب مدرن مثل ORM ها (Object-Relational Mapping)، مثلاً Django ORM که به ابزاره برای ارتباط بین کد برنامه و دیتابیس، یعنی به جای نوشتن کوئری‌های SQL خام، با کد پایتون دیتابیس رو مدیریت می‌کنی، مثلاً توی Django می‌تونی بنویسی `User.objects.filter(name=Ali)` و این خودش به کوئری SQL می‌سازه) و GraphQL (به زبان کوئری برای API ها که بهت اجازه می‌ده فقط داده‌هایی که نیاز داری رو از سرور بگیری، مثلاً بتونی بگی فقط نام و ایمیل کاربرها رو بده، ولی می‌تونه کوئری‌های پیچیده‌ای داشته باشه که سرور رو مشغول کنه) خیلی مناسب برای تکنیک‌های افزایش تأخیر، یعنی می‌تونی از این فناوری‌ها استفاده کنی تا زمان پردازش سرور رو بیشتر کنی و تفاوت زمانی (سیگنال) رو توی حملات زمان‌بندی بزرگ‌تر کنی.

جیمز می‌گه یادت باشه که به حمله DoS (Denial of Service)، یعنی حمله‌ای که سرور رو انقدر مشغول یا کند کنه که نتونه به کاربرهای عادی جواب بده، مثلاً با فرستادن هزاران درخواست جعلی، سرور رو از کار بندازی) خودش به حمله تایمینگ خیلی آسونه، چون وقتی سرور کند می‌شه، زمان پاسخش بیشتر می‌شه و این تفاوت زمانی رو می‌تونی توی حملات زمان‌بندی استفاده کنی (مثلاً اگه به پارامتر خاص باعث کنی بیشتر بشه، می‌تونی بفهمی که سرور اون پارامتر رو پردازش می‌کنه). می‌تونی از تکنیک‌های کلاسیک DoS مثل REDOS (Regular Expression Denial of Service)، یعنی با فرستادن به رشته پیچیده که سرور بخواد با به الگوی منظم (Regular Expression) پردازشش کنه، مثلاً به رشته مثل `^a^a^a...^a^a^a` بفرستی که سرور انقدر مشغول پردازش بشه که کند بشه)، دسته‌بندی (Batching)، یعنی چند تا درخواست سنگین رو با هم بفرستی، مثلاً توی به درخواست GraphQL، بخوای اطلاعات 1000 کاربر رو با کلی جزئیات بگیری که سرور مجبور بشه کلی کوئری دیتابیس اجرا کنه)، و موجودیت‌های XML بازگشتی (Recursive XML Entities)، یعنی توی به فایل XML، موجودیت‌هایی تعریف کنی که به صورت بازگشتی (Recursive) همدیگه رو صدا کنن، مثلاً به موجودیت `&entity1;` که خودش به `&entity2;` اشاره کنه و این زنجیره ادامه پیدا کنه، که باعث می‌شه سرور موقع پردازش XML انقدر مشغول بشه که کند بشه) استفاده کنی و اونا رو برای حملات تایمینگ تطبیق بدی، یعنی از این تکنیک‌ها برای کند کردن سرور استفاده کنی تا تفاوت زمانی بین درخواست‌ها بیشتر بشه و راحت‌تر بتونی رفتار سرور رو تشخیص بدی.



Hidden attack surface

توی این بخش می‌خواهیم درباره Hidden Attack Surface صحبت کنیم، یعنی جاهایی که آسیب‌پذیری‌ها معمولاً پنهان و دیده نمی‌شن.

آسیب‌پذیری‌های مخفی:

آسیب‌پذیری‌ها معمولاً توی ویژگی‌های بلااستفاده و فراموش‌شده پنهان که هم توسعه‌دهنده‌ها و هم تست‌کننده‌های امنیتی ازشون غافل می‌شن. برای همین، پیدا کردن آسیب‌پذیری‌ها اغلب با پیدا کردن یه پارامتر مخفی، کوکی، یا هدر HTTP شروع می‌شه.

چطور این ورودی‌های مخفی رو پیدا کنیم؟

اساس پیدا کردن این ورودی‌های مخفی اینه که اسم‌های احتمالی پارامترها رو حدس بزنی و ببینی که آیا این پارامترها روی پاسخ سرور تغییری ایجاد می‌کنن یا نه.

پارامترهایی که پاسخ رو تغییر ندن ممکنه کشف نشده بمونن، و به همراهشون هر آسیب‌پذیری‌ای که بهشون مربوط باشه هم دیده نشه.

جیمز می‌گه برای اولین حمله تایمینگ انبوه (Bulk Timing-Attack) که انجام دادم، تصمیم گرفتم این مشکل رو حل کنم.

معرفی Param Miner

جیمز می‌گه خوشبختانه من توسعه‌دهنده اصلی ابزار Param Miner هستم، که احتمالاً اولین ابزار برای پیدا کردن انبوه پارامترها بوده.

Param Miner پاسخ‌ها رو با استفاده از ویژگی‌هایی مثل تعداد کلمات، وضعیت (Status)، و تعداد خطوط (Line Count) مقایسه می‌کنه.

برای این تحقیق، من فقط Response Time رو به‌عنوان یه ویژگی اضافی به Param Miner اضافه کردم، تعداد تکرارها (Repeat Count) رو بیشتر کردم، و اسکن رو شروع کردم.

Listen to the whispers web timing attacks that actually works



چرا از Single-Packet Attack استفاده نکردم؟

جیمز می‌گه می‌تونستم Param Miner رو طوری تغییر بدم که از روش Single-Packet Attack برای این اندازه‌گیری‌ها استفاده کنه، ولی این کار نیاز به بازنویسی و تغییرات زیادی داشت.

چون داشتم روی مفاهیم اثبات نشده تحقیق می‌کردم، ترجیح دادم هر میان‌بر ممکن رو استفاده کنم تا وقت تلف نکنم، برای همین این کار رو نکردم.

روش جایگزین برای اندازه‌گیری زمان:

به‌جای استفاده از Single-Packet Attack که یه روش پیچیده برای اندازه‌گیری زمان توی حملات زمان‌بندی هست، یه روش ساده‌تر انتخاب کردم:

زمان رو از آخرین بایت درخواست (Last Byte of the Request)، یعنی لحظه‌ای که آخرین بخش از درخواست، مثلاً یه درخواست HTTP، به سرور می‌رسه تا اولین بایت پاسخ (First Byte of the Response)، یعنی لحظه‌ای که سرور اولین بخش از پاسخش، مثلاً اولین بایت از یه صفحه HTML، رو برمی‌گردونه اندازه‌گیری کردم، که به این می‌گن Time to First Byte یا TTFB، و یه روش رایج برای اندازه‌گیری زمان پاسخ سروره.

بعد، دو مجموعه از 30 اندازه‌گیری زمانی (Timing Measurements)، یعنی 30 بار زمان رو برای هر مجموعه اندازه‌گیری کردم، یه مجموعه با پارامتر موردنظر و یه مجموعه بدون اون پارامتر (رو مقایسه کردم، مثلاً یه بار 30 درخواست با پارامتر debug=true فرستادم و یه بار 30 درخواست بدون این پارامتر. برای این مقایسه، از چارک پایین (Bottom Quartile)، یعنی 25 درصد پایین این اندازه‌گیری‌ها، یا به عبارتی، 7-8 تا از سریع‌ترین زمان‌ها توی هر مجموعه 30 تایی) این اندازه‌گیری‌ها استفاده کردم تا ببینم آیا این دو مجموعه از هم جدا هستن (Distinct)، یعنی زمان‌های دو مجموعه کاملاً متفاوت باشن و هیچ هم‌پوشانی‌ای نداشته باشن، مثلاً یه مجموعه همه زمان‌هاش بین 100 تا 150 میلی‌ثانیه باشه و مجموعه دیگه بین 300 تا 350 میلی‌ثانیه یا هم‌پوشانی دارن (Overlapped)، یعنی زمان‌های دو مجموعه با هم تداخل داشته باشن، مثلاً هر دو مجموعه زمان‌هاشون بین 100 تا 200 میلی‌ثانیه باشه).



اگر جدا باشند، یعنی پارامتر معتبره (Valid Parameter)، مثلاً پارامتر `debug=true` واقعاً روی سرور تأثیر گذاشته و باعث شده زمان پاسخش بیشتر بشه) و روی زمان پاسخ تأثیر گذاشته. اگر هم پوشانی داشته باشند، یعنی پارامتر تأثیری نداشته (مثلاً فرستادن `debug=true` هیچ فرقی توی زمان پاسخ ایجاد نکرده و سرور این پارامتر رو نادیده گرفته).

چارک پایین برای این مقایسه ایده آله، چون اندازه گیری هایی رو نشون می ده که کمترین نویز (Least Noise)، یعنی کمترین تغییرات تصادفی توی زمان پاسخ، مثلاً تأخیرهای ناشی از شلوغی شبکه، بار سرور، یا مشکلات موقتی) رو دارن (یعنی تأخیرهای تصادفی مثل نویز شبکه یا سرور توی این اندازه گیری ها کمتره، چون سریع ترین زمان ها معمولاً توی شرایط ایده آل ثبت می شن و نویز کمتری دارن، مثلاً وقتی شبکه شلوغ نیست یا سرور مشغول نیست).

Discovery overload

وقتی Param Miner روی یه مجموعه آزمایشی با 30,000 سایت فعال اجرا شد، تعداد زیادی پارامتر مخفی پیدا شد، از جمله چندتا پارامتر خیلی عجیب.

کشف اول: هدر `commonconfig`

یکی از یافته های جالب، یه وب سرور بود که وقتی هدر HTTP به اسم `commonconfig` توی درخواست بود، 5 میلی ثانیه بیشتر طول می کشید تا جواب بده، مگه اینکه مقدار این هدر یه JSON معتبر باشه:

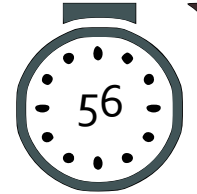
اگر هدر `foo: x` باشه: پاسخ `HTTP/1.1 200 OK` با زمان 50 میلی ثانیه.

اگر هدر `commonconfig: x` باشه: پاسخ `HTTP/1.1 200 OK` با زمان 55 میلی ثانیه (5 میلی ثانیه تأخیر بیشتر).

اگر هدر `commonconfig: {}` باشه (یه JSON معتبر): پاسخ `HTTP/1.1 200 OK` با زمان 50 میلی ثانیه (برمی گرده به حالت عادی).

این نشون می ده که سرور هدر `commonconfig` رو می شناسه و اگر مقدارش JSON معتبر نباشه، پردازشش بیشتر طول می کشه.

Listen to the whispers web timing attacks that actually works



Header	Response	Time
foo: x	HTTP/1.1 200 OK	50ms
commonconfig: x	HTTP/1.1 200 OK	55ms
commonconfig: {}	HTTP/1.1 200 OK	50ms

سروری که اتصال رو قطع می‌کنه

یه کشف دیگه روی یه وب‌سرور بود که به هیچ درخواستی جواب نمی‌داد و همیشه اتصال رو قطع می‌کرد. این رفتار دفاعی خیلی شدید بود، ولی بازم نتونست جلوی اسکن رو بگیره. اسکن نشون داد که این سرور یه هدر HTTP خاص رو پشتیبانی می‌کنه، چون وقتی اون هدر توی درخواست بود، قطع کردن اتصال خیلی بیشتر طول کشید:

اگه هدر foo: x باشه: پاسخ --connection reset-- با زمان 30 میلی‌ثانیه.

اگه هدر authorization: x باشه: پاسخ --connection reset-- با زمان 50 میلی‌ثانیه (20 میلی‌ثانیه تأخیر بیشتر).

این کشف جالبه، ولی خیلی مفید نیست، چون سرور کلاً جواب نمی‌ده.

Header	Response	Time
foo: x	--connection reset--	30ms
authorization: x	--connection reset--	50ms

کشف سوم: یه یافته کاربردی‌تر

یه یافته خیلی کاربردی‌تر هم بود:

Listen to the whispers web timing attacks that actually works



Request	Response	Time
GET /?id=random	HTTP/1.1 200 OK	310ms
GET /?foo=random	HTTP/1.1 200 OK	22ms

درخواست GET /?id=random: پاسخ HTTP/1.1 200 OK با زمان 310 میلی ثانیه.

درخواست GET /?foo=random: پاسخ HTTP/1.1 200 OK با زمان 22 میلی ثانیه.

این دو پاسخ دو چیز مهم رو نشون می دن:

سایت فقط پارامترهای خاصی مثل id رو توی کلید کش (Cache Key) در نظر می گیره، برای همین خیلی در برابر حملات مسموم سازی کش مبتنی بر پارامتر (Parameter-Based Cache Poisoning) آسیب پذیره.

پارامتر id توی کل سایت استفاده می شه و این پیکربندی معمولاً توی کل سایت اعمال می شه. این یعنی با تحلیل زمان، Param Miner تونسته یه پارامتر رو پیدا کنه که به یه صفحه دیگه مربوط می شه!

سخت ترین مشکل

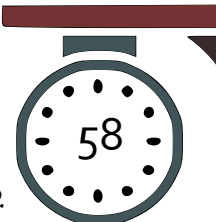
وقتی این مفهوم (حملات تایمینگ) رو آزمایش کردم، دو مشکل رو پیش بینی می کردم. اول، فکر می کردم خیلی از تکنیک ها کاملاً شکست بخورن. دوم، حدس می زدم که هر نتیجه معتبری که پیدا کنم، توی یه انبوه از نتایج اشتباه (False Positives) گم بشه.

بزرگ ترین چالش:

اما بزرگ ترین چالش نه از این دوتا بود. مشکل اصلی اینه که حملات تایمینگ خیلی قدرتمندن. این حملات می تونن چیزهای زیادی رو تشخیص بدن، و به همین خاطر خیلی آسونه که اشتباه بفهمی چی رو تشخیص دادی. این حملات واقعاً خوبن توی تشخیص دادن یه چیزی، ولی اون چیزی که تشخیص می دن، لزوماً همون چیزی نیست که تو دنبالش هستی.

https://www.youtube.com/watch?v=tZx_dmcOfqk

Listen to the whispers web timing attacks that actually works



یه ویدیو نشون داده شده که توی اون، اول به نظر میاد یه آسیب پذیری Remote Code Execution

پیدا شده، چون پارامتر exec باعث یه تأخیر قابل مشاهده توی پاسخ می شه.

ولی بعد معلوم می شه که این تأخیر به خاطر یه فایروال برنامه وب هست که روی درخواست های مشکوک تر پردازش اضافی انجام می ده.

بعدش می بینیم که اگه پارامتر exec تکرار بشه، این تأخیر بیشتر می شه (تأخیرها روی هم انباشته می شن)، مگه اینکه اندازه بدنه درخواست (Request Body) از یه حد خاص بیشتر بشه.

در نهایت، این تحلیل به کشف یه روش دور زدن کامل WAF (WAF Bypass) منجر می شه. این کشف برای من کاملاً غیرمنتظره بود، ولی بعداً توسط بقیه هم پیدا شد و حالا توی ابزار nowafpls پیاده سازی شده.

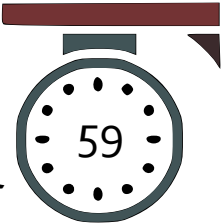
این یه نمونه قشنگ از اینه که تحلیل زمان (Timing Analysis) چطور می تونه جریان کنترلی (Control Flow) هدف رو نشون بده.

هشدار درباره نتایج:

این یکی از موارد ساده بود. گاهی ممکنه هیچ وقت کامل نفهمی چی رو تشخیص دادی. باید فرضیات رو با احتیاط در نظر بگیری و تا جایی که می تونی از زوایای مختلف آزمایششون کنی.

Proving the concept

برای اینکه با فرضیات اشتباه گمراه نشم، تصمیم گرفتم روی پارامترهای خاصی تمرکز کنم که تأثیر امنیتی واضحی دارن، بدون اینکه نیاز به تحقیقات دستی زمان بر باشه و بتونم به راحتی شواهد بیشتری برای تأییدشون جمع کنم.



جعل آدرس IP از طریق هدرهای HTTP:

جعل آدرس (IP Address Spoofing) از طریق هدرهای HTTP دقیقاً این شرایط رو داشت. این به اشتباه پیکربندی نسبتاً رایجه و مستقیماً به مهاجم اجازه می‌ده از exploits مختلفی استفاده کنه، مثل دور زدن ریت لیمیت (Rate-Limit Bypasses)، جعل لاگ‌ها، و حتی توی بعضی موارد دور زدن کنترل دسترسی.

وقتی به آدرس IP رو توی به هدر جعلی سمت فرانت‌اند قرار می‌دی، عملاً داری خودت رو به عنوان فرانت‌اند جا می‌زنی. بعداً حملات جعل فرانت‌اند رو با جزئیات بیشتری بررسی می‌کنیم.

تشخیص با DNS Lookup:

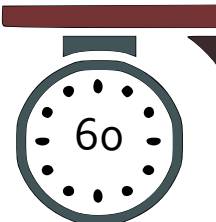
نکته جالب اینه که اگه به دامنه رو توی به هدر جعلی بذاری، سرورهای آسیب‌پذیر معمولاً به جستجوی DNS داخلی انجام می‌دن تا اون دامنه رو Resolve کنن، و این باعث به تأخیر قابل تشخیص می‌شه. به نمونه تشخیص معمولی این شکلیه:

اگه هدر Random-header: xyz.example.com باشه: زمان پاسخ 65 میلی ثانیه (چون DNS Lookup انجام نمی‌شه).

اگه هدر True-Client-IP: xyz.example.com باشه: زمان پاسخ 70 میلی ثانیه (چون سرور به DNS Lookup برای xyz.example.com انجام می‌ده و این باعث تأخیر می‌شه).

اگه دوباره هدر True-Client-IP: xyz.example.com باشه: زمان پاسخ 65 میلی ثانیه (چون پاسخ DNS کش شده و دیگه نیازی به جستجوی دوباره نیست).

Header	Time
Random-header: xyz.example.com	65ms
True-Client-IP: xyz.example.com	70ms
True-Client-IP: xyz.example.com	65ms

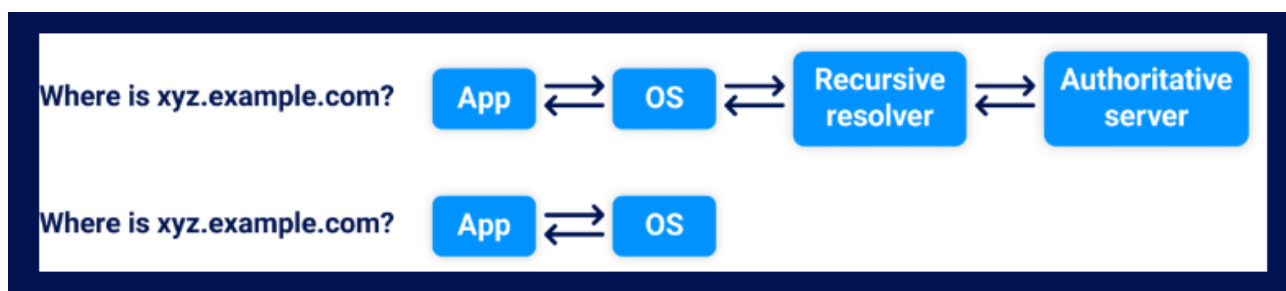


جریان DNS Lookup:

جریان این فرآیند این جوریه:

درخواست اول: برنامه (App) از سیستم عامل (OS) می پرسه xyz.example.com کجاست؟ سیستم عامل از یه Recursive Resolver می پرسه، و Resolver از سرور معتبر (Authoritative Server) جواب رو می گیره. این باعث تأخیر می شه.

درخواست دوم: برنامه دوباره از سیستم عامل می پرسه، ولی چون جواب قبلاً کش شده، سیستم عامل مستقیم جواب می ده و نیازی به Resolver یا سرور معتبر نیست. برای همین سریع تره.



بعداً بیشتر درباره کش DNS صحبت می کنیم. اما در کل، اسکن برای جعل آدرس IP این نتایج رو نشون داد:

375 دامنه آسیب پذیر پیدا شد.

206 تا از این دامنه ها یه DNS Pingback هم ایجاد کردن (یعنی یه درخواست DNS به سرور مهاجم برگشت).

217 تا از این دامنه ها به صورت واضح نتیجه DNS رو کش کردن (یعنی تأخیر توی درخواست های بعدی کم شد).



آیا نتایج اشتباه بودن؟

از 375 دامنه آسیب پذیری که توی بخش قبلی پیدا شدن، حدود 170 تاشون DNS Pingback ایجاد نکردن (یعنی درخواست DNS به سرور مهاجم برنگشت).

این ممکنه این سؤال رو پیش بیا ره که آیا این 170 تا نتایج اشتباه بودن؟ یه مثال آورده شده تا این موضوع رو توضیح بده:

اگه هدر Random-header: x.psres.net باشه: زمان پاسخ 170 میلی ثانیه.

اگه هدر True-Client-IP: x.psres.net باشه: زمان پاسخ 90 میلی ثانیه (خیلی سریع تر).

اگه هدر True-Client-IP: 1.1.1.1 باشه: زمان پاسخ 170 میلی ثانیه (برمی گرده به حالت عادی).

Header	Time
Random-header: x.psres.net	170ms
True-Client-IP: x.psres.net	90ms
True-Client-IP: 1.1.1.1	170ms

سؤال: اینجا چه اتفاقی داره می افته؟

برای فهمیدن این موضوع، یه سرنخ داده شده: توی تاریخچه ورود سایت، آدرس IP ورود و موقعیت مکانی مشخص شده:

5 دقیقه پیش، مرورگر Chrome توی ویندوز، با IP 1.1.1.1 و موقعیت مکانی Cloudflare.

Time	Browser	IP	Location
5 minutes ago	Chrome in Windows	1.1.1.1	Cloudflare

Listen to the whispers web timing attacks that actually works



تحلیل اتفاقی که افتاده:

به نظر میاد سیستم، آدرس IP جعلی (Spoofed IP) رو به یه کتابخونه فرستاده، و این کتابخونه فرمت IP رو بررسی کرده قبل از اینکه اون رو به یه سرویس Geolookup (جستجوی موقعیت جغرافیایی) بفرسته.

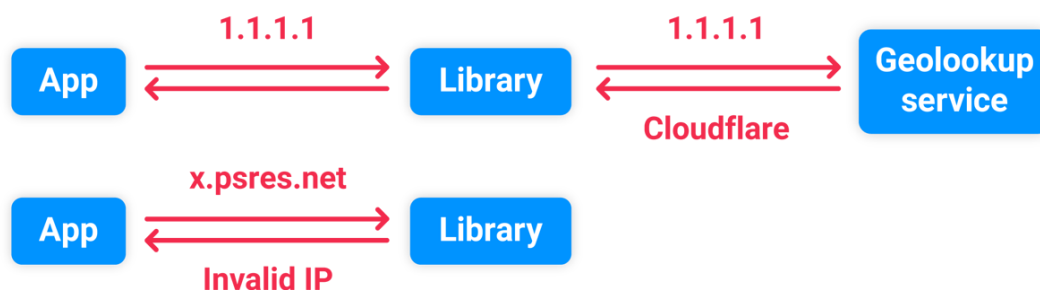
وقتی یه IP نامعتبر مثل x.psres.net فرستاده می شه، یه خطا ایجاد می شه و این باعث می شه جستجوی کند IP (IP-Lookup) انجام نشه. برای همین زمان پاسخ 90 میلی ثانیه ست (سریع تر).

اما وقتی یه IP معتبر مثل 1.1.1.1 فرستاده می شه، جستجوی Geolookup انجام می شه و زمان پاسخ به 170 میلی ثانیه می رسه (کند تر).

جریان این فرآیند این جوریه:

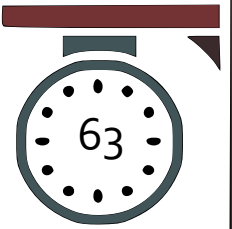
برای IP معتبر (1.1.1.1): برنامه به کتابخونه می فرسته، و کتابخونه به سرویس Geolookup می فرسته، و جواب (مثلاً Cloudflare) برمی گرده.

برای IP نامعتبر (x.psres.net): برنامه به کتابخونه می فرسته، ولی کتابخونه خطای Invalid IP می ده و دیگه به سرویس Geolookup نمی ره.



با این کار، یه تکنیک جدید برای پیدا کردن پارامترها به دست اومد، و ثابت شد که حملات تایمینگ می تونن توی مقیاس بزرگ و توی دنیای واقعی کار کنن.

یه چیز مهم دیگه هم پیدا شد: ورودی هایی که باعث خطا می شن، می تونن مسیرهای بزرگ کد (Large Code Paths) رو کوتاه کنن و باعث پاسخ های خیلی سریع تر بشن. به عبارت دیگه، حملات تایمینگ خیلی خوبن توی تشخیص خطاها.



Server-side injection

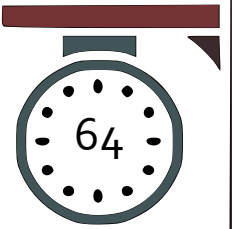
ایجاد و تشخیص خطاها به بخش اساسی توی تست آسیب پذیری های تزریق سمت سرور هست، از تزریق SQL (SQLi) گرفته تا OS Command Injection. به همین دلیل، تحلیل زمان (Timing Analysis) به ابزار عالی برای تشخیص این نوع اینجکشنه.

تلاش اول با Backslash Powered Scanner:

سعی کردم موفقیتیم با Param Miner رو تکرار کنم، برای همین Time رو به عنوان به ویژگی پاسخ (Response Attribute) به ابزار Backslash Powered Scanner اضافه کردم، ولی این کار نتیجه نداد. بدون استفاده از روش Single-Packet Attack، فقط می تونستم تفاوت های زمانی بزرگ رو تشخیص بدم، و این تفاوت ها بیشتر به خاطر فایروال های برنامه وب بودن تا آسیب پذیری های واقعی. علاوه بر این، پیچیدگی ابزار باعث شد که سازگار کردنش برای حل چالش ها سخت باشه.

تلاش دوم با به تست ساده تر:

توی تلاش دوم، به کد ساده تر از Param Miner استفاده کردم و به تست جدید ساختم که از روش Single-Packet Attack استفاده می کرد. توی این تست، برای هر آزمایش تا 50 جفت درخواست فرستادم و ترتیب پاسخ ها رو برای هر جفت ثبت کردم. اگر ترتیب پاسخ ها حداقل 80٪ به سمت به Payload خاص متمایل بود (یعنی به Payload همیشه زودتر یا دیرتر جواب می داد)، اون رو به عنوان به یافته معتبر گزارش کردم.



حمله Fully blind SQLi

اولین یافته به SQL injection کاملاً Blind بود که با به جفت Payload کلاسیک تشخیص داده شد:

درخواست GET /api/alert?mic='{}' پاسخ {} با زمان 162 میلی ثانیه.

درخواست GET /api/alert?mic='*': پاسخ {} با زمان 170 میلی ثانیه (8 میلی ثانیه تأخیر بیشتر). این نشون می ده که Payload دوم (*) باعث به تأخیر شده، که می تونه نشونه SQL injection باشه.

Request	Response	Time
GET /api/alert?mic=''	{}	162ms
GET /api/alert?mic='*'	{}	170ms

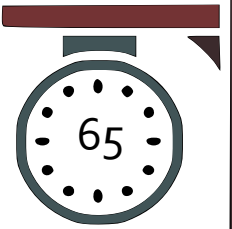
مشکل این یافته: متأسفانه، وقتی این یافته رو گزارش کردم، معلوم شد که قبلاً پیدا شده.

با نگاه به گذشته، باید این رو پیش بینی می کردم. می تونستی همین آسیب پذیری رو با Payload معروف '[sleep(5)]' به راحتی تشخیص بدی (چون این Payload باعث 5 ثانیه تأخیر می شه).

تحلیل زمان پیشرفته برای تشخیص آسیب پذیری هایی که می تونی دستورات sleep رو تزریق کنی، لازم نیست. به همین ترتیب، تحلیل زمان برای پیدا کردن Code Injection هم خیلی خوب نیست، چون معمولاً می تونی این نوع آسیب پذیری ها رو با تکنیک های OAST (Out-of-Band Application Security Testing) بهتر پیدا کنی.

کاربرد تحلیل زمان:

برای آسیب پذیری های قوی مثل Command Injection، تزریق SQL، و Code Injection، تشخیص مبتنی بر زمان فقط وقتی مفیده که به WAF یا فیلترینگ وجود داشته باشه که تکنیک های کلاسیک تشخیص رو بلاک کنه. پس بهتره جای دیگه رو نگاه کنیم.



Blind JSON Injection

تحلیل زمان وقتی به کار میاد که بخوای آسیب پذیری های سطح پایین تر (Injection Underclass) رو پیدا کنی. آسیب پذیری هایی که بهت اجازه می دن ساختارها و فرمت های داده رو دستکاری کنی، ولی به اجرای کد کامل نمی رسن. این شامل تزریق توی فرمت هایی مثل CSV، XML، JSON، و پارامترهای کوئری سمت سرور و هدرهای HTTP می شه. خیلی از این باگ ها به ندرت درباره شون صحبت می شه، چون تشخیصشون خیلی سخته.

چالش Exploit و استفاده از تحلیل زمان:

Exploit این باگ ها هم سخت، ولی گاهی می تونی اطلاعات زمان بندی رو با ویژگی های قابل مشاهده ترکیب کنی تا بفهمی پشت صحنه چه اتفاقی داره می افته.

مثلاً یه هدف پیدا شد که توش یه رشته Escape Sequence (Escape Sequence - یعنی ترکیب خاصی از کاراکترها که معنی ویژه داره، مثلاً \n یا \t) نامعتبر JSON باعث شد پاسخ ۲۰۰ میکروثانیه (۰.۲ میلی ثانیه) سریع تر برگرده.

Escape Sequence توی JSON به کاراکترهایی مثل \n (خط جدید) یا \t (تب) می گن. وقتی این رشته به شکل نامعتبر استفاده بشه (مثلاً \x بدون مقدار معتبر)، ممکنه پردازش JSON رو مختل کنه و باعث بشه پاسخ سرور زودتر برگرده.

اگه پارامتر key=a\\bb باشه: پاسخ

```
error: { message: Invalid Key: a\\bb }
```

با زمان 24.3 میلی ثانیه.

اگه پارامتر key=a\\nbb باشه: پاسخ

```
error: { message: Invalid Key: a\\nbb }
```

با زمان 24.1 میلی ثانیه (0.2 میلی ثانیه سریع تر).



Parameter	Response	Time
key=a\"bb	<pre>"error": { "message": "Invalid Key: a\"bb" }</pre>	24.3ms
key=a\"bb	<pre>"error": { "message": "Invalid Key: a\"bb" }</pre>	24.1ms

سؤال: سمت سرور چه اتفاقی داره می افته؟

یه سرنخ توی فرمت پاسخ وجود داره سینتکس نامعتبر JSON که تزریق کردیم، فرمت پاسخ رو تغییر نداده. انتظار می رفت که یه فرمت کننده JSON وقتی با سینتکس نامعتبر مواجه می شه، خطا بده یا حداقل یه خروجی متفاوت برگردونه، ولی این اتفاق نیفتاده.

یه نکته دیگه: ورودی های طولانی توی پاسخ کوتاه شدن:

اگه پارامتر key=aaa...a\"bb باشه: پاسخ

error: { message: Invalid Key: *bb }**

با زمان 24.3 میلی ثانیه.

Parameter	Response	Time
key=aaa...a\"bbb	<pre>"error": { "message": "Invalid Key: ****bbb" }</pre>	24.3ms

این کوتاه شدن یه سرنخ دوم می ده. وقتی رشته JSON نامعتبر کوتاه شد، تفاوت زمانی (0.2 میلی ثانیه) غیبش زد! این دو سرنخ با هم نشون می دن که تأخیر احتمالاً به خاطر یه کامپوننت داره اتفاق می افته که پاسخ رو قبل از ارسال به ما پردازش می کنه. بهترین حدس اینه که این یه سیستم ثبت خطا باشه.

از اینکه تونستم این رو از یه تفاوت زمانی 0.2 میلی ثانیه ای بفهمم خیلی خوشحال شدم، ولی چون مسیر واضحی برای Exploit پیدا نکردم، تصمیم گرفتم ادامه بدم و سراغ مورد دیگه ای برم.

Listen to the whispers web timing attacks that actually works



Blind Server-Side Parameter Pollution

مؤثرترین آزمایش من برای پیدا کردن آلودگی پارامتر سمت سرور Blind بود. این روش با مقایسه زمان پاسخ برای کاراکترهای رزرو شده URI (مثل ? و #) با کاراکترهای غیررزرو شده (مثل !) کار می کرد.

توی بعضی موارد، فرستادن یه # کد گذاری شده (Encoded) باعث می شد پاسخ سریع تر برگرده:

درخواست /path?objectId=57%23 (که 23٪ همون # کد گذاری شده ست): پاسخ Can't parse parameter با زمان 180 میلی ثانیه.

درخواست /path?objectId=57%21 (که 21٪ همون ! کد گذاری شده ست): پاسخ Can't parse parameter با زمان 430 میلی ثانیه (خیلی کندتر).

Request	Response	Time
/path?objectId=57%23	Can't parse parameter	180ms
/path?objectId=57%21	Can't parse parameter	430ms

این می تونه به این دلیل باشه که # یه مسیر سمت سرور (Server-Side Path) رو خراب کرده و یه پاسخ استاتیک سریع از بک اند برگشته، یا اینکه کلاینت HTTP برنامه کلاً از فرستادن درخواست با # خام (Raw) خودداری کرده. البته، مهمه که فرض نکنیم تأخیر همیشه یه جور عمل می کنه توی هدف های دیگه، # کد گذاری شده باعث کندتر شدن پاسخ می شد.

اهمیت آلودگی پارامتر سمت سرور:

آلودگی پارامتر سمت سرور با اختلاف زیادی رایج ترین نوع تزریق (Injection) بود که کشف شد، برای همین فکر می کنم یه حوزه خیلی خوب برای تحقیقات بیشتره. برای اطلاعات بیشتر درباره این نوع حمله، می تونی به منابع Server-Side Parameter Pollution و Attacking Secondary Contexts in Web Applications نگاه کنی.

<https://portswigger.net/web-security/api-testing/server-side-parameter-pollution>

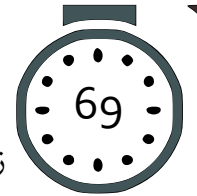
<https://www.youtube.com/watch?v=hWmXEAgz5w>

Listen to the whispers web timing attacks that actually works

Bug doppelgangers

همون طور که دیدیم، اندازه گیری زمان با دقت بالا (High-Precision Timing)، یعنی اندازه گیری زمان پاسخ سرور با دقت میکروثانیه ای برای پیدا کردن تفاوت های خیلی کوچیک (خیلی برای پیدا کردن باگ های Blind Injection Bugs)، یعنی باگ هایی که نمی تونی نتیجه شون رو مستقیماً ببینی، مثلاً توی یه حمله SQL Injection Blind، نمی تونی داده های دیتابیس رو مستقیماً ببینی و فقط با زمان پاسخ سرور، مثلاً یه تأخیر 5 ثانیه ای، می فهمی که تزریق موفق بوده (عالیه، ولی همیشه نمی تونی به راحتی این باگ ها رو Exploit کنی. وقتی این یافته ها رو تحلیل می کردم، معمولاً یه درک کلی از اتفاقاتی که توی سرور می افتاد به دست می آوردم (مثلاً می فهمیدم که سرور داره یه پارامتر خاص رو پردازش می کنه یا یه کوئری خاص رو اجرا می کنه، مثلاً یه کوئری SQL که باعث تأخیر می شه)، ولی نمی تونستم به مرحله سوءاستفاده واقعی برسم (یعنی نمی تونستم از این باگ برای استخراج داده یا دسترسی به بخش های حساس سرور استفاده کنم). همچنین، حملات زمان بندی معمولاً نوع خاصی از حملات رو پیدا می کنن که کمتر شناخته شده ان (Lesser-Known Attack Classes)، مثلاً باگ هایی توی فناوری های جدید یا رفتارهای عجیب سرور که توی حملات کلاسیک مثل XSS یا SQL Injection معمولی نمی بینیم) و ما تجربه کمتری توی سوءاستفاده از شون داریم (چون این نوع باگ ها کمتر دیده شدن و روش های سوءاستفاده از شون هنوز به خوبی شناخته شده نیست).

جمع آوری اطلاعات کافی برای سوءاستفاده (Exploit) فقط با استفاده از شواهد زمان بندی (Timing Evidence)، یعنی فقط با اندازه گیری زمان پاسخ سرور و مقایسه زمان ها برای فهمیدن رفتار سرور (معمولاً سخت و زمان بره (چون باید کلی درخواست بفرستی، زمان ها رو اندازه بگیری، و تحلیل کنی تا بفهمی باگ چطور کار می کنه). توی یه آسیب پذیری معمولی و غیر Blind (Non-Blind Vulnerability)، یعنی باگی که می تونی نتیجه ش رو مستقیماً ببینی، مثلاً توی یه SQL Injection معمولی، وقتی یه کوئری تزریق می کنی، سرور یه خطای دیتابیس یا داده های دیتابیس رو مستقیماً توی صفحه وب نشون می ده)، تست کردن هر ایده معمولاً فقط یه درخواست ساده توی Repeater (یه ابزار توی Burp Suite که می تونی باهاش درخواست های HTTP رو دستی بفرستی و نتیجه رو ببینی) می خواد (مثلاً یه درخواست می فرستی و سریع می فهمی که باگ کار می کنه یا نه، چون خطا یا داده رو می بینی). ولی توی این باگ های Blind که با زمان بندی پیدا می کنی، ممکنه برای هر تست به یه حمله 30 ثانیه ای با Turbo Intruder (یه ابزار توی Burp Suite که می تونی تعداد زیادی درخواست خود کار بفرستی،



زمان پاسخ‌ها رو اندازه‌گیری، و رفتار سرور رو تحلیل کنی) نیاز داشته باشی، چون باید کلی درخواست بفرستی (مثلاً 30 تا درخواست با پارامتر و 30 تا بدون پارامتر) و زمان‌ها رو مقایسه کنی تا بفهمی باگ واقعاً وجود داره یا نه، و این کار خیلی بیشتر طول می‌کشه.

یه چیزی که اینجا می‌تونه کمک کنه، دوقلوهای باگ (Bug Doppelgangers) هستن، یعنی نسخه‌های غیر Blind از همون نوع باگ‌هایی که داری روشن کار می‌کنی (Target Bug Class، یعنی باگ‌هایی از همون دسته، مثلاً اگه باگ کورت یه SQL Injection کوره، دوقلوش یه SQL Injection غیر کوره). مثلاً اگه یه باگ SQL Injection Blind پیدا کردی که فقط با زمان‌بندی معلومه (مثلاً با فرستادن `id=1 AND SLEEP(5)` می‌بینی سرور 5 ثانیه تأخیر می‌کنه)، یه دوقلوی باگ می‌تونه یه SQL Injection غیر Blind باشه توی همون سرور یا یه سرور مشابه که نتیجه‌ش رو می‌تونی مستقیماً ببینی (مثلاً با فرستادن `username=admin` — یه خطای دیتابیس می‌بینی). Param Miner این دوقلوهای باگ رو گزارش می‌ده (یعنی باگ‌های غیر Blind مشابه رو پیدا می‌کنه و بهت نشون می‌ده)، و اینا برای یادگیری خیلی خوبن، چون می‌تونی بفهمی که چطور این نوع باگ‌ها رو تفسیر کنی (مثلاً بفهمی سرور چطور به این باگ‌ها واکنش نشون می‌ده) و چطور ازشون سوءاستفاده کنی (مثلاً یاد بگیري چه کوئری‌هایی برای استخراج داده کار می‌کنه)، ولی توی یه محیط ساده‌تر (Less Challenging Environment، یعنی جایی که لازم نیست فقط به زمان‌بندی تکیه کنی و می‌تونی نتیجه رو مستقیماً ببینی، مثلاً خطای سرور یا داده‌های دیتابیس رو توی صفحه وب ببینی).

دوقلوهای باگ بخشی از یه موضوع کلی‌تر و تکرارشونده توی این تحقیقن (Broader, Recurrent Theme، یعنی یه درس مهم که توی کل تحقیق تکرار می‌شه). اگه زمان‌بندی (Timing) رو کاملاً نادیده بگیری، یه سری باگ مهم رو از دست می‌دی (چون خیلی از باگ‌ها، به‌خصوص باگ‌های کور، فقط با زمان‌بندی پیدا می‌شن، مثلاً یه پارامتر مخفی که سرور رو کند می‌کنه). ولی اگه بیش از حد روی زمان‌بندی تمرکز کنی، باز هم یه سری چیزا رو از دست می‌دی (مثلاً باگ‌های غیر Blind که راحت‌تر می‌تونی ازشون سوءاستفاده کنی و نیازی به زمان‌بندی ندارن). برای موفقیت، باید از هر کانال اطلاعاتی که داری (Every Available Information Channel) استفاده کنی، یعنی هم زمان‌بندی رو در نظر بگیری (برای پیدا کردن باگ‌های کور)، هم خروجی‌های مستقیم (مثل خطاها، تغییرات توی پاسخ سرور، یا داده‌هایی که سرور نشون می‌ده)، و هم هر سرنخ دیگه‌ای که می‌تونی پیدا کنی (مثلاً لاگ‌ها، هدرها، یا رفتارهای غیرعادی سرور).



مثال واقعی: فرض کن با Param Miner روی یک وبسایت کار می‌کنی و یک باگ Blind پیدا کردی: توی مسیر `http://example.com/api/user?id=1`، وقتی `id=1 AND SLEEP(5)` می‌فرستی، سرور 5 ثانیه تأخیر می‌کنه، ولی هیچ داده یا خطایی نشون نمی‌ده (یه SQL Injection کور). نمی‌دونی چطور از این باگ سوءاستفاده کنی، چون فقط با زمان‌بندی فهمیدی که باگ وجود داره. Param Miner یه دوقلوی باگ پیدا می‌کنه: توی مسیر `http://example.com/login?username=admin`، وقتی `'username=admin'--` می‌فرستی، سرور یه خطای دیتابیس نشون می‌ده (مثلاً `SQL Error: You have an error in your SQL syntax`)، که یه SQL Injection غیر کوره. حالا روی این باگ غیر Blind کار می‌کنی، می‌فهمی که سرور از MySQL استفاده می‌کنه و چه فیلترهایی داره، و بعد از این دانش برای سوءاستفاده از باگ Blind استفاده می‌کنی، مثلاً یه کوئری پیچیده‌تر می‌فرستی (مثل `id=1 AND IF(1=1, SLEEP(5), 0)`) تا داده‌ها رو به صورت Blind استخراج کنی.

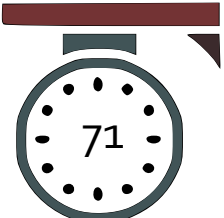
Reverse proxy misconfigurations

بزرگ‌ترین پیشرفت توی این تحقیق وقتی بود که فهمیدم می‌تونم از زمان‌بندی (Timing) برای تشخیص یک نوع SSRF (Server-Side Request Forgery) استفاده کنم که معمولاً نادیده گرفته می‌شه.

تحقیقات سال 2017

توی سال 2017، تکنیک‌هایی رو برای Exploit از ریورس پراکسی‌های که اشتباه پیکربندی شدن تحقیق کردم، تا بتونم از SSRF استفاده کنم و به سیستم‌های داخلی (Internal Systems) دسترسی پیدا کنم. رایج‌ترین آسیب‌پذیری این بود که سرورها درخواست‌ها رو به دامین مشخص شده توی هدر HTTP Host هدایت می‌کردن.

Listen to the whispers web timing attacks that actually works



برای تشخیص این آسیب پذیری، به درخواست با هدر Host می فرستادم که به یه دامین تحت

کنترل خودم اشاره می کرد:

مثلاً:

```
GET / HTTP/1.1
Host: uniq-token.burpcollaborator.net
```

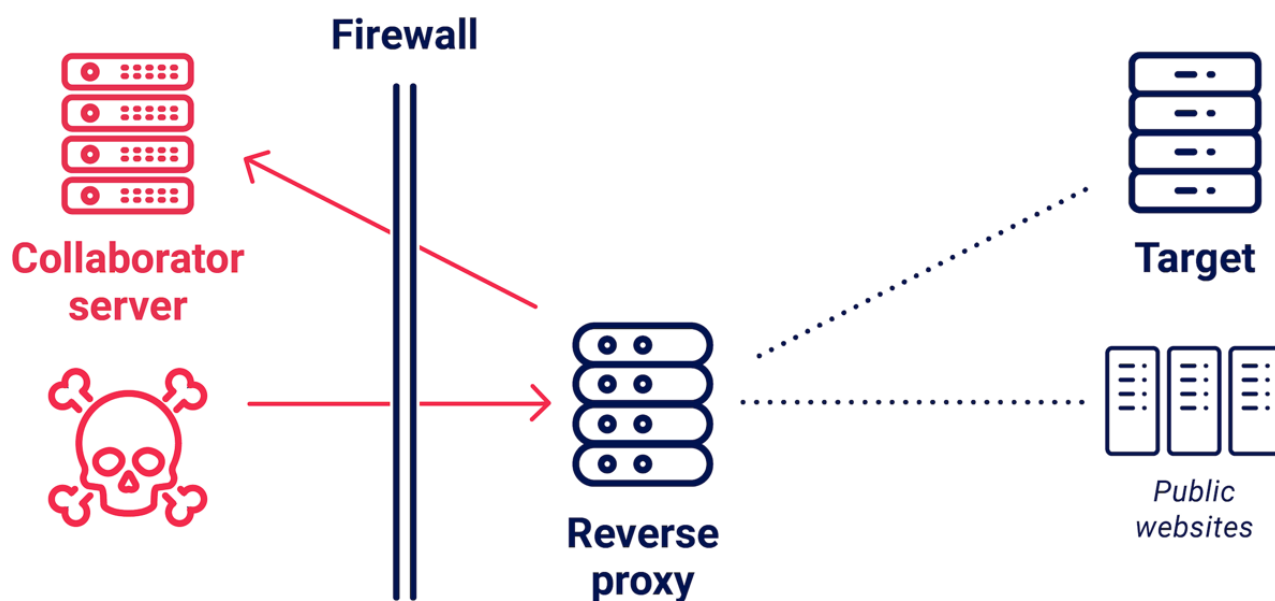
اگه هدف آسیب پذیر بود، می دیدم که درخواستم به سایت خودم توی burpcollaborator.net می رسه، چون Reverse Proxy آسیب پذیر درخواست رو فوروارد کرده.

گام بعدی

کاوش شبکه داخلی:

بعد از اینکه آسیب پذیری رو تأیید می کردم، آدرس های IP داخلی (Internal IPs) و نام های هاست (Hostnames) رو می فرستادم تا شبکه داخلی شون رو کاوش (Plunder) کنم.

این کار به یافته های شگفت انگیزی منجر شد، از جمله اینکه به صورت تصادفی یه سیستمی رو هک کردم که ISP (ارائه دهنده خدمات اینترنت) من برای MITM (Man-in-the-Middle) کردن مشتری هاش قرار داده بود.



Listen to the whispers web timing attacks that actually works



دیاگرام جریان درخواست:

یه سرور با عنوان Collaborator server (با آیکون هکر) نشون داده شده که به یه Reverse proxy وصل می شه.

Reverse Proxy به یه Target وصل می شه، و هدف به وبسایت های عمومی وصل می شه.

یه فلش از Collaborator server به Reverse proxy و بعد به Target نشون می ده که درخواست از **Reverse Proxy** فوروارد می شه.

یه درخواست HTTP با هدر Host که به `uniq-token.burpcollaborator.net` اشاره می کنه، برای تست آسیب پذیری **Reverse Proxy**.

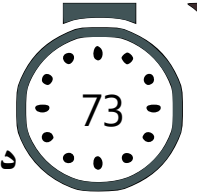
دیاگرام هم جریان رو نشون می ده: درخواست از Collaborator server (تحت کنترل مهاجم) به **Reverse Proxy** می ره، و اگه پراکسی آسیب پذیر باشه، درخواست به Target فوروارد می شه و در نهایت به Collaborator server برمی گرده.

SSRF محدود (Scoped SSRF)

تکنیک تشخیص من برای پیدا کردن (Server-Side Request Forgery) SSRF موفق بود، ولی یه مشکل بزرگ داشت: نمی تونست یه نوع خاص از SSRF رو تشخیص بده، که بهش می گم Scoped SSRF (SSRF محدود).

SSRF چیست و چرا Scoped SSRF متفاوت است؟

(Server-Side Request Forgery) SSRF یه نوع آسیب پذیری که به مهاجم اجازه می ده از یه سرور آسیب پذیر درخواست هایی به جاهایی بفرسته که نباید بتونه بهشون دسترسی داشته باشه، مثلاً سرورهای داخلی شرکت یا حتی سرورهای خارجی. Full SSRF یعنی مهاجم می تونه به هر آدرسی که بخواد درخواست بفرسته، ولی Scoped SSRF یه نسخه محدودتره: توی این حالت، سرور فقط به یه سری آدرس های خاص (مثلاً سابدامنه های شرکت) اجازه می ده درخواست ها رو فوروارد کنه. این محدودیت باعث می شه تشخیص Scoped SSRF با روش های معمولی (مثل Pingback) سخت تر بشه، ولی همچنان خطرناکه چون می تونه به مهاجم دسترسی به سیستم های داخلی بده.



داستان گوگل و یه نقطه کور:

بعد از اینکه تحقیق رو منتشر کردم، یکی از گوگل ازم پرسید که آیا توی سیستم هاشون آسیب پذیری پیدا کردم یا نه، و به طور غیرمستقیم گفت که اونا آسیب پذیر بودن.

کمی بعد، یه محقق به اسم Ezequiel Pereira یه گزارش با جایزه 10 هزار دلاری منتشر کرد که توی اون یه پراکسی باز (Open Proxy) متعلق به گوگل رو Exploit کرده بود. من این پراکسی رو نتونستم پیدا کنم!

دلیلش این بود که روش اسکن من شکست خورد: پراکسی گوگل طوری تنظیم شده بود که فقط درخواست ها رو به سیستم های خودشون (مثلاً دامین های گوگل) هدایت کنه. برای همین، سرور من هیچ وقت یه درخواست DNS Lookup از گوگل دریافت نکرد (چون درخواست به سرور من فرووارد نشد).

Ezequiel Pereira تونست یه پراکسی باز متعلق به گوگل رو پیدا کنه که به سابدامنه های گوگل (مثل google.com.*) اجازه می داد درخواست ها رو فرووارد کنه. اون با تنظیم هدر Host به یه سابدامنه خاص (مثلاً internal.google.com) تونست به یه سیستم داخلی گوگل دسترسی پیدا کنه که به صورت عمومی قابل دسترسی نبود. این کار بهش اجازه داد اطلاعاتی رو ببینه که نباید می دید، و برای همین گوگل بهش جایزه 10 هزار دلاری داد. این نشون می ده که حتی شرکت های بزرگی مثل گوگل هم ممکنه به Scoped SSRF آسیب پذیر باشن.

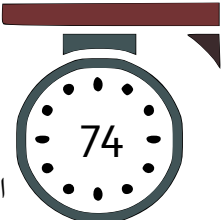
یه سناریوی رایج:

این ماجرا به من نشون داد که یه سناریوی خیلی رایج وجود داره. خیلی از شرکت ها پراکسی هاشون رو طوری تنظیم می کنن که فقط به سابدامنه های دلخواه (Arbitrary Subdomains) اجازه فرووارد کردن درخواست ها رو می دن.

مثلاً فرض کن یه شرکت دامینش example.com باشه:

اگه هدر Host توی درخواست random.example.com باشه (یعنی یه سابدامنه از example.com)، پراکسی درخواست رو فرووارد می کنه و پاسخ 404 Not Found می گیرم (چون دامین وجود نداره).

اما اگه هدر Host یه دامین کاملاً متفاوت مثل random.notexample.com باشه، پراکسی درخواست رو بلاک می کنه و پاسخ 403 Forbidden می گیرم (چون دامین خارج از محدوده مجاز).



این رفتار با Full SSRF فرق داره: توی Full SSRF، پراکسی هر درخواستی رو فوروارد می‌کنه (چه random.example.com باشه، چه random.notexample.com) و همیشه پاسخ 404 Not Found می‌گیرم (چون دامین‌ها وجود ندارن).

Host header	Full SSRF	Scoped SSRF
random.example.com	404 Not Found	404 Not Found
random.notexample.com	404 Not Found	403 Forbidden

این جدول رفتار Full SSRF و Scoped SSRF رو مقایسه می‌کنه:

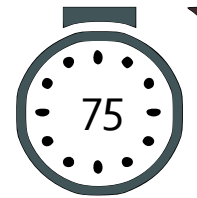
توی Full SSRF، پراکسی همه درخواست‌ها رو فوروارد می‌کنه، چه هدر Host به سادامنه از دامین مجاز باشه (random.example.com)، چه به دامین کاملاً متفاوت (random.notexample.com). برای همین توی هر دو حالت پاسخ 404 Not Found می‌گیرم (چون دامین‌ها وجود ندارن).

توی Scoped SSRF، پراکسی فقط به سادامنه‌های مجاز (مثل random.example.com) اجازه فوروارد کردن می‌ده و پاسخ 404 Not Found می‌گیرم. اما اگه دامین غیرمجاز باشه (مثل random.notexample.com)، پراکسی درخواست رو بلاک می‌کنه و پاسخ 403 Forbidden می‌گیرم.

تعریف Scoped SSRF: به نظرم اسم مشخصی برای این نوع SSRF وجود نداره، برای همین من بهش می‌گم Scoped SSRF (محدود).

این محدودیت می‌تونه به روش‌های مختلفی اعمال بشه:

- با به سرور DNS داخلی (Internal DNS Server) که فقط دامین‌های مجاز رو Resolve کنه.
- با به اعتبارسنجی ساده نام هاست که فقط سادامنه‌های خاص رو قبول کنه.
- با به فایروال که درخواست‌های DNS خروجی (Outbound DNS) رو بلاک کنه.
- یا با به تنظیمات سخت‌گیرانه برای Listener (Tight Listener Config) که فقط به دامین‌های خاص اجازه بده.



نتیجه همیشه همونه یه باگ داری که تأثیرش خیلی شبیه Full SSRF هست (چون می تونی به سیستم های داخلی دسترسی پیدا کنی)، ولی نمی تونی با تکنیک های معمولی مثل Pingback یا OAST (Out-of-Band Application Security Testing) تشخیصش بدی، چون درخواست ها به سرور خارجی (مثل سرور من) فوروارد نمی شن.

برای اینکه Scoped SSRF رو بهتر بفهمی، بذار یه مثال ساده تر بزنم:

فرض کن یه شرکت یه ریورس پراکسی داره که فقط به دامین های خودش (مثلاً *.google.com) اجازه می ده درخواست ها رو فوروارد کنه.

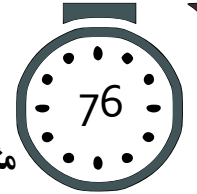
اگه من یه درخواست با هدر Host: test.google.com بفرستم، پراکسی درخواست رو فوروارد می کنه (چون سابدامنه google.com هست)، ولی چون test.google.com وجود نداره، پاسخ 404 Not Found می گیرم. حالا اگه هدر رو عوض کنم و بذارم Host: my-server.com (یه دامین که تحت کنترل منه)، پراکسی می گه این دامین جزو google.com نیست و درخواست رو بلاک می کنه، و من پاسخ 403 Forbidden می گیرم.

توی Full SSRF، پراکسی این درخواست رو به my-server.com فوروارد می کرد و من می تونستم با تکنیک های OAST (مثلاً یه DNS Lookup) تشخیص بدم که پراکسی آسیب پذیره. ولی توی Scoped SSRF، چون درخواست به سرور من نمی رسه، نمی تونم با روش های معمولی پیداش کنم اینجاست که زمان بندی (Timing) به کار میاد (که توی بخش های بعدی مقاله توضیح داده می شه).

شناسایی SSRF های محدود شده (Detecting scoped SSRF)

برای اینکه بتونیم SSRF محدود (Scoped SSRF) رو تشخیص بدیم، باید به این سؤال جواب بدیم: آیا سرور سعی کرده به نام هاست (Hostname) مشخص شده وصل بشه؟

تحلیل زمان (Timing) برای این کار خیلی مناسبه، چون می تونه نشون بده که سرور چه رفتاری داره.



مثال اول: بررسی اولیه با هدر Host

فرض کن یه سرور توی `www.example.com` داریم که این پاسخ ها رو می ده:

اگه هدر Host باشه `foo.example.com` (یعنی یه سابدامنه از `example.com`):

پاسخ: 404 Not Found

زمان: 25 میلی ثانیه

اگه هدر Host باشه `foo.bar.com` (یعنی یه دامین کاملاً متفاوت):

پاسخ: 403 Forbidden

زمان: 20 میلی ثانیه

این دو پاسخ نشون می دن که سرور یه جور اعتبارسنجی (Validation) روی هدر Host انجام می ده:

برای `foo.example.com` که سابدامنه مجاز، درخواست رو پردازش می کنه (ولی چون دامین وجود نداره، 404 می ده).

برای `foo.bar.com` که دامین غیرمجاز، درخواست رو بلاک می کنه و 403 می ده.

Host header	Response	Time
<code>foo.example.com</code>	404 Not Found	25ms
<code>foo.bar.com</code>	403 Forbidden	20ms

اما این اطلاعات کافی نیست که بفهمیم آیا این سرور یه پراکسی باز (Open Proxy) هست یا نه. اگه فقط به محتوای پاسخ (مثلاً 404 و 403) نگاه کنیم، ممکنه به نتایج اشتباه برسیم (False Positives یا False Negatives).

Listen to the whispers web timing attacks that actually works



Host header	Response	Time
abc.example.com	404 Not Found	25ms
abc.example.com	404 Not Found	20ms

مثال دوم: اثبات مشکل با کش DNS

حالا به جفت درخواست دیگه می فرستیم که مشکل رو ثابت می کنه:

درخواست اول: هدر Host برابر abc.example.com

پاسخ: 404 Not Found

زمان: 25 میلی ثانیه

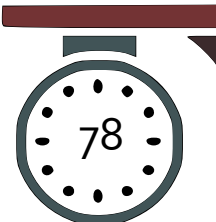
درخواست دوم: دوباره هدر Host برابر abc.example.com

پاسخ: 404 Not Found

زمان: 20 میلی ثانیه (5 میلی ثانیه سریع تر)

چرا زمان دوم سریع تره؟ چون سرور توی درخواست اول به DNS Lookup برای abc.example.com انجام داده و نتیجه رو کش کرده. برای همین توی درخواست دوم نیازی به DNS Lookup دوباره نیست و پاسخ سریع تر میاد.

این تفاوت زمانی نشون می ده که سرور واقعاً داره سعی می کنه به هاست مشخص شده وصل بشه (یعنی به Scoped SSRF وجود داره)، چون اگه این کار رو نمی کرد، زمان ها برابر می بودن.



مثال سوم: راه حل برای سیستم‌هایی که کش نمی‌کنن

یه مشکل وجود داره: بعضی سیستم‌های DNS، نتایج DNS Lookup ناموفق (Failed DNS Lookups) رو کش نمی‌کنن. برای همین ممکنه توی مثال بالا تفاوت زمانی نبینیم.

من یه راه حل جایگزین پیدا کردم: استفاده از یه لیبل DNS خیلی بلند (Overlong DNS Label — یعنی یه بخش از نام دامنه که طولش از حد استاندارد ۶۳ کاراکتر بیشتره و می‌تونه باعث رفتار غیرعادی در سرورهای DNS بشه).

یه لیبل DNS (مثلاً یه بخش از نام دامین مثل abc توی abc.example.com) طبق استاندارد نباید بیشتر از 63 کاراکتر باشه.

اگه یه لیبل 64 کاراکتری بفرستم، کلاینت DNS از انجام Lookup خودداری می‌کنه و پاسخ سریع‌تر میاد.

مثلاً:

هدر Host برابر aaa(62).example.com (یعنی aaa با 62 کاراکتر):

پاسخ: 404 Not Found

زمان: 25 میلی‌ثانیه (چون DNS Lookup انجام می‌شه).

هدر Host برابر aaa(63).example.com (یعنی aaa با 63 کاراکتر):

پاسخ: 404 Not Found

زمان: 20 میلی‌ثانیه (چون کلاینت DNS از Lookup خودداری می‌کنه).

Host header	Response	Time
aaa{62}.example.com	404 Not Found	25ms
aaa{63}.example.com	404 Not Found	20ms



این هم به راه دیگه برای تشخیص Scoped SSRF هست: اگه زمان پاسخ برای لیبل خیلی بلند سریع تر باشه، یعنی سرور داره DNS Lookup انجام می ده (و به Scoped SSRF وجود داره).

برای اینکه Scoped SSRF رو بهتر بفهمی، بذار به مثال ساده تر بزنم:

فرض کن به شرکت به ریورس پراکسی داره که فقط به سابدامنه های خودش (مثلاً *.example.com) اجازه می ده درخواست ها رو فوروارد کنه.

من دو درخواست می فرستم:

با هدر:

Host: test.example.com

پراکسی درخواست رو فوروارد می کنه (چون سابدامنه example.com هست)، ولی چون test.example.com وجود نداره، پاسخ 404 Not Found می گیرم و زمانش 25 میلی ثانیه ست (چون به DNS Lookup انجام شده). دوباره با هدر:

Host: test.example.com

این بار هم 404 می گیرم، ولی زمانش 20 میلی ثانیه ست، چون نتیجه DNS Lookup از درخواست اول کش شده و نیازی به Lookup دوباره نیست.

این تفاوت زمانی (25ms به 20ms) نشون می ده که پراکسی داره به هاست وصل می شه، یعنی به Scoped SSRF داریم.

حالا اگه سیستم DNS کش نکنه، من می تونم از به لیبل بلند (مثلاً 64 کاراکتر) استفاده کنم. اگه لیبل بلند باشه، کلاینت DNS از Lookup خودداری می کنه و زمان پاسخ سریع تر می شه، که بازم نشون می ده به Scoped SSRF وجود داره.

SECTION

4

یه مثال عملی از
صفر تا صد

سناریو

بیا فرض کنیم داری روی shop.example.com کار می‌کنی که یه فرم ورود داره. می‌خوای بفهمی آیا نام کاربری admin وجود داره یا نه، بدون اینکه به متن پاسخ سرور نگاه کنی. حملات زمان‌بندی اینجا می‌درخشن! اول یه درخواست ساده می‌فرستیم:

POST /login HTTP/1.1

Host: shop.example.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 27

username=admin&password=123

پاسخ می‌گیریم: 401 Unauthorized با زمان 150 میلی‌ثانیه. حالا یه کاربر که مطمئنیم نیست رو تست می‌کنیم:

POST /login HTTP/1.1

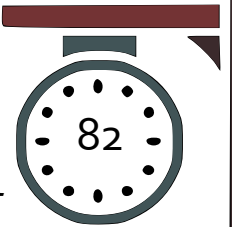
Host: shop.example.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 32

username=nobody&password=123

پاسخ: 401 Unauthorized ولی زمانش 120 میلی‌ثانیه‌ست. این 30 میلی‌ثانیه تفاوت چی می‌گه؟ وقتی admin رو می‌فرستیم، سرور رمز رو با دیتابیس چک می‌کنه و یه کم بیشتر طول می‌کشه، ولی برای nobody سریع می‌فهمه که همچین کاربری نیست و رد می‌شه. این یه سیگنال قویه که admin وجود داره!



حالا بیایم اینو خودکار کنیم. توی Burp Suite، درخواست رو به Turbo Intruder می‌فرستیم و به اسکریپت می‌نویسیم:

```
def queueRequests(target, wordlists):
```

```
    engine = RequestEngine(endpoint=target.endpoint, concurrentConnections=10,  
requestsPerConnection=100)
```

```
    for user in ['admin', 'nobody', 'test', 'guest']:
```

```
        engine.queue(target.req, [user])
```

```
def handleResponse(req, interesting):
```

```
    table.add(req)
```

این اسکریپت 4 تا نام کاربری رو تست می‌کنه و زمان پاسخ‌ها رو توی جدول نشون می‌ده. خروجی ممکنه اینجوری باشه:

- admin: 150ms
- nobody: 120ms
- test: 119ms
- guest: 148ms

واضحه که admin و guest کاربرهای واقعی‌ان چون تأخیر بیشتری دارن. حالا اگه بخوای به لیست بزرگ‌تر (مثلاً 1000 تا نام کاربری) رو تست کنی، فقط کافیه به فایل wordlist بذاری و به اسکریپت بدی. مثلاً:

```
def queueRequests(target, wordlists):
```

```
    engine = RequestEngine(endpoint=target.endpoint, concurrentConnections=10)
```

```
    for word in wordlists.words:
```

```
        engine.queue(target.req, [word])
```



با این روش می‌تونیم کل کاربرهای یه سیستم رو حدس بزنی، بدون اینکه حتی یه خط کد سرور رو ببینی! این فقط یه مثال ساده‌ست توی دنیای واقعی، می‌تونیم اینو با Param Miner ترکیب کنی و پارامترهای مخفی (مثل debug=true) رو هم تست کنی تا سطح حمله بزرگ‌تری پیدا کنی.

حالا فرض کن یه ریورس پراکسی روی proxy.company.com که فقط به سابدامنه‌های *.company.com اجازه می‌ده درخواست‌ها رو فوروارد کنه. می‌خواهیم ببینیم آیا این یه Scoped SSRF داره یا نه. درخواست اول:

GET / HTTP/1.1

Host: test.company.com

پاسخ: 404 Not Found ولی زمانش ms20 چون DNS کش شده. حالا یه هدر بلند تست می‌کنیم:

GET / HTTP/1.1

Host:

aaa.company.com

پاسخ: 404 Not Found با زمان ms18. چرا سریع‌تر شد؟ چون این دامنه بیشتر از حد مجاز (مثلاً 63 کاراکتر) بود و DNS Lookup انجام نشد این نشون می‌ده پراکسی داره دامنه‌ها رو چک می‌کنه و یه Scoped SSRF وجود داره!

بیایم اینو خودکار کنیم. توی Param Miner، یه لیست از سابدامنه‌ها (مثل test.company.com, app.company.com) رو می‌دیم و Timing Attack رو فعال می‌کنیم. خروجی ممکنه اینجوری باشه:

- (دومین بار) ms (اولین بار), 25ms20 test.company.com:
- (دومین بار) ms (اولین بار), 26ms21 secret.company.com:
- aaa...company.com: 18ms

اینجا معلومه که secret.company.com هم یه سابدامنه واقعیه. حالا می‌تونیم با یه ابزار مثل dnsdumpster سابدامنه‌های بیشتری پیدا کنی و تست رو گسترش بدی تا مسیرهای مخفی رو پیدا کنی.

جستجوی مسیرهای پنهان

جستجو برای مسیرهای مخفی

با استفاده از تکنیک‌های اسکن که قبلاً توضیح داده شد، صدها ریورس پراکسی آسیب‌پذیر پیدا کردم که مسیرهای جایگزین به ده‌ها هزار دامین رو در دسترس قرار می‌دادن. این حجم زیاد از یافته‌ها باعث شد به شدت به اتومیشن نیاز پیدا کنم.

گام اول: پیدا کردن همه مقصدها با پراکسی باز

وقتی به ریورس پراکسی باز پیدا می‌کنی، اولین کار اینه که ببینی می‌تونی ازش برای دسترسی به چه مقصدهایی استفاده کنی. برای این کار، به کد نویسی خودکار به لیست از سابدامنه‌های هدف رو از سه منبع اصلی جمع‌آوری کنه:

یه لیست ثابت از سابدامنه‌های عمومی: یه لیست از پیش آماده‌شده که شامل سابدامنه‌های رایج مثل 'www'، 'api'، 'mail' و غیره بود.

لیست سابدامنه‌های شناخته‌شده از Rapid7's Project Sonar: این یه فایل 58 گیگابایتی به اسم 'fdns' هست که شامل اطلاعات DNS سابدامنه‌های مختلفه. برای اینکه بتونم سریع سابدامنه‌های یه هدف خاص (مثلاً 'example.com') رو از این فایل پیدا کنم:

از ابزار rev استفاده کردم تا هر خط رو برعکس کنم (مثلاً 'sub.example.com' بشه 'moc.elpmaxe.bus').

بعد فایل رو به صورت الفبایی مرتب کردم تا سابدامنه‌های مرتبط (Sibling Domains) کنار هم قرار بگیرن.

در نهایت از ابزار قدیمی یونیکس به اسم 'look' استفاده کردم که یه جستجوی باینری انجام می‌ده. این روش زمان جستجو رو 99.999٪ نسبت به استفاده از 'grep' کم کرد!

سرویس آنلاین سابدامنه‌ها در columbus.elmasy.com: این سرویس بیشتر اطلاعاتش رو از لاگ‌های شفافیت گواهینامه (Certificate Transparency Logs) جمع‌آوری می‌کنه و سابدامنه‌های یه دامین رو نشون می‌ده.



روش کار Param Miner

Param Miner رو طوری تنظیم کردم که هر هاست (سابدامنه) رو دو بار تست کنه:

یه بار مستقیم (Directly): یعنی بدون استفاده از پراکسی، مستقیم به سابدامنه وصل بشه.

یه بار از طریق پراکسی: یعنی از ریورس پراکسی برای دسترسی به همون سابدامنه استفاده کنه.

بعد پاسخ‌های این دو روش رو مقایسه کردم. اگه پاسخ‌ها تفاوت چشمگیری داشتن، اون سابدامنه رو به‌عنوان یه یافته گزارش می‌کردم.

برای مقایسه پاسخ‌ها، روی این سه چیز تمرکز کردم که بیشترین اطلاعات رو می‌دادن:

– کد وضعیت پاسخ (Response Status Code): مثلاً 200، 404، 403 و غیره.

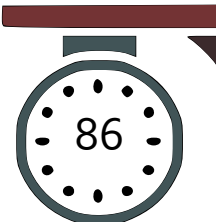
– نام هدرها (Header Names): مثلاً چه هدرهایی توی پاسخ هستن.

– هدر Location: اگه ریدایرکت اتفاق بیفته، این هدر نشون می‌ده به کجا هدایت می‌شه.

این روش کلی یافته به دست داد که توی چهار دسته کلی قرار گرفتن (دسته‌ها توی بخش‌های بعدی توضیح داده می‌شن).

تفاوت Host Bruteforcing و Exploit ریورس پراکسی: حدس زدن نام هاست‌ها توی هدر Host معمولاً به‌عنوان Host Bruteforcing شناخته می‌شه، ولی اکسپلویت ریورس پراکسی کاملاً متفاوت و باید این تفاوت رو درک کنیم:

Host Bruteforcing: این روش فقط بهت اجازه می‌ده به وب‌سایت‌های دیگه‌ای که روی همون سرور هستن دسترسی پیدا کنی. مثلاً اگه site1.com و site2.com روی یه سرور باشن، با Bruteforcing می‌تونی از site1.com به site2.com برسی.



ریورس پراکسی: ریورس پراکسی درخواست‌ها رو به سیستم‌های مختلف هدایت می‌کنه، نه فقط

همون سرور. این باعث می‌شه حملات منحصربه‌فردی ممکن بشه، مثل:

دور زدن قوانین فرانت‌اند (Front-End Rule Bypass).

جعل هویت فرانت‌اند (Front-End Impersonation).

زنجیره کردن Exploit ها (Exploit Chaining).

برای همین، باید این دو مفهوم رو از هم جدا کنیم و بفهمیم که ریورس پراکسی چه فرصت‌های متفاوتی ایجاد می‌کنه. حالا بیایم وارد جزئیات بشیم!

برای اینکه موضوع بهتر جا بیفته، بذار یه مثال ساده بزنم:

فرض کن یه ریورس پراکسی آسیب‌پذیر پیدا کردی که روی `proxy.example.com` کار می‌کنه. این پراکسی درخواست‌ها رو بر اساس هدر `Host` هدایت می‌کنه. حالا می‌خواهی ببینی چه سابدامنه‌هایی از `example.com` رو می‌تونی از طریق این پراکسی پیدا کنی.

اول یه لیست از سابدامنه‌ها درست می‌کنی:

از یه Wordlist عمومی: مثلاً `api.example.com`، `mail.example.com`، `www.example.com`.

از Rapid7's Project Sonar: مثلاً `dev.example.com`، `internal.example.com`.

از سرویس `columbus.elmasy.com`: مثلاً `staging.example.com`.

حالا Param Miner هر سابدامنه رو دو بار تست می‌کنه:

مستقیم: مثلاً به `internal.example.com` وصل می‌شه در نتیجه پاسخ `Forbidden 403` (چون مستقیم دسترسی نداریم). از طریق پراکسی: هدر `Host` رو می‌ذاره `internal.example.com` و از پراکسی درخواست می‌کنه و پاسخ `OK 200` (چون پراکسی به سیستم داخلی دسترسی داره). این تفاوت (403 در مقابل 200) نشون می‌ده که پراکسی یه مسیر مخفی به `internal.example.com` باز کرده!

این روش صدها پراکسی آسیب‌پذیر پیدا کرد که به هزاران سابدامنه دسترسی دادن، و برای همین نیاز به اتومیشن داشتیم.

دور زدن فایروال (Firewall bypass)

ساده ترین نوع اکسپلویت وقتی که می تونی هدف رو از بیرون ببینی (یعنی آدرسش رو داری)، ولی نمی تونی مستقیم بهش دسترسی پیدا کنی.

دیاگرام جریان:

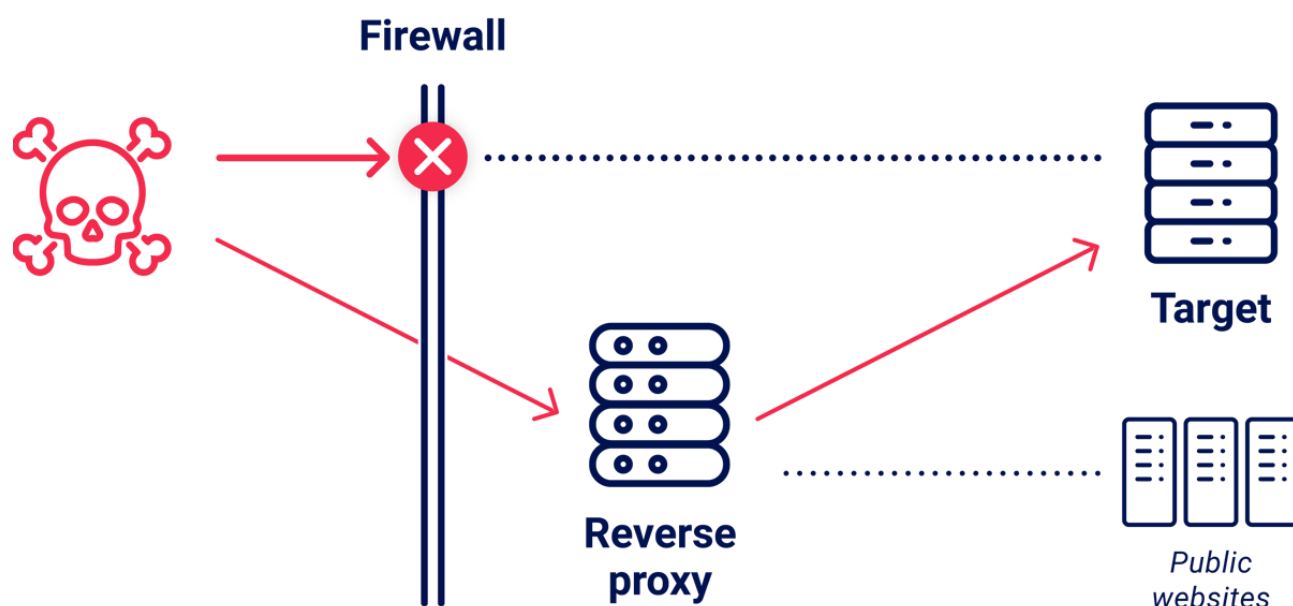
یه هکر نشون داده شده که می خواد به یه Target وصل بشه.

یه فایروال بین هکر و هدف قرار داره و جلوی دسترسی مستقیم رو می گیره.

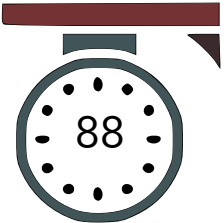
یه ریورس پراکسی هم هست که به هدف وصل می شه.

هکر می تونه از طریق ریورس پراکسی به هدف دسترسی پیدا کنه (فلش قرمز از هکر به پراکسی و بعد به هدف).

هدف به وبسایت های عمومی هم وصل، ولی این توی این سناریو مهم نیست.



توی یه شرکت، دامین `sonar.qube.redacted.com` به یه آدرس IP عمومی (Public IP) Resolve می شد (یعنی می تونستم آدرسش رو پیدا کنم)، ولی وقتی سعی کردم بهش وصل بشم، یه فایروال اتصال رو قطع کرد. آزمایش های من قبلاً نشون داده بود که `app.redacted.com` یه ریورس پراکسی هست.



با استفاده از این رپورس پراکسی، تونستم فایروال رو دور بزنم و به نمونه داخلی SonarQube (یه ابزار

برای تحلیل کد) دسترسی پیدا کنم.

جریان این جوری بود:

اگه مستقیم به sonar.qube.redacted.com وصل بشم (یعنی Entry Point و Host Header هر دو sonar.qube.redacted.com باشن). نتیجه: --reset-- (فایروال اتصال رو قطع می‌کنه).

اما اگه از طریق app.redacted.com (رپورس پراکسی) درخواست بفرستم و هدر Host رو بذارم sonar.qube.redacted.com. نتیجه: 200 OK (دسترسی موفق به SonarQube).

Entry point	Host header	Result
sonarqube.redacted.com	sonarqube.redacted.com	--reset--
app.redacted.com	sonarqube.redacted.com	200 OK

اینجا رپورس پراکسی (app.redacted.com) به من اجازه داد که به یه سیستم داخلی (SonarQube) که به صورت مستقیم به خاطر فایروال قابل دسترسی نبود، وصل بشم. این یه نمونه ساده از دور زدن فایروال با استفاده از یه رپورس پراکسی آسیب‌پذیره.

برای اینکه موضوع بهتر جا بیفته، بذار یه مثال ساده‌تر بزنم:

فرض کن یه شرکت یه سیستم داخلی داره به اسم internal.company.com که برای کارمنداشه و از بیرون نمی‌تونی بهش وصل بشی، چون فایروال جلوی دسترسی رو می‌گیره.

تو یه رپورس پراکسی آسیب‌پذیر پیدا می‌کنی روی proxy.company.com.

حالا دو روش رو تست می‌کنی:

مستقیم به internal.company.com وصل می‌شی و فایروال اتصال رو قطع می‌کنه و نمی‌تونی به سیستم دسترسی پیدا کنی.



از proxy.company.com استفاده می‌کنی و توی درخواست هدر Host رو می‌ذاری internal.company.com و پراکسی درخواست رو به سیستم داخلی فوروارد می‌کنه و تو می‌تونی به internal.company.com دسترسی پیدا کنی (مثلاً به صفحه وب بینی).

این دقیقاً همون چیزیه که توی این بخش اتفاق افتاده: sonar.qube.redacted.com به سیستم داخلی بود که فایروال جلوی دسترسی مستقیم رو می‌گرفت، ولی با استفاده از ریورس پراکسی (app.redacted.com) تونستم بهش وصل بشم.

Firewall bypass - invisible route variant

یه نوع رایج از دور زدن فایروال وجود داره که توی اون سیستم داخلی (Internal System) به رکورد DNS عمومی (Public DNS Record) مناسب نداره که بهت بگه این سیستم وجود داره.

به عبارت دیگه، نمی‌تونی با یه جستجوی ساده DNS بفهمی که همچین سیستمی هست، چون آدرسش توی DNS عمومی ثبت نشده.

یه هکر نشون داده شده که می‌خواد به یه سیستم وصل بشه.

یه فایروال بین هکر و سیستم قرار داره.

یه ریورس پراکسی هم هست که به یه هدف وصل می‌شه، ولی این هدف با یه علامت سؤال (?) نشون داده شده، یعنی هکر نمی‌دونه این سیستم وجود داره (چون رکورد DNS عمومی نداره).

هکر می‌تونه از طریق ریورس پراکسی به این هدف نامرئی دسترسی پیدا کنه (فلش قرمز از هکر به پراکسی و بعد به هدف).

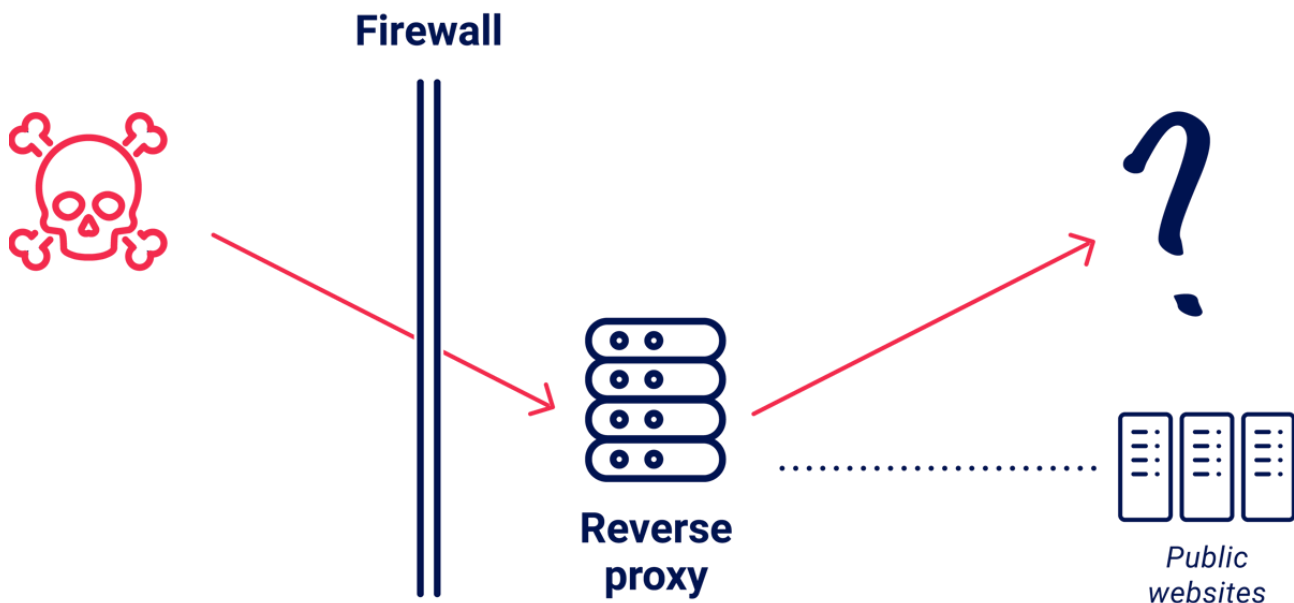
هدف به وبسایت‌های عمومی هم وصل، ولی این توی این سناریو مهم نیست.

این دیاگرام نشون می‌ده که هکر نمی‌تونه مستقیم به هدف وصل بشه، چون:

فایروال جلوی دسترسی رو می‌گیره.



هدف یه رکورد DNS عمومی نداره، برای همین هکر نمی‌دونه همچین سیستمی وجود داره (علامت سؤال). اما با استفاده از ریورس پراکسی، هکر می‌تونه به این سیستم نامرئی (Invisible) دسترسی پیدا کنه.



سیستم‌های در معرض خطر: تعداد زیادی سرور Pre-Prod، استیجینگ (Staging)، و توسعه (Development) وجود دارن که با استفاده از این تکنیک (یعنی اکسپلویت ریورس پراکسی) در دسترس قرار می‌گیرن.

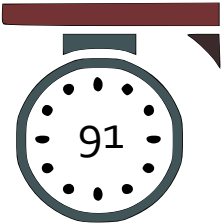
اگه شانس بیاری، این سرورها ممکنه:

دیباگینگ فعال داشته باشن (که باعث می‌شه اطلاعات بیشتری نشت کنن).

یا اطلاعات ورود آزمایشی روشن تنظیم شده باشه (مثلاً رمزهای پیش‌فرض).

این باعث می‌شه این سرورها هدف‌های آسونی باشن.

نکته مهم‌تر: این سیستم‌ها ممکنه داده‌های واقعی هدف داشته باشن، یا حتی کلیدهای استفاده شده توی محیط تولید (Production) رو دوباره استفاده کرده باشن (Reused Keys)، که خطرناکه.



جالب ترین یافته ها: جذاب ترین هدف هایی که پیدا کردم، سیستم های پیش راه اندازی (Pre-

Launch) بودن که هنوز توی مرحله توسعه فعال قرار داشتن.

به طور خاص، به کنسول ادمین پیدا کردم که به نظر می اومد دسترسی عمومی داره و روی به سیستم خیلی جالب دولتی آمریکا (دیوید دیوید او اس آ) بود.

متأسفانه نمی تونم جزئیات بیشتری درباره این سیستم بدم (چون احتمالاً حساسه).

من این مشکل رو گزارش کردم، و چند ماه بعد این سیستم لایو شد (یعنی راه اندازی شد)، ولی دیگه اثری از اون کنسول ادمین نبود (احتمالاً بعد از گزارش من، دسترسیش رو بستن).

برای اینکه موضوع بهتر جا بیفته، بذار به مثال ساده بزنم:

فرض کن به شرکت به سرور توسعه داره به اسم `dev.internal.company.com` که برای تست استفاده می شه.

این سرور توی DNS عمومی ثبت نشده، یعنی اگه توی اینترنت `dev.internal.company.com` رو جستجو کنی، هیچ آدرسی برات پیدا نمی شه (رکورد DNS عمومی نداره).

تو به ریورس پراکسی آسیب پذیر روی `proxy.company.com` پیدا می کنی.

حالا با استفاده از تکنیک های قبلی (مثل حدس زدن ساب دامنه ها)، به درخواست به پراکسی می فرستی و هدر Host رو می ذاری `dev.internal.company.com`.

پراکسی درخواست رو به `dev.internal.company.com` فوروارد می کنه، و تو به پاسخ 200 OK می گیری، یعنی تونستی به به سرور داخلی که از بیرون قابل مشاهده نبود دسترسی پیدا کنی!

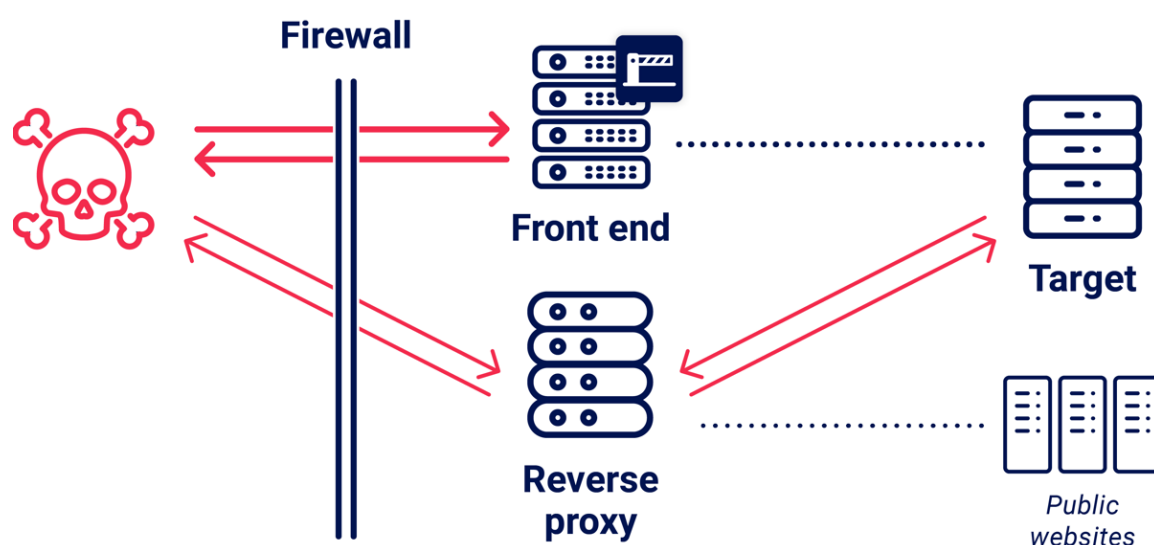
این سرور ممکنه داده های حساسی داشته باشه (مثلاً کلیدهای API یا اطلاعات تست) یا حتی دیباگینگ روش فعال باشه، که باعث می شه به هدف آسون برای حمله باشه.

این دقیقاً همون چیزیه که توی این بخش توضیح داده شده: پیدا کردن سیستم های داخلی که به صورت عمومی قابل مشاهده نیستن، ولی با ریورس پراکسی می تونی بهشون دسترسی پیدا کنی.

دور زدن قوانین فرانت اند (Front-End Rule Bypass)

بعضی از هدف‌ها به صورت عمومی قابل دسترسی هستند، ولی پشت سرورهای فرانت اند قرار دارند که قوانین امنیتی سخت گیرانه‌ای رو اعمال می‌کنن. این قوانین می‌تونن حملات رو بلاک کنن یا دسترسی به نقاط پایانی (Endpoints) مهم و باارزش رو محدود کنن (مثلاً به پنل ادمین).

روش کلاسیک برای دور زدن این قوانین اینه که مستقیم با بک‌اند صحبت کنی، ولی این کار معمولاً به خاطر فایروال‌ها غیرممکنه (چون فایروال جلوی دسترسی مستقیم به بک‌اند رو می‌گیره).



دیاگرام جریان:

یه هکر نشون داده شده که می‌خواد به یه Target وصل بشه.

یه فایروال بین هکر و هدف قرار داره.

یه سرور فرانت اند هم هست که به هدف وصل و قوانین امنیتی رو اعمال می‌کنه.

یه ریورس پراکسی هم وجود داره که به هدف وصل می‌شه.

هکر می‌تونه از طریق ریورس پراکسی به هدف دسترسی پیدا کنه (فلش قرمز از هکر به پراکسی و بعد به هدف)، و این طوری قوانین فرانت اند رو دور بزنه.

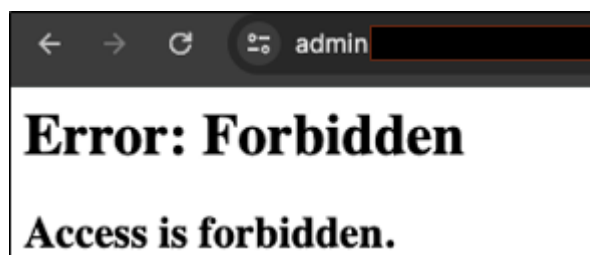
هدف به وبسایت‌های عمومی هم وصل، ولی این توی این سناریو مهم نیست.



نقش **ریورس پراکسی**: ریورس پراکسی ها به راه جایگزین جذاب ارائه می دن: به جای اینکه مستقیم با بک اند حرف بزنی، می تونی از ریورس پراکسی استفاده کنی و این مانع رو دور بزنی.

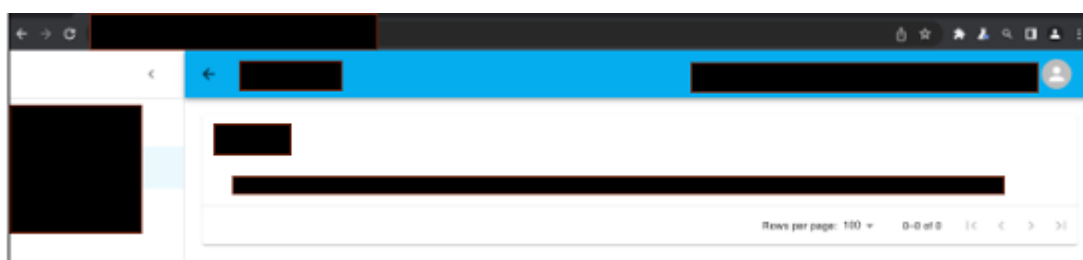
مثال عملی: توی یه هدف، با استفاده از یه مسیر جایگزین از طریق ریورس پراکسی، تونستم این موقعیت رو تغییر بدم:

وقتی مستقیم به هدف وصل می شدم، این پاسخ رو می گرفتم:



یعنی سرور فرانت اند دسترسی رو بلاک کرده بود (احتمالاً به خاطر قوانین امنیتی).

اما وقتی از ریورس پراکسی استفاده کردم (یعنی درخواستم رو از طریق پراکسی فرستادم)، به این نتیجه رسیدم:



یه پنل ادمین بهم نشون داده شد (که توی تصویر می بینیم).

این یعنی ریورس پراکسی تونست قوانین امنیتی فرانت اند رو دور بزنه و من به یه بخش حساس (پنل ادمین) دسترسی پیدا کردم.



یه توضیح اضافی برای روشن تر شدن:

برای اینکه موضوع بهتر جا بیفته، بذار یه مثال ساده بزنم:

فرض کن یه وبسایت عمومی داری به اسم `public.company.com` که یه سرور فرانت‌اند روشه و قوانین امنیتی داره. مثلاً اگه بخوای به مسیر `admin/` بری، سرور فرانت‌اند می‌گه `Forbidden` و نمی‌ذاره وارد بشی. این وبسایت به یه بک‌اند وصل که پنل ادمین واقعی رو میزبانی می‌کنه، ولی فایروال جلوی دسترسی مستقیم به بک‌اند رو می‌گیره.

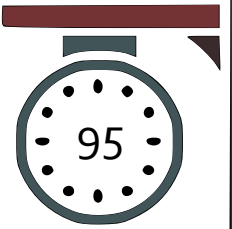
تو یه ریورس پراکسی آسیب‌پذیر پیدا می‌کنی روی `proxy.company.com`.

حالا دو روش رو تست می‌کنی:

مستقیم به `public.company.com/admin` وصل می‌شی که سرور فرانت‌اند می‌گه `Forbidden` و نمی‌تونی به پنل ادمین دسترسی پیدا کنی.

از `proxy.company.com` استفاده می‌کنی و توی درخواستت هدر `Host` رو می‌ذاری `public.company.com` و مسیر رو می‌ذاری `admin/` که پراکسی درخواست رو مستقیم به بک‌اند فوروارد می‌کنه، قوانین فرانت‌اند دور زده می‌شن، و تو به پنل ادمین دسترسی پیدا می‌کنی.

این دقیقاً همون چیزیه که توی این بخش اتفاق افتاده: هکر با ریورس پراکسی تونست قوانین امنیتی سرور فرانت‌اند رو دور بزنه و به یه پنل ادمین دسترسی پیدا کنه.



Front-End Impersonation Attacks

جذاب ترین و غافلگیر کننده ترین اکسپلویت ها وقتی اتفاق می افتد که به رابطه اعتماد (Trust Relationship) بین فرانت اند و بک اند وجود داشته باشد.

به چیز شناخته شده اینه که می تونی با هدرهایی مثل X-Forwarded-For آدرس IP خودت رو جعل (Spoof) کنی (یعنی وانمود کنی از یه IP دیگه داری درخواست می فرستی).

اما چیزی که کمتر بهش توجه شده، اینه که این فقط یه بخش از یه کلاس باگ خیلی بزرگ تر و قوی تره.

این نوع حمله اسم مشخصی نداره، برای همین من بهش می گم حمله جعل هویت فرانت اند (Front-End Impersonation Attack).

رابطه فرانت اند و بک اند: سرورهای فرانت اند معمولاً هدرهای HTTP رو به درخواست ها اضافه می کنن و بعد اونا رو به بک اند فوروارد می کنن. این هدرها اطلاعات اضافی دارن که برای بک اند مفیدن، مثل:

آدرس IP واقعی کاربر (Remote IP Address).

پروتکل اولیه درخواست (Originating Protocol).

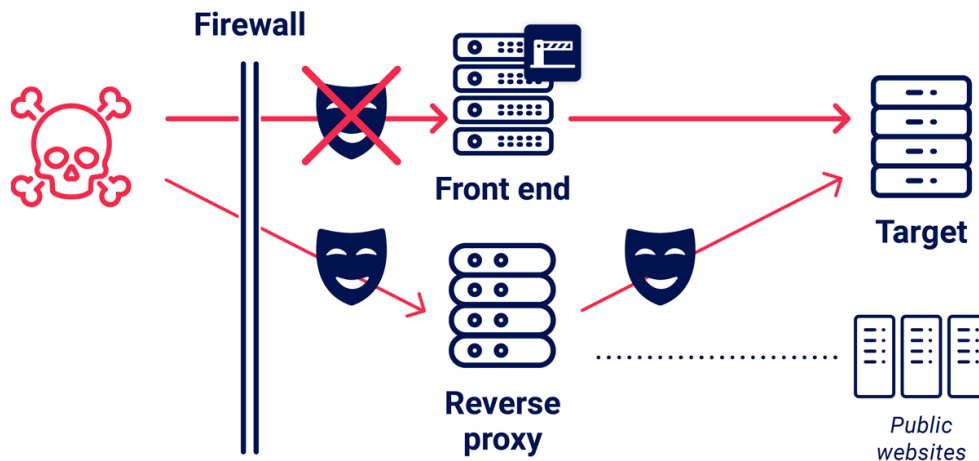
توی سیستم های پیچیده تر، گاهی از هدرهای سفارشی (Custom Headers) برای انتقال اطلاعات احراز هویت مهم استفاده می شه.

بک اند به این هدرها کاملاً اعتماد می کنه.

مشکل امنیتی: اگه یه مهاجم سعی کنه این هدرها رو جعل کنه (Spoof)، سرور فرانت اند معمولاً این هدرها رو بازنویسی می کنه.

مثلاً اگه تو هدر X-Forwarded-For بذاری 1.2.3.4، فرانت اند این هدر رو با IP واقعی تو جایگزین می کنه.

این رفتار بازنویسی هدر تنها خط دفاعی شکننده در برابر حملات جعل هویت فرانت اند هست.



دیاگرام:

یه هکر نشون داده شده که می‌خواد به یه Target وصل بشه.

یه فایروال بین هکر و هدف قرار داره و جلوی دسترسی مستقیم رو می‌گیره (با یه علامت X قرمز).

یه سرور فرانت‌اند هم هست که به هدف وصل و هدرها رو به درخواست‌ها اضافه می‌کنه.

یه ریورس پراکسی هم وجود داره که به هدف وصل می‌شه.

هکر می‌تونه از طریق ریورس پراکسی به هدف دسترسی پیدا کنه (فلش قرمز از هکر به پراکسی و بعد به هدف)، و این‌طوری هدرهای جعل شده رو مستقیم به بک‌اند بفرسته بدون اینکه فرانت‌اند اونا رو بازنویسی کنه.

راه‌های دور زدن این دفاع

صحبت مستقیم با بک‌اند: ساده‌ترین راه اینه که مستقیم با بک‌اند حرف بزنی (تا فرانت‌اند هدرها رو بازنویسی نکنه)، ولی این معمولاً به خاطر فایروال‌های شبکه غیرممکنه.

HTTP Request Tunneling: من از این روش برای نفوذ کامل به API داخلی New Relic استفاده کردم. توی اون مورد، از یه هدر به اسم Service-Gateway-Is-Newrelic-Admin استفاده کردم که نشون می‌داد درخواست از یه ادمین New Relic هست. با تونل کردن، تونستم این هدر رو جعل کنم و به API دسترسی پیدا کنم.



توی مورد New Relic، من یه هدر سفارشی به اسم Service-Gateway-Is-Newrelic-Admin پیدا کردم که برای احراز هویت ادمین‌ها استفاده می‌شد. این هدر به بک‌اند می‌گفت که درخواست از یه ادمین New Relic اومده و باید دسترسی کامل به API داخلی داده بشه. با تونل کردن درخواست از یه مسیر غیرمستقیم، تونستم این هدر رو جعل کنم و به بک‌اند بفرستم، بدون اینکه فرانت‌اند بتونه هدر رو بازنویسی کنه. این کار بهم اجازه داد به API داخلی New Relic دسترسی پیدا کنم و اطلاعات حساسی رو ببینم که فقط برای ادمین‌ها بود.

مبهم‌سازی هدرها (Obfuscating Headers): می‌تونن هدرها رو طوری تغییر بدی (مثلاً با تغییر فرمت یا نام) که از فرانت‌اند رد بشن (Smuggle).

استفاده از رپورس **پراکسی‌های اشتباه پیکربندی‌شده**: رپورس پراکسی‌ها یه راه جذاب برای دور زدن دفاع بازنویسی هدرها و انجام حملات جعل هویت فرانت‌اند ارائه می‌دن.

چطور این کار رو انجام بدیم

برای امتحان کردن این روش:

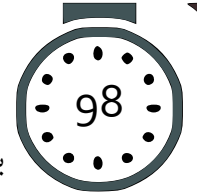
از اسکن Detect scoped SSRF توی Param Miner استفاده کن تا یه رپورس پراکسی پیدا کنی.

بعد Exploit scoped SSRF رو اجرا کن تا مسیرهای جایگزین به سیستم‌های داخلی پیدا کنی.

توی هر مسیر جایگزین، Guess headers رو اجرا کن تا هدرهای مفیدی که می‌تونن جعل کنی رو پیدا کنی.

برای موفقیت توی این روش، باید بتونی یه تصویر ذهنی قوی از اتفاقاتی که پشت صحنه می‌افته داشته باشی.

برای کمک، من یه چالش CTF توی سایت listentothewhispers.net درست کردم برو ببین می‌تونن حلش کنن!



برای اینکه موضوع بهتر جا بیفتد، بذار یه مثال ساده بزنم:

فرض کن یه وبسایت داری به اسم `public.company.com` که یه سرور فرانت‌اند روشه. این فرانت‌اند یه هدر به اسم `X-Is-Admin: false` به همه درخواست‌ها اضافه می‌کنه و بعد درخواست رو به بک‌اند می‌فرسته.

بک‌اند به این هدر اعتماد داره: اگه `X-Is-Admin: true` باشه، بهت دسترسی ادمین می‌ده (مثلاً به یه پنل ادمین)، و اگه `false` باشه، فقط دسترسی معمولی می‌ده.

تو به عنوان هکر می‌خواهی وانمود کنی که ادمین هستی، برای همین توی درخواست هدر `X-Is-Admin: true` رو می‌ذاری.

اما وقتی درخواستت از فرانت‌اند می‌گذره، فرانت‌اند این هدر رو بازنویسی می‌کنه و می‌ذاره `X-Is-Admin: false`، برای همین نمی‌تونی به پنل ادمین دسترسی پیدا کنی.

حالا یه ریورس پراکسی `proxy.company.com` پیدا می‌کنی.

از طریق پراکسی درخواستت رو می‌فرستی و هدر `X-Is-Admin: true` رو می‌ذاری. چون پراکسی مستقیم با بک‌اند حرف می‌زنه (و از فرانت‌اند رد نمی‌شه)، هدرت بازنویسی نمی‌شه و بک‌اند فکر می‌کنه تو ادمینی و بهت دسترسی می‌ده!

این دقیقاً همون چیزیه که توی این بخش توضیح داده شده: با ریورس پراکسی می‌تونی هدرهای جعل شده رو به بک‌اند برسونی و وانمود کنی که یه کاربر مورد اعتماد (مثلاً ادمین) هستی.



زنجیره کردن (Chaining)

در نهایت، SSRF محدود (Scoped SSRF) از طریق ریورس پراکسی ها فرصت های خوبی برای زنجیره کردن اکسپلویت ها (Exploit Chaining) فراهم می کند.

تبدیل Scoped SSRF به Full SSRF: اگر بتونی یه سابدامنه از شرکت هدف رو تحت کنترل بگیری و رکورد DNS اون سابدامنه رو به یه آدرس IP دلخواه (Arbitrary IP Address) اشاره بدی، می تونی یه Scoped SSRF رو به یه Full SSRF ارتقا بدی و به هر آدرس IP که بخوای حمله کنی.

این کار خیلی شبیه زنجیره کردن یه SSRF سنتی با یه Open Redirect (ریدایرکت باز) هست:

توی Open Redirect، یه SSRF رو با یه ریدایرکت ترکیب می کنی تا به یه آدرس دلخواه برسی.

اینجا هم با کنترل یه سابدامنه و تغییر DNS، می تونی Scoped SSRF رو به Full SSRF تبدیل کنی.

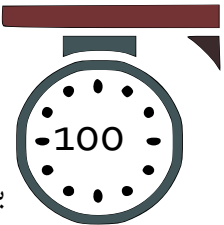
مثلاً: اگر سابدامنه sub.target.com رو کنترل کنی و DNSش رو به IP سرور خودت (مثلاً 1.2.3.4) اشاره بدی، ریورس پراکسی درخواست ها رو به اون IP فوروارد می کنه و تو می تونی به هر سروری که بخوای وصل بشی (Full SSRF).

HTTP Request Smuggling با ریورس پراکسی: از اونجایی که ریورس پراکسی ها بهت اجازه می دن بک اند رو خودت انتخاب کنی، برای حمله HTTP Request Smuggling خیلی مناسبن.

من وقت کافی برای بررسی کامل این موضوع نداشتم، ولی به طور خلاصه:

فکر می کنم پیدا کردن بک اند هایی که به Request Smuggling آسیب پذیر باشن باید آسون باشه.

اما سوءاستفاده بین کاربر ها (Cross-User Exploitation — یعنی حمله ای که از یک کاربر به کاربر دیگه سرایت کنه) معمولاً غیرممکنه، چون هیچ کاربر قانونی دیگه ای از اون اتصال فرانت اند/بک اند استفاده نمی کنه یعنی اتصالات کاملاً ایزوله ست.



برای اینکه بتونی تأثیر این حمله رو نشون بدی، باید سراغ اکسپلویت‌های مبتنی بر تونل کردن بری، مثل:

جعل هویت فرانت‌اند (Front-End Impersonation): که توی بخش قبلی توضیح داده شد.

افشای هدر (Header Disclosure): یعنی هدرهایی که اطلاعات حساسی (مثلاً توکن‌های احراز هویت) دارن رو به دست بیاری.

برای اینکه موضوع بهتر جا بیفته، بذار یه مثال ساده بزنم

تبدیل Scoped SSRF به Full SSRF: فرض کن یه ریورس پراکسی روی `proxy.target.com` پیدا کردی که Scoped SSRF داره، یعنی فقط به سابدامنه‌های `*.target.com` اجازه می‌ده درخواست‌ها رو فوروارد کنه. تو یه سابدامنه مثل `sub.target.com` رو تحت کنترل می‌گیری (مثلاً با یه آسیب‌پذیری DNS یا Social Engineering).

رکورد DNS این سابدامنه رو به IP سرور خودت (مثلاً 1.2.3.4) اشاره می‌دی.

حالا یه درخواست به پراکسی می‌فرستی و هدر Host رو می‌ذاری `sub.target.com`.

پراکسی درخواست رو به `sub.target.com` فوروارد می‌کنه، که حالا به IP سرور تو (1.2.3.4) اشاره می‌کنه.

این‌طوری Scoped SSRF به Full SSRF تبدیل می‌شه، چون می‌تونی به هر IP که بخوای وصل بشی (مثلاً سرورهای داخلی یا حتی سرورهای خارجی).

HTTP Request Smuggling: فرض کن ریورس پراکسی بهت اجازه می‌ده بک‌اند رو انتخاب کنی (مثلاً با هدر Host).

تو یه درخواست Smuggling می‌فرستی (مثلاً با ترکیب هدرهای Content-Length و Transfer-Encoding به صورت مبهم).

اگه بک‌اند به Request Smuggling آسیب‌پذیر باشه، می‌تونی درخواست‌های جعلی رو به بک‌اند تزریق کنی.

اما مشکل اینه که این نوع حمله معمولاً برای تأثیر روی کاربران دیگه (Cross-User) کار نمی‌کنه، چون اتصال تو با بک‌اند ایزوله‌ست و کاربرهای دیگه از همون اتصال استفاده نمی‌کنن.

برای همین، باید سراغ حملات دیگه بری، مثل جعل هویت فرانت‌اند (مثلاً جعل هدرهای احراز هویت) یا افشای هدرهای حساس.

قدم بعدی چیه؟

برنامه من: هدف من از این تحقیق اینه که مردم رو به استفاده روزمره از حملات زمان‌بندی (Timing Attacks) تشویق کنم. برای همین، برنامه دارم که ماه آینده رو صرف بهبود ابزارهای Param Miner و Turbo Intruder کنم.

به‌طور خاص، فکر می‌کنم می‌تونم بیشتر حملات زمان‌بندی رو خیلی سریع‌تر کنم، فقط با استفاده از t-test برای تصمیم‌گیری:

آیا یه کشف (Discovery) رو گزارش کنم؟

یا کار رو متوقف کنم (Bail)؟

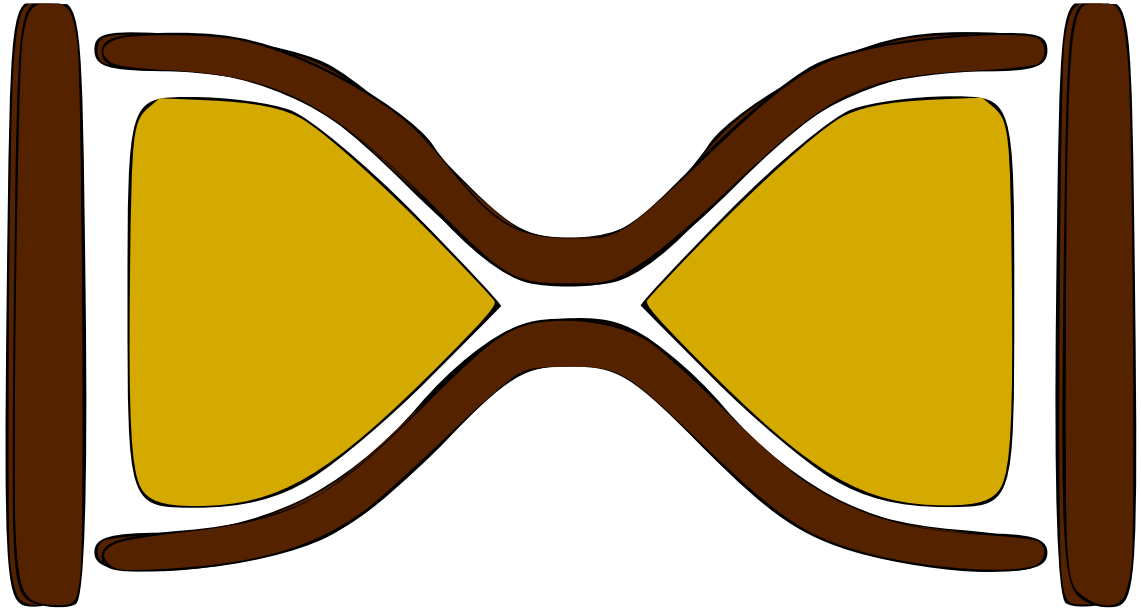
یا نمونه‌های بیشتری (Samples) جمع کنم؟

همچنین دنبال بازخورد کاربران هستم اگه درخواست یا نظری دارید، از طریق GitHub یا ایمیل بهم بگید.

آزمون t (t-test) یعنی یک روش آماری که بهت کمک می‌کنه تصمیم‌گیری **آیا تفاوت زمان پاسخ بین دو حالت (مثلاً درخواست درست و درخواست نادرست) واقعاً معنی‌داره یا فقط تصادفیه.**

فرض کن داری زمان پاسخ سرور رو برای دو نوع درخواست مختلف اندازه می‌گیری. گاهی این زمان‌ها کمی کم و زیاد می‌شن. آزمون t بهت می‌گه که: این اختلاف زمان، واقعاً به خاطر باگ هست یا فقط نویز و شانس؟

اینجا چه کاربردی داره؟ تو حملات زمان‌بندی، مهاجم باید خیلی سریع تصمیم بگیره که آیا این تفاوت کوچک توی زمان، نشونه وجود باگ هست یا نه؟ به جای اینکه کلی نمونه بگیره و میانگین بگیره، می‌تونه با آزمون t روی همون تعداد کم درخواست، سریع تصمیم بگیره و حمله رو ده‌ها برابر سریع‌تر کنه.



SECTION

5

ادامہ

مسیر

نقشه راه تحقیقات زمان‌بندی (Timing research roadmap)

این یافته‌ها فقط سطح موضوع رو خراش دادن (یعنی خیلی ابتدایی هستن)، و حملات زمان‌بندی هنوز پتانسیل خیلی زیادی برای تحقیقات بیشتر دارن. اگه علاقه داری ببینی این کلاس حمله قراره به کجا برسه، یا خودت بخوای بیشتر روش کار کنی، مسیرهای مختلفی برای بررسی وجود داره:

جستجو برای کاربردهای جدید حملات زمان‌بندی:

به نظرم باارزش‌ترین حوزه اینه که کاربردهای جدید برای حملات زمان‌بندی پیدا کنیم.

این کار نسبتاً آسونه و نیاز به تعهد زمانی زیادی برای شروع نداره.

خطر اصلی اینجاست که ممکنه به‌طور تصادفی روی مفهومی کار کنی که سیگنال موردنیازت توی نویز گم بشه (یعنی تأخیر موردنظر قابل تشخیص نباشه).

خوشبختانه، این مشکل رو می‌تونی به راحتی حل کنی:

اول فکر کن که دلیل تأخیر چیه؟ آیا از یه تعامل اضافی با یه سیستم ریموت، سیستم LAN، هارد دیسک، رم، یا رجیستر CPU میاد؟

وقتی توی سطح درست کار می‌کنی، یه معیار لوکال (Local Benchmark) بساز تا اندازه سیگنالی که باید تشخیص بدی رو اندازه‌گیری کنی.

اگه سیگنال خیلی کوچیکه، تکنیک‌های تقویت رو بررسی کن:

یادت باشه که بیشتر حملات (DoS (Denial of Service در واقع یه جور حمله زمان‌بندی هستن، پس ازشون استقبال کن.

شاید بتونی تأخیر رو با روش‌هایی مثل موجودیت‌های تودرتو (XML (Nested XML Entities، ReDoS (Regular Expression Denial of Service، یا HashTable Collisions بیشتر کنی.

تکنیک‌های کاهش لرزش (Jitter-Reduction):

این تکنیک‌ها خیلی باارزشن و به‌طور گسترده نادیده گرفته شدن ممکنه تکنیک‌های خوبی توی این حوزه منتظر تحقیق باشن. (لرزش یا Jitter به تغییرات تصادفی توی زمان‌بندی اشاره داره که می‌تونه تشخیص سیگنال رو سخت کنه.)

بهبودهای کلی در سطح تکنیک:

یه سری بهبودهای کلی توی سطح تکنیک‌ها هم ممکنه:

شاید حمله Single-Packet اگه توی لایه TCP تکه‌تکه بشه بهتر کار کنه.

یا شاید بهتر باشه به‌جای دو درخواست، ده درخواست رو توی یه Packet بفرستی؟

آینده حملات زمان‌بندی

بیا یه کم تخیل کنیم و بینیم 5-10 سال دیگه حملات تایمینگ کجا می‌رن. من چندتا حدس دارم که می‌تونه مسیر رو نشون بده. اولین حدس، ترکیب با هوش مصنوعیه. فرض کن یه مدل AI بسازی که بتونه Jitter رو پیش‌بینی کنه مثلاً 1000 تا پاسخ از یه سرور بگیره و AI بگه کدوم تأخیر به خاطر شبکه‌ست و کدوم به خاطر پردازش یه پارامتر. اینجوری حتی توی شبکه‌های شلوغ هم می‌تونی سیگنال $\mu s200$ رو جدا کنی و دقتت بره بالا. مثلاً یه مدل می‌تونه بگه: این $\mu s250$ واقعیه، اون $ms50$ نویزه!

دومین ایده، محاسبات کوانتومیه. اگه کوانتوم بیاد، زمان پردازش توابع انقدر کم می‌شه که شاید بتونیم تأخیرهای نانوثانیه رو هم بگیریم. مثلاً یه تابع مقایسه کاراکتر به کاراکتر که الان $ns5$ طول می‌کشه، توی کوانتوم بشه $ns0.1$. این برای مهاجم‌ها عالیه، ولی دفاع‌ها هم عوض می‌شن شاید توابع زمان ثابت بی‌اثر بشن و دفاع‌های جدید مثل تأخیر مصنوعی باب بشه.

سومین احتمال، حملات ترکیبیه. فکر کن تایمینگ رو با یه Zero-Day یا Side-Channel ترکیب کنی. مثلاً یه باگ توی پردازش JSON پیدا کنی که تأخیر می‌ده بعد با تایمینگ بفهمی کدوم ورودی درسته و یه Blind injection بزنی. یا با Request Smuggling یه پراکسی رو دور بزنی و با تایمینگ مسیرهای مخفی رو پیدا کنی. اینجوری یه سرور رو کامل می‌تونی باز کنی!

دفاع‌ها هم بیکار نمی‌مونن شاید سرورها تأخیر پویا بذارن که با هر درخواست عوض بشه، یا WAFها بتونن حملات Single-Packet رو تشخیص بدن. این یه مسابقه تسلیحاتیه که تازه داره داغ می‌شه!

توصیه نهایی:

هر مسیری که انتخاب می‌کنی، سعی کن از تمرکز بیش از حد روی یک هدف خاص دوری کنی.

تکنیک‌های عمومی و قابل استفاده مجدد خیلی بیشتر به توسعه این حوزه کمک می‌کنن.

دفاع (Defence)

دفاع در برابر حملات زمان‌بندی (Timing Attacks) سخته. در درجه اول، توسعه‌دهنده‌ها باید بفهمن که دید مهاجم به عملکرد داخلی سیستم‌شون فراتر از محتوای پاسخ هست.

بیشتر از حد تصور کن: امن‌ترین راه اینه که توانایی‌های مهاجم رو بیش از حد تخمین بزنی. فرض کن یک مهاجم می‌تونه هر خط کدی که اجرا می‌شه رو بخونه.

این شبیه اینه که کدت Open Source باشه، ولی یک کم جدی‌تره، چون داده‌های زنده روی جریان اجرا تأثیر می‌ذارن.

مهاجم‌ها نمی‌تونن مستقیم به متغیرها دسترسی پیدا کنن، ولی می‌تونن بین کدوم شاخه‌ها توی کد اجرا می‌شن و هر حلقه چند بار تکرار می‌شه.

بهینه‌سازی‌های عملکرد و سیگنال‌های بزرگ: خیلی مهمه که این موضوع رو موقع پیاده‌سازی بهینه‌سازی‌های عملکرد مثل کش کردن (Caching) در نظر بگیری، چون این بهینه‌سازی‌ها معمولاً سیگنال‌های خیلی بزرگی (Massive Signal) ایجاد می‌کنن که برای حملات زمان‌بندی قابل استفاده‌ست.

مثلاً توی بخش‌های قبلی دیدیم که کش کردن DNS باعث شد یک تأخیر 5 میلی‌ثانیه‌ای قابل تشخیص بشه و Scoped SSRF رو پیدا کنیم.

کاهش سیگنال‌های کوچک: برای کاهش حملاتی که از سیگنال‌های کوچک‌تر سوءاستفاده می‌کنن، می‌تونی حمله Single-Packet رو با اعمال یک ریت‌لیمیت بشکنی:

هر آدرس IP رو به یک درخواست در هر 1 تا 5 میلی‌ثانیه محدود کن.

این باعث می‌شه مهاجم نتونه درخواست‌های زیادی توی یک زمان کوتاه بفرسته و تأخیرها رو اندازه‌گیری کنه.

برای فروشندگان WAF: اگر فروشنده WAF هستی، می‌تونی تشخیص بدی که یه Packet شامل چند درخواست HTTP هست یا نه. اگر چند درخواست توی یه Packet بود، اونا رو به Packet های جداگانه بشکن و بین هر کدوم یه تأخیر خیلی کم بذار.

این کار باعث می‌شه حملات زمان‌بندی که به درخواست‌های همزمان وابسته هستن سخت‌تر بشن.

استفاده از توابع زمان ثابت: بله، من توصیه می‌کنم موقع مقایسه ورودی کاربر با کلیدهای مخفی از توابع زمان ثابت استفاده کنی. ولی اگر کسی گفت این یه تهدید واقعیه، ازش بخواه یه POC ارائه بده (یعنی عملاً نشون بده که می‌تونه این حمله رو انجام بده).



نکات کلیدی (Takeaways)

موضوع فقط اکسپلویت‌ها نیست. توی هسته‌شون، حملات زمان‌بندی وب درباره پاسخ دادن به سؤال‌های سخت هستن.

با حمله (Single-Packet Attack)، حملات زمان‌بندی وب حالا لوکال، Portable، و شدنی (Feasible) شدن.

اوراکل‌های زمان‌بندی (Timing Oracles) همه‌جا هستن. هر چیزی که تست می‌کنی، زمزمه‌های زمان‌بندی (Timing Murmurs) همیشه وجود دارن و منتظرن که بهشون گوش بدی.